Infinite Automata

Winter term 2025

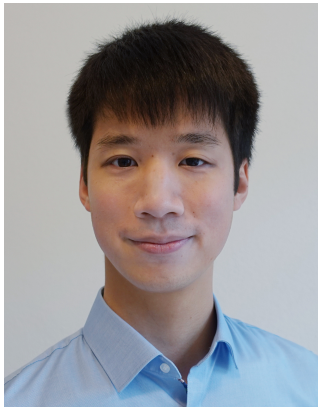
Rheinland-Pfälzische Technische Universität Kaiserslautern Landau

MAX PLANCK INSTITUTE FOR SOFTWARE SYSTEMS

Team for Infinite Automata 2025

Lectures

Georg Zetzsche
georg@mpi-sws.org



Moses Ganardi
moses.ganardi@cs.rptu.de

Tutorials

Marin Ricros
mricros@mpi-sws.org

Organization

Lecture
- Mondays 14.15–15.45, Tuesdays 14.15–15.45.
- Room: 26-111 (MPI-SWS).
- Slides will be made available.

Tutorials
- Tutorial sessions held by Marin Ricros
- We will put up a poll for the scheduling of tutorials.
- Please make your choice until Tuesday (28 Oct).

Problem sets
- New sheet on the day of the tutorial.
- To be turned in before the next tutorial.
- We will create groups in OpenOLAT. Students can pick groups via the forum.
- You can take the exam if you score $\geq 50\%$.

OpenOLAT
- `https://shlink.mpi-sws.org/infaut2025`

Questions?

Part I

Introduction

Background: Why infinite automata?

What we would like

For correctness property φ, decide:

Given Program P

Question Does P satisfy φ?

Unfortunately, this is not possible:

Theorem (Rice's Theorem)

Every non-trivial property of Turing machines is undecidable.

General Theme

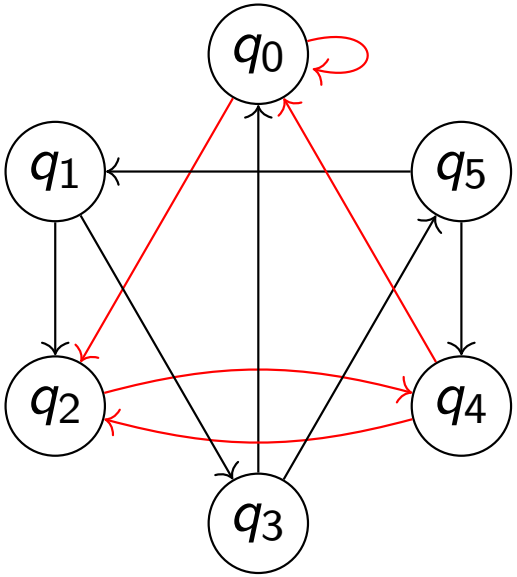
1. Find classes of automata that we can analyze.
2. Abstract programs by automata in such classes.

This requires automata with infinitely many states!

Background: Why infinite automata?

$i \leftarrow 0$
while $i + 3$ not divisible by 6 **do**
  if rand() is even **then**
    $i \leftarrow 2 \cdot i$
  else
    $i \leftarrow i + 2$

$\rightsquigarrow$



Observation

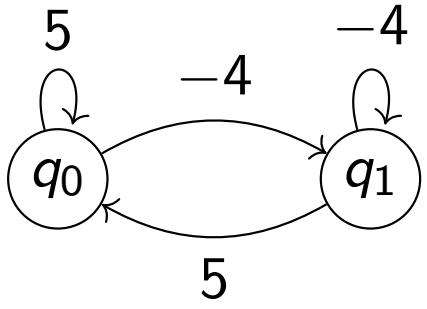
- Abstracted program with finite-state automaton
- Does the program terminate?
- 3 is not reachable from 0 $\rightarrow$ program runs forever
- Finite-state abstraction shows non-termination

Background: Why infinite automata?

Sometimes finitely many states are not enough:

$i \leftarrow 1$
$j \leftarrow 0$
while $i \neq 7$ **do**
  $j \leftarrow j + \text{rand}()$
  if j is even **then**
    $i \leftarrow i + 5$
  else
    $i \leftarrow i - 4$

$\rightsquigarrow$



Observation

- Program abstracted by a counter automaton $\rightarrow$ infinitely many configurations!
- We can algorithmically decide whether $(q_0, 7)$ or $(q_1, 7)$ is reachable.

Background: Why infinite automata?

Instead of one counter, what else can we add without losing decidability?

- What if the counter can be tested for zero?
- What if one can store strings?
- What if one can store numbers and strings?

Research program since $\approx$ 40–50 years

- Identify automata models where "things" are decidable
- Trade-off between expressiveness and efficiency

In this course, you will get to know:

- The most prominent kinds of such *infinite-state systems*
- How to decide properties for them
- How expressive they are

A quick overview

We will consider the following infinite state systems:

1. Vector addition systems with states (VASS)
   - Finite automata with counters
   - Model the behavior of (concurrent) systems
   - There are multiple semantics and extensions:
     - Acceptance by reaching a state or a configuration
     - Additional transitions like resets or zero-tests
     - Counters in $\mathbb{N}$ or $\mathbb{Z}$
2. Channel Systems
   - Finite automata with queues
   - Model the communication between network instances
   - We consider reliable and lossy channel systems
3. Pushdown Systems
   - Finite automata with pushdowns
   - Model the function call stack of a system
4. Well-structured transition systems
   - Abstraction of several systems with nice properties

A quick overview

We consider multiple verification problems like:

- Reachability Problem
  - For a given system S and two configurations c, d of S, does $c \to^* d$ hold?
  - Motivation: Check whether a given system can enter a failure state by applying only valid computational steps?
- Coverability Problem
  - For a given system S with an ordering $\geq$ on its configurations and for two configurations c, d of S, is there a configuration $d' \geq d$ with $c \to^* d'$?
  - Motivation: This is an approximation of the reachability problem.
- Recurrent Reachability Problem
  - For a given system S, a configuration c, and a state s, is there an infinite run of S starting in c which visits s infinitely often?
  - Motivation: This problem is equivalent to the model checking problem of (e.g.) linear temporal logic
- Termination Problem
  - For a given system S, is there an infinite run in S?
  - Motivation: Check whether some algorithm eventually halts.

Part II

Simple undecidable infinite automata

A *d-counter automaton* is a tuple (Q, T, q_0), where

- Q is a finite set of *(control) states*
- $T \subseteq Q \times (\mathbb{Z} \cup \{\mathsf{z}\})^d \times Q$
- $q_0 \in Q$ is its *initial state*

A *configuration* is a pair $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$.
Intuitively, in a transition $(q, \mathbf{v}, q')$, if $\mathbf{v}[i] \in \mathbb{Z}$, then we add $\mathbf{v}[i]$ to counter i. If $\mathbf{v}[i] = \mathsf{z}$, then we check if counter i is zero.
Formally, we write $(q, \mathbf{u}) \to (q', \mathbf{u}')$ for configurations $(q, \mathbf{u}), (q', \mathbf{u}')$ if there is a $(q, \mathbf{v}, q') \in T$ with, where for each $i \in \{1, \ldots, d\}$:

- If $\mathbf{v}[i] \in \mathbb{Z}$: $\mathbf{u}'[i] = \mathbf{u}[i] + \mathbf{v}[i]$
- If $\mathbf{v}[i] = \mathsf{z}$, then $\mathbf{u}[i] = \mathbf{u}'[i] = 0$.

We write $\to^*$ for the reflexive transitive closure of $\to$ on the set of configurations.

State reachability

Given A d-counter automaton $V = (Q, T, q_0)$ and $q \in Q$.

Question Is there a $\mathbf{u} \in \mathbb{N}^d$ with $(q_0, \mathbf{0}) \to^* (q, \mathbf{u})$?

Very simple types of infinite automata can be undecidable:

Theorem (Minsky 1961)

The state reachability problem for 2-counter machines is undecidable.

In the lecture: Proof for 3-counter machines.
Exercises: 2-counter machines.
The proof consists of two steps:

1. Undecidability in automata with two pushdowns.
2. Undecidability in 3-counter machines.

A *2-pushdown automaton (2-PDA)* is a tuple (Q, Γ, T, q_0), where

- Q is a finite set of *(control) states*
- Γ is a stack alphabet
- $T \subseteq Q \times \{\gamma_i, \bar{\gamma}_i, \varepsilon \mid i \in \{1, 2\}\} \times Q$
- $q_0 \in Q$ is its *initial state*

A *configuration* is a triple $(q, u, v) \in Q \times \Gamma^* \times \Gamma^*$.

We write $(q, u_1, u_2) \to (q', u_1', u_2')$ if there is a $(q, w, q') \in T$ with:

- if $w = \gamma_i$, then append γ to u_i: $u_i' = u_i\gamma$
- if $w = \bar{\gamma}_i$, then remove γ from u_i: $u_i = u_i'\gamma$

We write $(q, u_1, u_2) \to^* (q', u_1', u_2')$ if there is a sequence

$$(q_0, u_1^{(0)}, u_2^{(0)}) \to (q_1, u_1^{(1)}, u_2^{(1)}) \to \cdots \to (q_n, u_1^{(n)}, u_2^{(n)})$$

with $(q_0, u_1^{(0)}, u_2^{(0)}) = (q, u_1, u_2)$ and $(q_n, u_1^{(n)}, u_2^{(n)}) = (q', u_1', u_2')$.

State reachability

Given A 2-PDA $V = (Q, \Gamma, T, q_0)$ and $q \in Q$.

Question Is there a $(u_1, u_2) \in \Gamma^* \times \Gamma^*$ with $(q_0, \varepsilon, \varepsilon) \to^* (q, u_1, u_2)$?

Step I in our undecidability proof for 3-counter machines:

Theorem

State reachability is undecidable for 2-PDA.

Proof:

"Slinky trick"

Turing machines configuration $uqv \quad (u, v \in \Gamma^*)$

$$\rightsquigarrow$$

2-PDA configuration (q, u, v^{rev})

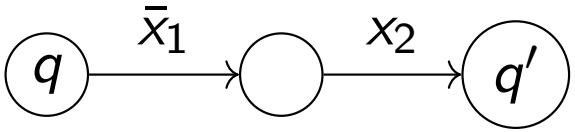
For a detailed proof, we use the following slight variant of Turing machines.
They have a tape alphabet Γ and a configuration is of the form uqv, where $u \in \Gamma^*$, $v \in \Gamma^*$ are tape contents, and $q \in Q$ is the current state.
In such configuration uqv, the read/write head of the TM always reads the last letter of u.
The Turing machine is specified by transitions of the following form:

(q, x, left, q')	if x is read, move to the left
(q, x, right, q')	if x is read, move to the right
$(q, x, \text{replace}(y), q')$	if x is read, replace by y
$(q, x, \text{insert}(y), q')$	if x is read, insert y

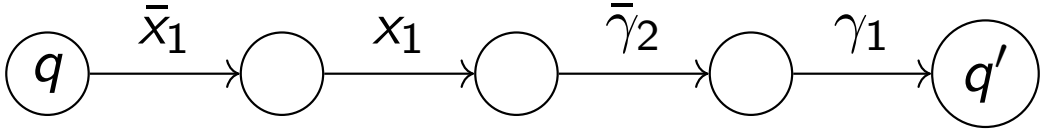
Translating Turing machines into 2-PDA

For each transition (left), we include the gadget(s) on the right:

(q, x, left, q')

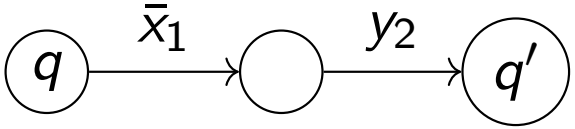


(q, x, right, q')

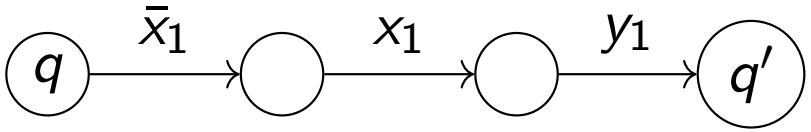


for every $\gamma \in \Gamma$

$(q, x, \text{replace}(y), q')$



$(q, x, \text{insert}(y), q')$



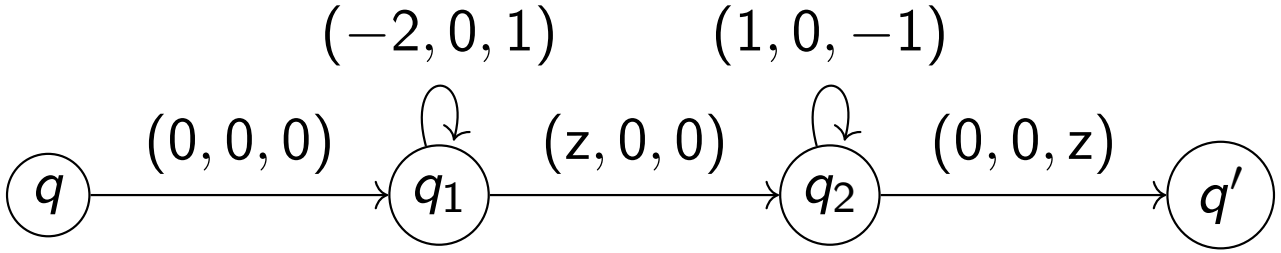
Step II in our undecidability proof for 3-counter machines:
Translate 2-PDA into 3-counter machines.
Suppose we have 2-PDA with $\Gamma = \{0, 1\}$, configuration (q, u, v).
Write $u = u_1 \cdots u_k,\ v = v_1 \cdots v_\ell$, with $u_i, v_j \in \{0, 1\}$.
Idea: Encode stack contents using binary encodings. Define:

$$a = 2^k + 2^{k-1} \cdot u_1 + 2^{k-2} \cdot u_2 + \cdots + 2^0 \cdot u_k$$
$$b = 2^\ell + 2^{\ell-1} \cdot v_1 + 2^{\ell-2} \cdot v_2 + \cdots + 2^0 \cdot v_\ell$$

Here, the term 2^k and 2^ℓ guarantee that the encoding is injective: If we did not include this extra digit, then e.g. the words $u = 001$ and $v = 01$ would both be encoded by the numbers $a = 1$ and $b = 1$. But with the encoding above, we get $a = 2^2 + 2^0 = 5$ and $b = 2^1 + 2^0 = 3$.

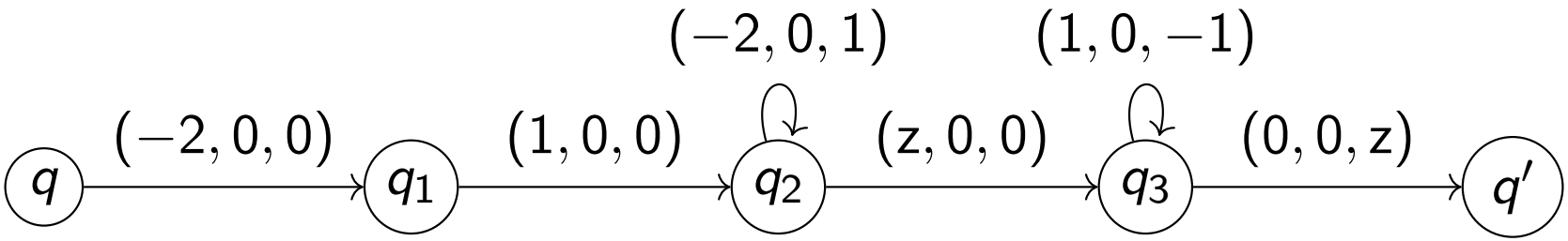
How do we implement popping 0 with this encoding?
We need to divide the counter value by 2!
We do this by transferring counter values to the third counter and back. In the process, we can double the value.

To implement a $\bar{0}_1$ (i.e. pop 0 on first stack), we use:



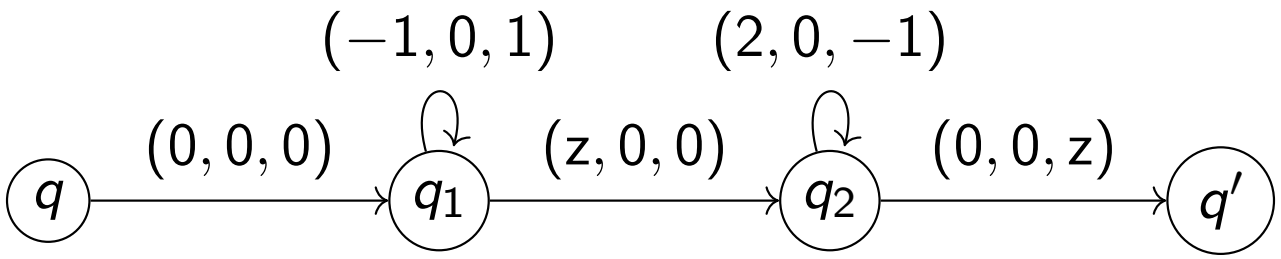
In the loop q_1 loop, we subtract 2 from the first counter and add 1 to the third. The loop runs until the first counter is zero (transition to q_2). This means, the third counter now contains the original value of the first counter, divided by 2. In the q_2-loop we then move the content of the third counter back to the second counter, until the third counter is zero (transition to q_3).
Note that this only succeeds if the original value of the first counter is even. In particular, the word on the stack must be non-empty and end in 0.

To implement a $\bar{1}_1$ (i.e. pop 1 on first stack), we need to first subtract 1 and then divide by 2:

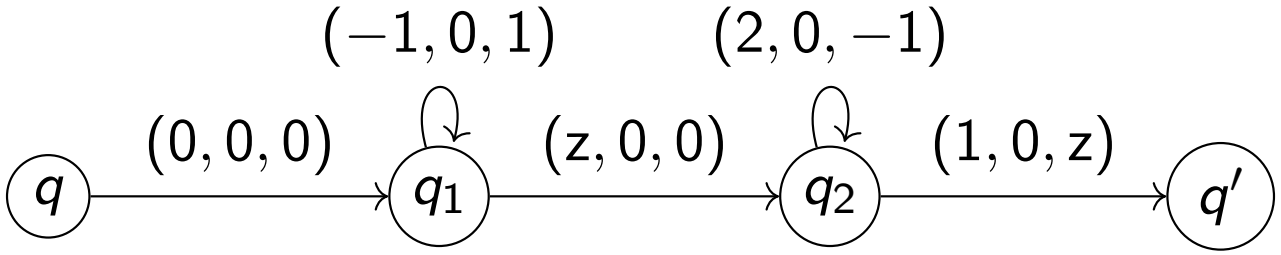


This works like popping 0, except that we now need to (i) subtract 1 before dividing by 2 and (ii) make sure the initially, the counter value was not exactly 1. If we did not check (ii), it could be that the counter value is 1, meaning the stack is empty. Therefore, we first subtract 2 (to ensure the value is ≥ 2), and then add 1. Thus, when we arrive in q_2, we know the counter did encode a string that ends in 1.

To implement a 0_1 (i.e. push 0 on first stack), we double the counter value:



To implement a 1_1 (i.e. push 1 on first stack), we double the counter value, and then add 1:



The second stack works completely analogously: just use the second counter instead of the first.

Part III

Well-structured transition systems

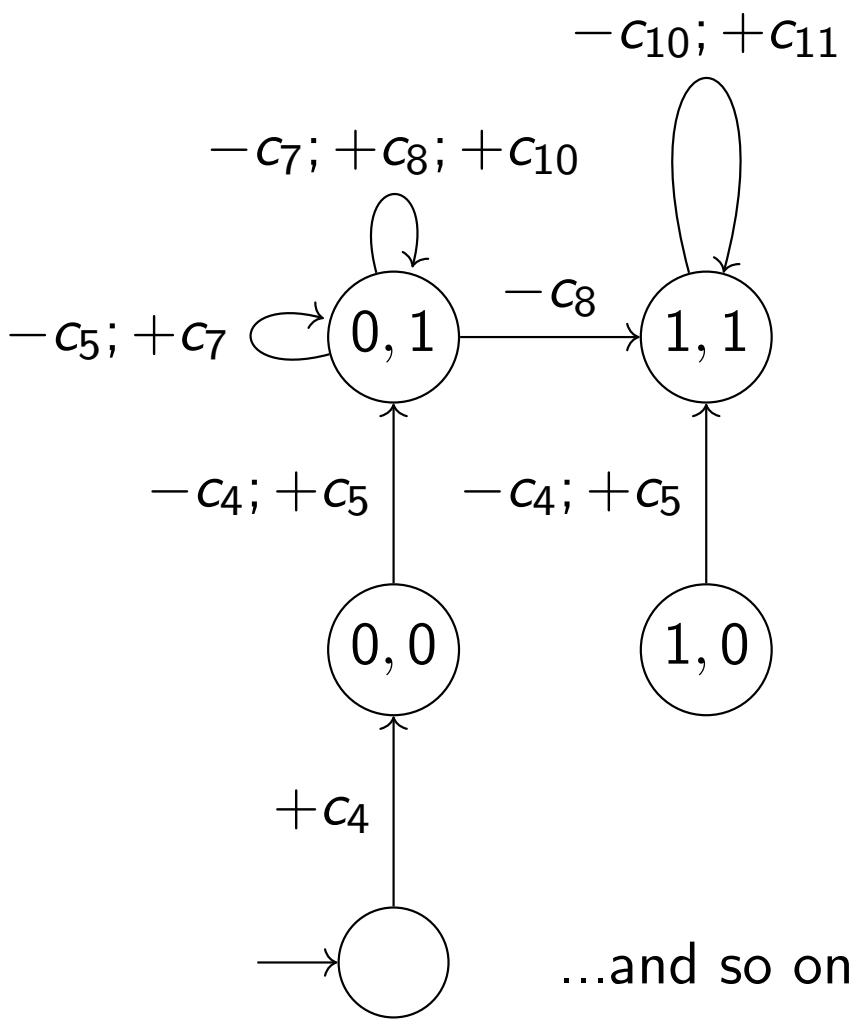
Section 1

An example

An example with concurrency

```
global bool x ← 0
global bool y ← 0
procedure main
    y ← 1
    spawn foo
procedure foo
    spawn bar
    x ← 1
procedure bar
    if x = 1 then
        spawn foo
        spawn foo
        y ← 0
```

$\rightsquigarrow$



Can we reach $x = 1$ and $y = 1$?
Build counter automaton: One counter per program location!
(Not all edges are shown.)

Vector addition systems

A (d-dimensional) *vector addition system with states (d-VASS)* is a tuple $V = (Q, T, q_0)$, where:

- Q is a finite set of *states*,
- $T \subseteq Q \times \mathbb{Z}^d \times Q$ is a finite set of *transitions*,
- an initial state $q_0 \in Q$.

A *configuration* is a pair $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$.
We write $(q, \mathbf{u}) \to (q', \mathbf{u}')$ if there is a $(q, \mathbf{v}, q') \in T$ with $\mathbf{u}' = \mathbf{u} + \mathbf{v}$.
We write $(q, \mathbf{u}) \to^* (q', \mathbf{u}')$ if there is a sequence

$$(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \cdots \to (q_n, \mathbf{u}_n)$$

with $(q_0, \mathbf{u}_0) = (q, \mathbf{u})$ and $(q_n, \mathbf{u}_n) = (q', \mathbf{u}')$.

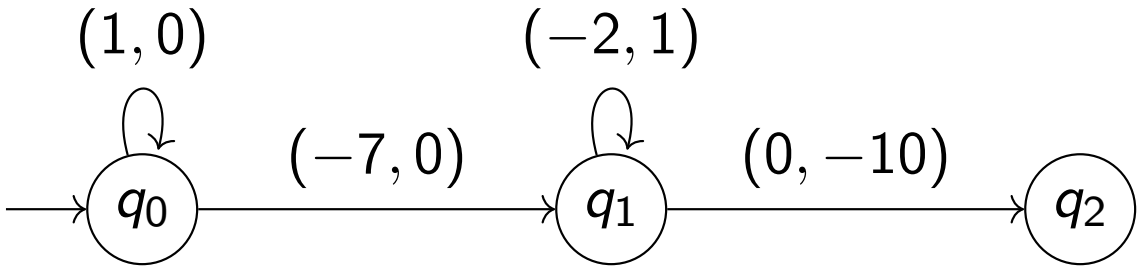
State reachability

Given A d-VASS $V = (Q, T, q_0)$ and a state $q \in Q$.

Question Is there a $\mathbf{u} \in \mathbb{N}^d$ with $(q_0, \mathbf{0}) \to^* (q, \mathbf{u})$?

This can be decided with methods from this part!

Example of a 2-VASS



Can we reach state q_2? Yes:

$$(q_0, (0,0)) \to^* (q_0, (27,0)) \to (q_1, (20,0))$$
$$\to^* (q_1, (0,10)) \to (q_2, (0,0))$$

Rough idea

Explore backwards which configurations can be *covered*.
To this end, we need to represent subsets of $\mathbb{N}^2$.

Part III

Well-structured transition systems

Section 2

Well-quasi-orderings

- With infinite automata, need to compute with infinite sets
- Unfortunately, $\mathbb{N}$ has uncountably many subsets
- No way to find a finite description for every subset of $\mathbb{N}$.

Motto

Well-quasi-orderings give us finite descriptions.

Using well-quasi-orderings, we can show: If a set has a certain structural property, then it can be finitely represented in a certain way.

Let's see some examples!

For $\mathbf{u}, \mathbf{v} \in \mathbb{N}^k$, with $\mathbf{u} = (u_1, \ldots, u_k)$ and $\mathbf{v} = (v_1, \ldots, v_k)$, let

$$\mathbf{u} \leq \mathbf{v} \iff u_i \leq v_i \text{ for every } i$$

For a subset $X \subseteq \mathbb{N}^k$, let $X{\uparrow} = \{\mathbf{v} \in \mathbb{N}^k \mid \exists \mathbf{u} \in X \colon \mathbf{u} \leq \mathbf{v}\}$ be the *upward closure of* X. We call X *upward closed* if $X{\uparrow} = X$.

Proposition (WQO Fact I)

If $X \subseteq \mathbb{N}^k$ is an upward closed subset with $X \neq \emptyset$, then there is a finite subset $X_0 \subseteq X$ with $X = X_0{\uparrow}$.

WQO applications: More examples

Definition

A subset $S \subseteq \mathbb{N}$ is called *ultimately periodic* if there are numbers n and k such that: for every $m \geq n$, we have $m \in S$ if and only if $m + k \in S$.

Proposition (WQO Fact II)

Let $S \subseteq \mathbb{N}$ such that for any $a, b \in S$, we have $a + b \in S$. Then S is ultimately periodic.

WQO applications: More examples

$$\texttt{radar} \preccurlyeq \texttt{abracadabra}$$

A word $u \in \Sigma^*$ is a *subword* of $v \in \Sigma^*$ if there are words $u_1, \ldots, u_n, v_0, v_1, \ldots, v_n \in \Sigma^*$ such that

$$u = u_1 \cdots u_n, \quad v = v_0 u_1 v_1 \cdots u_n v_n.$$

In this case, we write $u \preccurlyeq v$.
For a language $L \subseteq \Sigma^*$, the *downward closure of* L is

$$L{\downarrow} = \{u \in \Sigma^* \mid \exists v \in L \colon u \preccurlyeq v\}.$$

Example: If $L = \{a^n b^n \mid n \in \mathbb{N}\}$, then $L{\downarrow} = a^* b^*$.

Proposition (WQO Fact III)

If L is any subset of Σ^*, then $L{\downarrow}$ is a regular language.

Recall: A pair $(X, \leq)$ is called a *quasi-ordering* if $\leq \subseteq X \times X$,

- $\leq$ is *reflexive*, meaning $x \leq x$ for every $x \in X$, and
- $\leq$ is *transitive*, meaning: if $x \leq y$ and $y \leq z$, then $x \leq z$.

Moreover, we write $x < y$ if $x \leq y$ and $y \not\leq x$.

Definition

A quasi-ordering $(X, \leq)$ is a *well-quasi-ordering* (*WQO*) if for every infinite sequence $x_1, x_2, \ldots$ in X, there are indices i, j with $i < j$ such that $x_i \leq x_j$.

1. $(\mathbb{N}, \leq)$ is a well-quasi-ordering: Let $x_1, x_2, \ldots$ be an infinite sequence of natural numbers.
   - Suppose there is a bound $B \in \mathbb{N}$ with $x_i \leq B$ for every $i \in \mathbb{N}$. Then, some number must appear at least twice. Hence, there are i, j with $i < j$ such that $x_i = x_j$.
   - If there is no such bound, then for every $B \in \mathbb{N}$, there is an $i \in \mathbb{N}$ with $x_i > B$. Then, there must be some $i \in \mathbb{N}$ with $x_1 < x_i$.
2. $(\mathbb{Z}, \leq)$ is not a well-quasi-ordering: The sequence $0, -1, -2, -3, \ldots$ is a counter-example.

Definition

Let $(X, \leq)$ be a quasi-ordering. If $x_1, x_2, \ldots$ is an infinite sequence in X, then a *subsequence* is a (finite or infinite) sequence of the form $x_{i_1}, x_{i_2}, \ldots$ for some indices $i_1 < i_2 < \cdots$. In other words, $y_1, y_2, \ldots$ is a subsequence of $x_1, x_2, \ldots$ if it is obtained by deleting elements.

For example the sequence $1, 2, 3, 4, \ldots$ has $2, 4, 8, 16, 32, \ldots$ as a subsequence, but $1, 2, 1, 2$ is not a subsequence.

Definition

Let $(X, \leq)$ be a quasi ordering and $x_1, x_2, \ldots$ be a <u>finite or infinite</u> sequence in X.

- The sequence is an *ascending chain* if $x_i \leq x_j$ for every i, j with $i \leq j$.
- It is *strictly decreasing* if $x_i > x_j$ for every i, j with $i < j$.
- A subset $Y \subseteq X$ is an *anti-chain* if for $x, y \in Y$, $x \neq y$, we have $x \not\leq y$ and $y \not\leq x$.

For a subset $U \subseteq X$, we define the *upward closure* of U by

$$U{\uparrow} = \{x \in X \mid \exists u \in U \colon u \leq x\}.$$

We call $U \subseteq X$ *upward closed* if $U{\uparrow} = U$. Equivalently: U is upward closed if and only if $x \in U$ and $x \leq y$ implies $y \in U$.

Lemma ($\uparrow$ is idempotent)

Let $(X, \leq)$ be a quasi-ordering and $U \subseteq X$ be a subset. Then $U{\uparrow}{\uparrow} = U{\uparrow}$.

Clearly, $U{\uparrow} \subseteq U{\uparrow}{\uparrow}$ holds.
Conversely, let $x \in U{\uparrow}{\uparrow}$. Then there is $y \in U{\uparrow}$ with $y \leq x$. Since $y \in U{\uparrow}$, there is also $z \in U$ with $z \leq y$. By transitivity of $\leq$ we also have $z \leq x$ implying $x \in U{\uparrow}$. $\square$

Lemma ($\uparrow$ is compatible with inclusion)

Let $(X, \leq)$ be a quasi-ordering and $U, V \subseteq X$. If $U \subseteq V$ then $U{\uparrow} \subseteq V{\uparrow}$.

Let $x \in U{\uparrow}$. Then there is $u \in U$ with $u \leq x$. By $U \subseteq V$ we also have $u \in V$ implying $x \in V{\uparrow}$. $\square$

Proposition

Let $(X, \leq)$ be a quasi-ordering. Then the following are equivalent:

(1) $(X, \leq)$ *is a WQO.*

(2) *Every infinite sequence in X has an* <u>*infinite*</u> *ascending chain as a subsequence.*

(3) $(X, \leq)$ *has no infinite strictly decreasing sequence and no infinite anti-chain.*

(4) *For every non-empty upward closed $U \subseteq X$, there is a finite subset $U_0 \subseteq U$ such that $U = U_0{\uparrow}$.*

Note: A finite subset U_0 with $U = U_0{\uparrow}$ is called a *basis* of U.

Lemma (Equivalence, (1) $\Rightarrow$ (2))

If $(X, \leq)$ is a WQO, then every infinite sequence of X has an <u>infinite</u> ascending chain as a subsequence.

Suppose $x_1, x_2, \ldots$ is an infinite sequence of X.
We say that $i \in \mathbb{N}$ is *dominated* if there is a $j > i$, with $x_i \leq x_j$.
Otherwise, $i \in \mathbb{N}$ is *undominated*.
Let $U \subseteq \mathbb{N}$ be the set of undominated indices.

Claim: U is finite

If U is infinite, then let $U = \{i_1, i_2, \ldots\}$ with $i_1 < i_2 < \cdots$.
Since $(X, \leq)$ is a WQO, the infinite sequence $x_{i_1}, x_{i_2}, \ldots$ in X must have k, ℓ with $k < \ell$ and $x_{i_k} \leq x_{i_\ell}$.
But then i_k is dominated. Contradiction!

Since U is finite, we can set $M = \max U$. Observe: Every $n > M$ is dominated. We construct the subsequence $x_{i'_1}, x_{i'_2}, \ldots$ as follows:

- Set $i'_1 = M + 1$.
- For every k we have $i'_k > M$, hence i'_k is dominated. So there exists $j > i'_k$ with $x_{i'_k} \leq x_j$. Hence, set $i'_{k+1} = j$. $\square$

Lemma (Equivalence, (2) $\Rightarrow$ (3))

Suppose in $(X, \leq)$, every infinite sequence has an infinite ascending chain as a subsequence. Then $(X, \leq)$ has no infinite strictly decreasing chain and no infinite anti-chains.

- Suppose $(X, \leq)$ has an infinite strictly decreasing chain $x_1 > x_2 > x_3 > \cdots$. Then this sequence can clearly not have an infinite ascending chain as a subsequence. Contradiction!
- Suppose there is an infinite anti-chain $A \subseteq X$. Consider a sequence $x_1, x_2, \ldots$ of pairwise distinct elements of A. Since this sequence has an infinite ascending chain as a subsequence, there must be $i < j$ with $x_i \leq x_j$. Contradiction! $\square$

Lemma (Equivalence, step (3) $\Rightarrow$ (4))

Suppose $(X, \leq)$ has no infinite strictly decreasing sequences and no infinite anti-chains. Then for every non-empty upward closed subset $U \subseteq X$, there is a finite subset $U_0 \subseteq U$ such that $U = U_0\uparrow$.

Claim: Every nonempty subset $U \subseteq X$ has a minimal element

Suppose U has no minimal element. Then for any $x_1 \in U$, there is $x_2 < x_1$. Inductively, $x_1 > x_2 > x_3 \ldots$. Contradiction!

Let $U \subseteq X$ be a nonempty upward closed set. Let x_1 be a minimal element of U. Consider $U_1 = U \setminus x_1\uparrow$. If U_1 is empty, we are done. Otherwise there is $x_2 \in U_1$ which is minimal. Note that $x_1 \not\leq x_2$ and $x_2 \not\leq x_1$. Inductively, build a sequence $x_1, x_2, \ldots$. This sequence is an anti-chain and, by assumption, not infinite. So, $U_0 = \{x_1, x_2, \ldots\}$ is a finite set with $U = U_0\uparrow$. $\square$

Lemma (Equivalence, (4) $\Rightarrow$ (1))

Suppose $(X, \leq)$ is a quasi-order such that for every upward closed subset $U \subseteq X$, there exists a finite subset $U_0 \subseteq U$ with $U = U_0{\uparrow}$. Then $(X, \leq)$ is a WQO.

Let $x_1, x_2, \ldots$ be an infinite sequence. Define

$$U = \{x_1, x_2, \ldots\}{\uparrow}.$$

U is upward closed, so there exists a finite $U_0 \subseteq U$ with $U = U_0{\uparrow}$. Since U_0 is finite, there must be a $k \in \mathbb{N}$ with $U_0 \subseteq \{x_1, \ldots, x_k\}{\uparrow}$. Consider the element x_{k+1}. It belongs to U, hence $x_{k+1} \in U_0{\uparrow} \subseteq \{x_1, \ldots, x_k\}{\uparrow}{\uparrow} = \{x_1, \ldots, x_k\}{\uparrow}$ (by compatibility of $\uparrow$ with inclusion and idempotence of $\uparrow$).
Therefore, we have $x_i \leq x_{k+1}$ for some $i \in \{1, \ldots, k\}$.
Thus, $(X, \leq)$ is a WQO. $\square$

We have thus established:

Proposition

Let $(X, \leq)$ be a quasi-ordering. Then the following are equivalent:

(1) $(X, \leq)$ *is a WQO.*

(2) *Every infinite sequence in X has an* <u>*infinite*</u> *ascending chain as a subsequence.*

(3) $(X, \leq)$ *has no infinite strictly decreasing sequence and no infinite anti-chain.*

(4) *For every non-empty upward closed $U \subseteq X$, there is a finite subset $U_0 \subseteq U$ such that $U = U_0{\uparrow}$.*

Example WQOs

1. Let X be a <u>finite</u> set. Then $(X, =)$ is a WQO:
   - $=$ is reflexive and transitive.
   - In every infinite sequence $x_1, x_2, \ldots$ over X, some element must occur infinitely often. In particular, there are i, j with $i < j$ and $x_i = x_j$.
2. Let $(X, \leq_X)$ and $(Y, \leq_Y)$ be WQOs.
   We define relation $\leq_\times$ on $X \times Y$ by

$$(x_1, y_1) \leq_\times (x_2, y_2) \iff x_1 \leq_X x_2 \text{ and } y_1 \leq_Y y_2$$

   Then $(X \times Y, \leq_\times)$ is a WQO as well. Proof: Exercise.

Proof of WQO Fact I

Recall that for $\mathbf{u}, \mathbf{v} \in \mathbb{N}^k$ with
$\mathbf{u} = (u_1, \ldots, u_k)$ and $\mathbf{v} = (v_1, \ldots, v_k)$, we have

$$\mathbf{u} \leq \mathbf{v} \iff u_i \leq v_i \text{ for every } i \in \{1, \ldots, k\}.$$

Proposition (WQO Fact I)

If $X \subseteq \mathbb{N}^k$ is an upward closed subset with $X \neq \emptyset$, then there is a finite subset $X_0 \subseteq X$ with $X = X_0{\uparrow}$.

- We have seen that the product ordering of two WQOs is again a WQO.
- By induction on k, this implies that $(\mathbb{N}^k, \leq)$ is also a WQO.
- Hence, the proposition follows from the last characterization of WQOs: Every non-empty upward closed subset U is the upward closure of a finite subset $U_0 \subseteq U$.

Proof of WQO Fact II

Proposition (WQO Fact II)

Let $S \subseteq \mathbb{N}$ such that for any $a, b \in S$, we have $a + b \in S$. Then S is ultimately periodic.

Let $k \in S$ be arbitrary. Then we have $S + k \subseteq S$.
We prove a stronger statement:

Proposition

If $S + k \subseteq S$ for some $k \in \mathbb{N} \setminus \{0\}$, then S is ultimately periodic.

Consider the following relation on $\mathbb{N}$:

$$n \leq_k m \iff n \leq m \text{ and } n \equiv m \pmod{k}$$

E.g., $0 \leq_3 3$, $3 \not\leq_3 0$, $1 \leq_3 4$, $0 \not\leq_3 1$.
Then $(\mathbb{N}, \leq_k)$ is a WQO. Proof: Exercise.

Proof of WQO Fact II

Claim: S is upward closed in $(\mathbb{N}, \leq_k)$

If $n \in S$ and $n \leq_k m$, then $m = n + \ell \cdot k$ for some $\ell \in \mathbb{N}$.
Since $S + k \subseteq S$, we have $m \in S$.

Hence, there is a finite subset $S_0 \subseteq S$ with $S = S_0{\uparrow}$.
Since S_0 is finite, define $M := \max S_0$ (wrt. $\leq$).
Now let $n \geq M$. We show $n \in S$ if, and only if, $n + k \in S$.
If $n \in S$, then $n + k \in S + k \subseteq S$.
If $n + k \in S$, then there is $s_0 \in S_0$ with $s_0 \leq_k n + k$ (since $S = S_0{\uparrow}$). We know $s_0 \leq M \leq n$ and $n \equiv n + k \equiv s_0 \pmod{k}$ (by $s_0 \leq_k n + k$). It follows $s_0 \leq_k n$ and therefore $n \in s_0{\uparrow} \subseteq S$.
Thus, S is ultimately periodic. $\square$

Embedding finite sequences

Let $(X, \leq)$ be a quasi-ordering. We define $(X^*, \leq_*)$ as follows:

- X^* is the set of finite sequences over X:

$$X^* = \{(x_1, \ldots, x_n) \mid n \in \mathbb{N},\ x_1, \ldots, x_n \in X\}.$$

- We have $(x_1, \ldots, x_n) \leq_* (y_1, \ldots, y_m)$ if and only if $n \leq m$ and there exists a function $f\colon \{1, \ldots, n\} \to \{1, \ldots, m\}$ with $f(i+1) > f(i)$ for $i \in \{1, \ldots, n-1\}$ and

$$x_i \leq y_{f(i)} \text{ for every } i \in \{1, \ldots, n\}.$$

- For $u = (x_1, \ldots, x_n)$ and $v = (y_1, \ldots, y_m)$ in X^*, we define $uv = (x_1, \ldots, x_n, y_1, \ldots, y_m)$.
- Moreover, for $u = (x_1, \ldots, x_n)$, the *length* of u is $|u| := n$.

Examples:

- If $(X, \leq) = (\Sigma, =)$ for some alphabet Σ, then $(X^*, \leq_*) = (\Sigma^*, \preccurlyeq)$ is the subword ordering.
- If $(X, \leq) = (\mathbb{N}, \leq)$, then $(X^*, \leq_*) = (\mathbb{N}^*, \leq_*)$ is the set of sequences of natural numbers with

$$(1, 5) \leq_* (2, 1, 6), \quad (3, 2, 1) \not\leq_* (1, 2, 3)$$

Higman's Lemma

Theorem (Higman 1952)

If $(X, \leq)$ is a WQO, then $(X^*, \leq_*)$ is a WQO.

We follow the proof of Nash-Williams (1963). Some terminology:
Let $(Y, \leq)$ be a quasi-ordering. A (finite or infinite) sequence $y_1, y_2, \ldots$ in Y is *bad* if there are no indices i, j with $i < j$ and $y_i \leq y_j$.
Note: $(Y, \leq)$ is a WQO if and only if $(Y, \leq)$ has no infinite bad sequences.
An infinite sequence $y_1, y_2, \ldots$ in Y is *minimally bad* if

- the sequence is bad and
- for every $i \in \mathbb{N}$, there is no $y_i' \in Y$ with $y_i' < y_i$ such that $y_1, \ldots, y_{i-1}, y_i'$ is the beginning of an infinite bad sequence.

A quasi-ordering $(Y, \leq)$ is called *well-founded* if it has no infinite strictly decreasing sequence. If $(Y, \leq)$ is well-founded then every non-empty subset $X \subseteq Y$ has a minimal element (why?).

Lemma 1

If $(Y, \leq)$ is a well-founded quasi-ordering, but not a WQO, then $(Y, \leq)$ has a minimal bad sequence.

There exists some bad sequence because $(Y, \leq)$ is not a WQO. We construct a minimal bad sequence $y_1, y_2, \ldots$ inductively. For $i = 1, 2, \ldots$, we choose y_i minimal such that $y_1, \cdots, y_i$ is the beginning of some infinite bad sequence. Clearly, $y_1, y_2, \ldots$ is a minimal bad sequence. $\square$

Lemma 2

Let $(Y, \leq)$ be a quasi-ordering. If $y_1, y_2, \ldots$ is a minimal bad sequence in Y, then for $I = \{y \in Y \mid \exists i \in \mathbb{N}\colon y < y_i\}$, the pair $(I, \leq)$ is a WQO.

Suppose there is an infinite bad sequence $y'_1, y'_2, \ldots$ in I. Since $y'_1 \in I$, there is a y_k such that $y'_1 < y_k$. Consider the sequence

$$y_1, \ldots, y_{k-1}, y'_1, y'_2, \ldots$$

- This is a bad sequence: First, there can be no indices $i < j$ with $y'_i \leq y'_j$ by assumption. Moreover, suppose $y_i \leq y'_j$ for some i, j with $i \in \{1, \ldots, k-1\}$. Then there is a y_ℓ with $y'_j < y_\ell$. Hence $y_i \leq y'_j < y_\ell$ and thus $y_i < y_\ell$.
  This is impossible: If $i < \ell$, this contradicts badness of $y_1, y_2, \ldots$. If $\ell < i$, this contradicts minimality of $y_1, y_2, \ldots$.
- This contradicts minimal badness of $y_1, y_2, \ldots$, because $y'_1 < y_k$. $\square$

We can now show that $(X^*, \leq_*)$ is a WQO:

- The ordering is well-founded: Suppose $w_1, w_2, \ldots,$ is an infinite strictly decreasing chain in X^*.
  Then $|w_1| \geq |w_2| \geq \cdots$, hence $|w_k| = |w_{k+1}| = |w_{k+2}| = \cdots$ for some $k \in \mathbb{N}$. Therefore, there must be a component with an infinite strictly decreasing chain in X, which cannot exist.
- Suppose $(X^*, \leq_*)$ is not a WQO. By Lemma 1, $(X^*, \leq_*)$ admits a minimal bad sequence $w_1, w_2, \ldots$.
  As in Lemma 2, let $I = \{w \in X^* \mid \exists i \in \mathbb{N}\colon w <_* w_i\}$. Then $(I, \leq_*)$ is a WQO by Lemma 2.
- Since the $w_1, w_2, \ldots$ must be non-empty (why?), we can write $w_i = x_i v_i$ for $x_i \in X$ and $v_i \in X^*$. Note that $v_i \in I$, hence we can select an infinite ascending chain $v_{i_1}, v_{i_2}, \ldots$ of $v_1, v_2, \ldots$.
- Since $(X, \leq)$ is a WQO, there must be two indices i_k and i_ℓ with $i_k < i_\ell$ with $x_{i_k} \leq x_{i_\ell}$. But then we have

$$w_{i_k} = x_{i_k} v_{i_k} \leq_* x_{i_\ell} v_{i_\ell} = w_{i_\ell},$$

contradiction! $\square$

We are now ready to prove:

Proposition (WQO Fact III)

If L is any subset of Σ^*, then $L{\downarrow}$ is a regular language.

Lemma

If $U \subseteq \Sigma^*$ is upward closed, then U is a regular language.

Note that for any word $w \in \Sigma^*$ with $w = a_1 \cdots a_n$, $a_1, \ldots, a_n \in \Sigma$, we have

$$\{w\}{\uparrow} = \Sigma^* a_1 \Sigma^* \cdots a_n \Sigma^*,$$

hence $\{w\}{\uparrow}$ is a regular language.
Since $(\Sigma^*, \preccurlyeq)$ is a WQO, U has a basis $U_0 = \{w_1, \ldots, w_m\}$. Then we have

$$U = \{w_1, \ldots, w_m\}{\uparrow} = \bigcup_{i=1}^{m} \{w_i\}{\uparrow},$$

hence U is a regular language. $\square$

Proposition (WQO Fact III)

If L is any subset of Σ^*, then $L{\downarrow}$ is a regular language.

Suppose $L \subseteq \Sigma^*$ is an arbitrary language.
Then $\Sigma^* \setminus L{\downarrow}$, the complement of $L{\downarrow}$, is upward closed.
Therefore, $\Sigma^* \setminus L{\downarrow}$ is a regular language by the lemma.
Hence, $L{\downarrow}$ is the complement of a regular language, and thus regular. $\square$

Embedding infinite sequences

Can we show the same for infinite sequences? Let $(X, \leq)$ be a quasi-ordering. We define $(X^\omega, \leq_\omega)$ as follows:

- X^ω is the set of infinite sequences over X:

$$X^\omega = \{(x_1, x_2, \ldots) \mid x_1, x_2, \ldots \in X\}.$$

- We have $(x_1, x_2, \ldots) \leq_\omega (y_1, y_2, \ldots)$ if and only if there exists a function $f : \mathbb{N} \to \mathbb{N}$ with $f(i+1) > f(i)$ for $i \in \mathbb{N}$ and

$$x_i \leq y_{f(i)} \text{ for every } i \in \mathbb{N}.$$

If $(X, \leq)$ is a WQO, can we say that $(X^\omega, \leq_\omega)$ is also a WQO? No! Let us see an example.

Rado's ordering

Define the following ordering on $R = \{(i,j) \in \mathbb{N}^2 \mid i < j\}$:

$$(i,j) \leq^{\mathrm{R}} (k,\ell) \iff (i = k \text{ and } j \leq \ell) \text{ or } i,j < k.$$

Claim: $(R, \leq^{\mathrm{R}})$ is a WQO

Let $(i_1,j_1),(i_2,j_2),\ldots$ be an infinite sequence.

- Suppose $i_1,i_2,\ldots$ is bounded, i.e. there is a $B \in \mathbb{N}$ with $i_k \leq B$ for every $k \in \mathbb{N}$. Then we can pick an infinite subsequence $(i,j_1'),(i,j_2'),\ldots$. Then again, we can pick an infinite subsequence $(i,j_1''),(i,j_2''),\ldots$ where $j_1'' \leq j_2'' \leq \cdots$, since $(\mathbb{N},\leq)$ is a WQO. But then $(i,j_1''),(i,j_2''),\ldots$ is also an infinite ascending chain w.r.t. $\leq^{\mathrm{R}}$.
- Suppose $i_1,i_2,\ldots$ is unbounded, i.e. for every $B \in \mathbb{N}$, there is a $k \in \mathbb{N}$ with $i_k > B$. Then in particular, there is a $k \in \mathbb{N}$ with $i_1 < i_k$ and $j_1 < i_k$. Thus, $(i_1,j_1) \leq^{\mathrm{R}} (i_k,j_k)$. $\square$

$$(i,j) \leq^{\mathsf{R}} (k,\ell) \iff (i = k \text{ and } j \leq \ell) \text{ or } i,j < k.$$

Claim: $(R^\omega, \leq^{\mathsf{R}}_\omega)$ is not a WQO.

Consider the sequence

$$\begin{aligned}
\sigma_1 &= ((1,2),(1,3),(1,4),(1,5),\ldots) \\
\sigma_2 &= ((2,3),(2,4),(2,5),(2,6),\ldots) \\
\sigma_3 &= ((3,4),(3,5),(3,6),(3,7),\ldots) \\
&\vdots \\
\sigma_i &= ((i,i+1),(i,i+2),\ldots) \\
&\vdots
\end{aligned}$$

Then $\sigma_i \not\leq^{\mathsf{R}}_\omega \sigma_j$ for $i < j$:
The element $(i, j+1)$ cannot embed into any element of $\sigma_j = ((j,j+1),(j,j+2),\ldots)$. $\square$

Part III

Well-structured transition systems

Section 3

Well-structured transition systems

Transition systems

We are now ready to define the central concept of this lecture part: well-structured transition systems.

Definition

A *transition system (TS)* is a triple $(\Gamma, \gamma_0, \rightarrow)$, where

- Γ is a (finite or infinite!) set of *configurations*,
- $\gamma_0 \in \Gamma$ is its *initial configuration*,
- $\rightarrow$ is a binary relation on Γ, called the *transition relation*. Hence: $\rightarrow \subseteq \Gamma \times \Gamma$.

For $\gamma, \gamma' \in \Gamma$, we write $\gamma \rightarrow^* \gamma'$ if there are configurations $\gamma_1, \ldots, \gamma_n \in \Gamma$ with

$$\gamma_1 \rightarrow \gamma_2 \rightarrow \cdots \rightarrow \gamma_n$$

and $\gamma = \gamma_1$ and $\gamma' = \gamma_n$.

Well-structured transition systems

Definition

A *well-structured transition system (WSTS)* is a tuple $(\Gamma, \gamma_0, \rightarrow, \leq)$, where

- $(\Gamma, \gamma_0, \rightarrow)$ is a transition system,
- $(\Gamma, \leq)$ is a WQO,
- The relation $\leq$ is *compatible* with $\rightarrow$: For all $\gamma_1, \gamma_2, \gamma_3 \in \Gamma$: If $\gamma_1 \rightarrow \gamma_2$ and $\gamma_3 \geq \gamma_1$, then there is a $\gamma_4 \in \Gamma$ with $\gamma_4 \geq \gamma_2$ and $\gamma_3 \rightarrow \gamma_4$:

$$\begin{array}{ccc} \gamma_3 & \rightarrow & \gamma_4 \\ \vee\!| & & \vee\!| \\ \gamma_1 & \rightarrow & \gamma_2 \end{array}$$

  Intuition: "Larger configurations allow more transitions"

Informally: We assume that every element of Γ has a unique finite description, say as a string.

Example: Vector Addition Systems

Example

Every d-VASS $V = (Q, T, q_0)$ is a WSTS:

- $\Gamma = Q \times \mathbb{N}^d$,

$$(q, \mathbf{u}) \leq (q', \mathbf{u}') \iff q = q' \text{ and } \mathbf{u} \leq \mathbf{u}'$$

  is a well-quasi-ordering.
- For every $c_1, c_2, c_1' \in \Gamma$, if $c_1 \to c_2$ and $c_1' \geq c_1$, then there is a configuration $c_2' \in \Gamma$ with $c_2' \geq c_2$ and $c_1' \to c_2'$.

The coverability problem

The coverability problem

Let $(\Gamma, \gamma_0, \rightarrow, \leq)$ be a WSTS.

Given A configuration $\gamma \in \Gamma$.

Question Is there a $\gamma' \geq \gamma$ with $\gamma_0 \rightarrow^* \gamma'$?

We now show: This is "almost always" decidable!

Given a subset $S \subseteq \Gamma$, define

$$\mathrm{pre}(S) = \{\gamma \in \Gamma \mid \exists \gamma' \in S \colon \gamma \to \gamma'\}\,.$$

Lemma

Let $(\Gamma, \gamma_0, \to, \leq)$ *be a WSTS. If* $U \subseteq \Gamma$ *is upward closed, then* $\mathrm{pre}(U)$ *is also upward closed.*

Proof.

Let $\gamma \in \mathrm{pre}(U)$ and $\gamma' \geq \gamma$.
Then $\gamma \to \delta$ for some $\delta \in U$.
By compatibility, there is a $\delta' \in \Gamma$ with $\gamma' \to \delta'$ and $\delta' \geq \delta$.
Since U is upward closed, we know that $\delta' \in U$.
Thus, $\gamma' \in \mathrm{pre}(U)$. $\square$

Theorem

Let $W = (\Gamma, \gamma_0, \rightarrow, \leq)$ *be a WSTS such that:*

1. *The ordering* $\leq$ *is decidable: Given* $\gamma, \gamma' \in \Gamma$*, one can decide whether* $\gamma \leq \gamma'$*.*
2. *Given* $\gamma \in \Gamma$*, one can compute a basis of* $\mathsf{pre}(\gamma\uparrow)$*.*

Then, the coverability problem for W *is decidable.*

Key idea

We store an upward closed set by storing a basis!

Obvervations

Suppose $(\Gamma, \gamma_0, \rightarrow, \leq)$ is a WSTS with the properties of the theorem.

1. Given upward closed sets $U, V \subseteq \Gamma$, we can compute $U \cup V$:
   For a basis U_0 of U and a basis V_0 of V, take $U_0 \cup V_0$.
2. Given an upward closed set $U \subseteq \Gamma$ and some $\gamma \in \Gamma$, we can decide whether $\gamma \in U$:
   Given a basis $U_0 = \{\gamma_1, \ldots, \gamma_n\}$ of U, we check for each $i = 1, 2, \ldots, n$, whether $\gamma_i \leq \gamma$. If $\gamma_i \leq \gamma$, then $\gamma \in U$. If $\gamma_i \not\leq \gamma$ for every i, then $\gamma \notin U$.
3. Given upward closed sets $U, V \subseteq \Gamma$, we can decide whether $U \subseteq V$:
   For a basis U_0 of U and a basis V_0 of V, we have $U \subseteq V$ if and only if for every $\gamma \in U_0$, there is a $\gamma' \in V_0$ with $\gamma' \leq \gamma$.

Computing with upward closed sets

Observation

4. Given an upward closed set U, we can compute $\mathrm{pre}(U)$: If $U_0 = \{\gamma_1, \ldots, \gamma_n\}$ is a basis of U, then
$$\mathrm{pre}(U) = \bigcup_{i=1}^{n} \mathrm{pre}(\gamma_i\uparrow).$$
Since we can compute a basis for each $\mathrm{pre}(\gamma_i\uparrow)$, we can compute a basis for $\mathrm{pre}(U)$.

Given a subset $S \subseteq \Gamma$, define

$$\mathrm{pre}^*(S) = \{\gamma \in \Gamma \mid \exists \gamma' \in S \colon \gamma \to^* \gamma'\}$$

Observation

For coverability, we are given $\gamma \in \Gamma$ and want to check whether $\gamma_0 \in \mathrm{pre}^*(\gamma\uparrow)$.

$$\mathrm{pre}^0(S) = S, \qquad \mathrm{pre}^{i+1}(S) = \mathrm{pre}^i(S) \cup \mathrm{pre}(\mathrm{pre}^i(S)) \text{ for } i \geq 0.$$

Lemma

Let $S \subseteq \Gamma$ be a subset. Then $\mathrm{pre}^*(S) = \bigcup_{i \geq 0} \mathrm{pre}^i(S)$.

Proof.

Show by induction: $\mathrm{pre}^i(S)$ contains exactly those $\gamma \in \Gamma$ from which one can reach an element from S in at most i steps. $\square$

Abdulla's backward algorithm

Let $(\Gamma, \gamma_0, \rightarrow, \leq)$ a WSTS such that $\leq$ is decidable and given $\gamma \in \Gamma$, one can compute $\mathrm{pre}(\gamma\uparrow)$.

Backward search algorithm

input: $\gamma \in \Gamma$.
$U \leftarrow \gamma\uparrow$
repeat
    $U \leftarrow U \cup \mathrm{pre}(U)$
until $\mathrm{pre}(U) \subseteq U$
Return "yes" if $\gamma_0 \in U$; otherwise return "no".

- Computes upward closed sets $U_1, U_2, \ldots$ with $U_0 = \gamma\uparrow$, $U_{i+1} = U_i \cup \mathrm{pre}(U_i)$ for $i \geq 0$.
- By induction: $U_i = \mathrm{pre}^i(\gamma\uparrow)$.
- If the algorithm terminates, we have $U = \mathrm{pre}^*(\gamma\uparrow)$ in the end.
- It must terminate: Otherwise $U_1 \subsetneq U_2 \subsetneq \cdots$ would be a strictly ascending chain of upward closed sets (Exercise!).

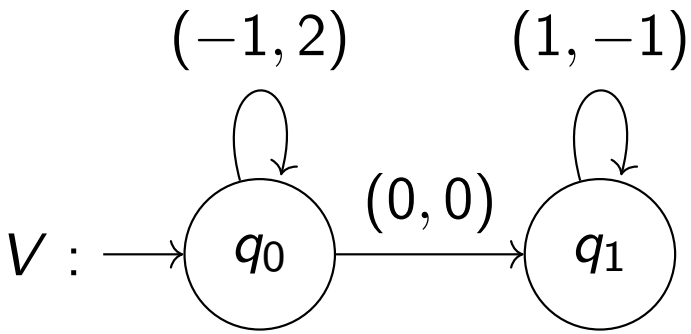
Decidability properties in WSTS

The following WSTS satisfy the conditions in Theorem on slide 62:

- Finite transition systems
- Vector addition systems with states

Next aim: Find more such WSTS!

Is $\gamma = (q_1, (3,0))$ coverable from $\gamma_0 = (q_0, (5,0))$ in the following VASS V?



$U_0 = \{(q_1, (3,0))\}\uparrow, \qquad U_1 = \{(q_1, (3,0)), (q_1, (2,1)), (q_0, (3,0))\}\uparrow,$

$U_2 = \{(q_1, (3,0)), (q_1, (2,1)), (q_1, (1,2)), (q_0, (3,0)), (q_0, (2,1))\}\uparrow,$

$$U_3 = \left\{\begin{matrix}(q_1, (3,0)), (q_1, (2,1)), (q_1, (1,2)), (q_1, (0,3)), \\ (q_0, (3,0)), (q_0, (2,1)), (q_0, (1,2))\end{matrix}\right\}\Bigg\uparrow,$$

$$U_4 = \left\{\begin{matrix}(q_1, (3,0)), (q_1, (2,1)), (q_1, (1,2)), (q_1, (0,3)), \\ (q_0, (2,0)), (q_0, (1,2)), (q_0, (0,3))\end{matrix}\right\}\Bigg\uparrow,$$

$$U_5 = \left\{\begin{matrix}(q_1, (3,0)), (q_1, (2,1)), (q_1, (1,2)), (q_1, (0,3)), \\ (q_0, (2,0)), (q_0, (1,1)), (q_0, (0,3))\end{matrix}\right\}\Bigg\uparrow,$$

$U_6 = U_5$, STOP. $\qquad \gamma_0 \in U_5$, therefore γ is coverable from γ_0.

Further WSTS

A (d-dimensional) *reset vector addition system with states (reset d-VASS)* is a tuple $V = (Q, T, q_0)$, where:

- Q is a finite set of *states*,
- $T \subseteq Q \times (\mathbb{Z}^d \cup \{\mathsf{r}_1, \ldots, \mathsf{r}_d\}) \times Q$ is a finite set of *transitions*,
- an initial state $q_0 \in Q$.

A *configuration* is a pair $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$. Consider $(q, \mathbf{u})$ and $(q', \mathbf{u}')$, $\mathbf{u} = (u_1, \ldots, u_d)$ and $\mathbf{u}' = (u'_1, \ldots, u'_d)$. We write $(q, \mathbf{u}) \to (q', \mathbf{u}')$ if either

- there is a $(q, \mathbf{v}, q') \in T$ with $\mathbf{v} \in \mathbb{Z}^d$ and $\mathbf{u}' = \mathbf{u} + \mathbf{v}$; or
- there is a $(q, \mathsf{r}_j, q') \in T$ and

$$u'_i = \begin{cases} 0 & \text{for } i = j \\ u_i & \text{for } i \neq j \end{cases}.$$

Exercise

Reset d-VASS are WSTS (with decidability property)!

Further WSTS

Exercise

Two-counter machines (with the same ordering as VASS) are not WSTS!

Channel systems

Idea: threads communicate via messages

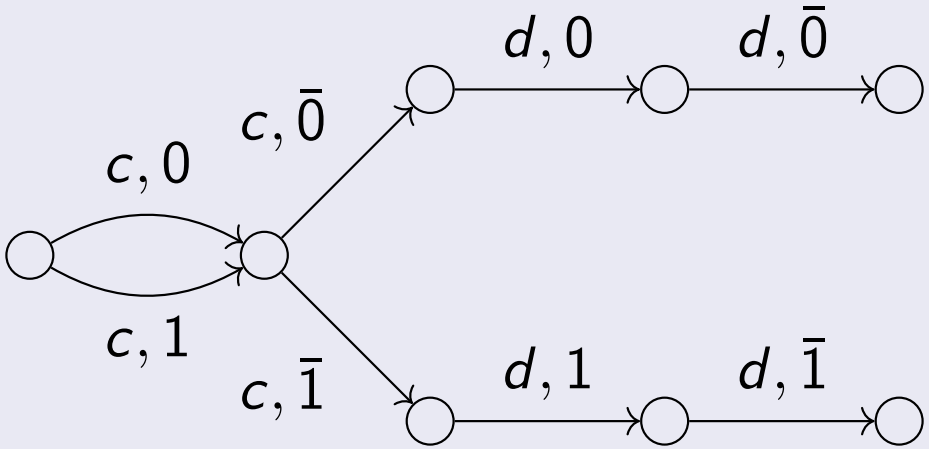
Example with two threads; two channels: c and d

bool $x = 0$
if rand() is even **then**
    send$(c, 0)$
else
    send$(c, 1)$
$x =$ receive(d)

bool $y = 0$
$y =$ receive(c)
send(d, y)

As an automaton

$c, 0$: send 0 into channel c; $c, \bar{0}$: read 0 from channel c, etc.



Some notation:

1. For an alphabet Σ, let $\bar{\Sigma} = \{\bar{a} \mid a \in \Sigma\}$.
   Hence, $\bar{\Sigma}$ is a "decorated version" of Σ.
   For channel systems, $\bar{a}$ is the instruction to read the letter a.
2. For sets A, B, let $A^B = \{f \mid f \colon B \to A\}$.
   In particular: For a finite set C of channels, $(\Sigma^*)^C$ is the set of functions $f \colon C \to \Sigma^*$.
   Hence, if $C = \{c_1, \ldots, c_n\}$, we can think of $(\Sigma^*)^C$ as

$$\underbrace{\Sigma^* \times \cdots \times \Sigma^*}_{n \text{ times}}$$

Channel systems

A *channel system* is a tuple $S = (Q, C, \Sigma, T, q_0)$ where

- Q is a finite set of *states*,
- C is a finite set of *channels*,
- Σ is an alphabet, called *channel alphabet*,
- $T \subseteq Q \times C \times (\Sigma^* \cup \bar{\Sigma}^*) \times Q$ is a finite set of *transitions*, and
- $q_0 \in Q$ is the *initial state*.

A *configuration* of S is a tuple in $Q \times (\Sigma^*)^C$.

We have $(q, \tau) \to (q', \tau')$ if there is a transition (q, c, x, q'), $x \in \Sigma^* \cup \bar{\Sigma}^*$, with

- $x = u \in \Sigma^*$ and

$$\tau'(d) = \begin{cases} \tau(c)u & \text{if } d = c \\ \tau(d) & \text{if } d \neq c \end{cases} \text{ for every } d \in C$$

- $x = \bar{u}$ for some $u \in \Sigma^*$ and $\tau(c) = uw$ for some $w \in \Sigma^*$ and

$$\tau'(d) = \begin{cases} w & \text{if } d = c \\ \tau(d) & \text{if } d \neq c \end{cases} \text{ for every } d \in C$$

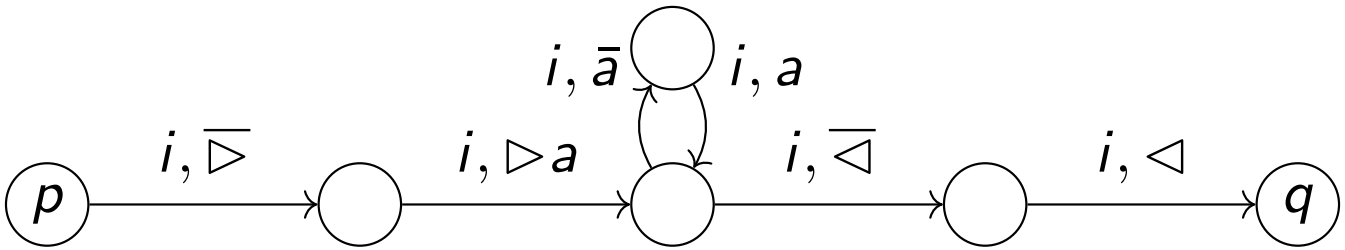
Unfortunately, even state reachability is undecidable in channel systems.

By reduction from 2-counter machines.

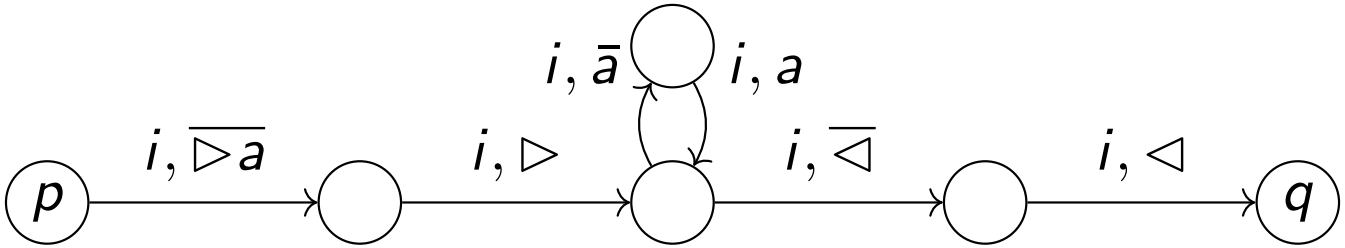
Idea: Encode $n \in \mathbb{N}$ as channel content $\triangleright a^n \triangleleft$

May assume: transitions are *increment*, *decrement*, or *zero tests*.

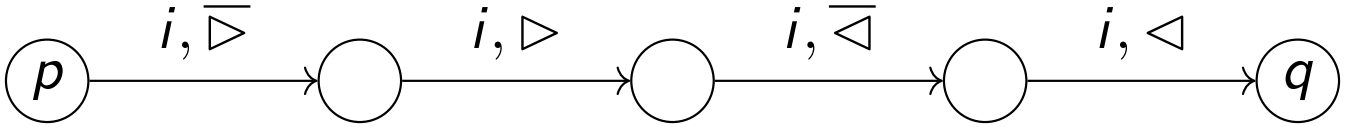
For increment of counter $i \in \{1, 2\}$, we use the following gadget:



To decrement counter $i \in \{1, 2\}$:



To zero-test counter $i \in \{1, 2\}$:



Idea

If the channels are *unreliable*, then we get WSTS!
Here, unreliable means: Letters in channels can disappear.

Lossy channel systems

A *lossy channel system* is a tuple $S = (Q, C, \Sigma, T, q_0)$ with components just as in a channel system.
Again, a *configuration* of S is a tuple in $Q \times (\Sigma^*)^C$.

Difference to channel systems

The step relation can "lose" before and after it modifies the channel:
We have $(q, \tau) \to (q', \tau')$ if there is a $\tau_1, \tau_2 \in (\Sigma^*)^C$ such that:

- for every $c \in C$, we have $\tau_1(c) \preccurlyeq \tau(c)$,
- $(q, \tau_1) \to (q', \tau_2)$ is allowed by the rules of channel systems, and
- for every $c \in C$, we have $\tau'(c) \preccurlyeq \tau_2(c)$.

Lossy channel systems as WSTS

For a lossy channel system $S = (Q, C, \Sigma, T, q_0)$, we define the following ordering:

$$(q, \tau) \leq (q', \tau') \iff q = q' \text{ and } \tau(c) \preccurlyeq \tau'(c) \text{ for every } c \in C$$

Then:

- With this ordering, lossy channel systems are WSTS.
- $\leq$ is decidable on configurations $Q \times (\Sigma^*)^C$
- Given $(q, \tau) \in Q \times (\Sigma^*)^C$, one can compute a basis for $\mathrm{pre}((q, \tau)\uparrow)$.

Proof: Exercise.

Therefore, coverability is decidable for lossy channel systems!

Part III

Well-structured transition systems

Section 4

The termination problem in WSTS

Checking termination

Definition

Let $S = (\Gamma, \gamma_0, \rightarrow)$ be a transition system.
An *infinite run* is an infinite sequence $\gamma_0, \gamma_1, \gamma_2, \ldots \in \Gamma$ of configurations with

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \cdots .$$

We say that S *terminates* if it has no infinite run.

Next goal

An algorithm for termination of WSTS.

Examples

- $(\mathbb{N}, 0, \rightarrow, \leq)$ with

$$x \rightarrow y \iff y = x + 2$$

- $(\mathbb{N}^2, (0, 5), \rightarrow, \leq)$ with

$$\begin{aligned}(x_1, y_1) \rightarrow (x_2, y_2) \iff & (x_2 = x_1 - 5 \text{ and } y_2 = y_1 + 6) \text{ or} \\ & (x_2 = x_1 + 2 \text{ and } y_2 = y_1 - 1)\end{aligned}$$

- $(\mathbb{N}^2, (0, 2), \rightarrow, \leq)$ with the same transition relation $\rightarrow$

Let $W = (\Gamma, \gamma_0, \rightarrow, \leq)$ be a WSTS.

Termination problem for W

Given A configuration $\gamma \in \Gamma$.

Question Is the WSTS $(\Gamma, \gamma, \rightarrow, \leq)$ terminating?

Definition

For $\gamma \in \Gamma$, let

$$\mathsf{post}(\gamma) = \{\gamma' \in \Gamma \mid \gamma \rightarrow \gamma'\}$$
$$\mathsf{post}^*(\gamma) = \{\gamma' \in \Gamma \mid \gamma \rightarrow^* \gamma'\}$$

We say that the transition system is *finitely branching* if for every $\gamma \in \Gamma$, the set $\mathsf{post}(\gamma)$ is finite.

Theorem

Let $W = (\Gamma, \gamma_0, \rightarrow, \leq)$ be a finitely branching WSTS such that (i) $\leq$ is decidable and (ii) given $\gamma \in \Gamma$, we can compute $\mathsf{post}(\gamma)$. Then, the termination problem for W is decidable.

Lemma

Let $S = (\Gamma, \gamma_0, \rightarrow)$ be a finitely branching transition system. If S is terminating, then $\mathrm{post}^*(\gamma_0)$ *is finite.*

We consider infinite trees.
The tree is called *infinite* if it has infinitely many nodes.
A tree is called *locally finite* if every node has only finitely many children.
A *branch* is a path $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots$ such that v_0 is the root and v_{i+1} is a child of v_i for every $i \geq 0$.

Theorem (Kőnig's Lemma)

Every infinite locally finite tree contains an infinite branch.

We construct the infinite branch inductively. We know that all of the infinitely many vertices can be reached from v_0. Suppose $u_1, u_2, \ldots, u_k$ are the children of v_0. At least one of them can be used to reach infinitely many vertices, say u_i. Set $v_1 = u_i$ and iterate. $\square$

Proof of the lemma

Suppose $\mathcal{S} = (\Gamma, \gamma_0, \rightarrow)$ is finitely branching and terminating.
Towards a contradiction, suppose $\text{post}^*(\gamma_0)$ is infinite.
Consider the following tree:

1. Create a root node and label it with γ_0
2. Create child nodes as follows:
   For each node labeled with γ, add a new child for each γ' with $\gamma \rightarrow \gamma'$.

Observe:

- Since $\mathcal{S}$ is finitely branching, the tree is locally finite.
- Since $\text{post}^*(\gamma_0)$ is infinite, the tree is infinite.
- By Kőnig's Lemma, it has an infinite branch.
- This infinite branch yields an infinite run, contradiction! $\square$

Compatibility on runs

Lemma

Let $(\Gamma, \gamma_0, \rightarrow, \leq)$ be a WSTS. If $\gamma_1 \rightarrow^* \gamma_2$ and $\gamma_3 \geq \gamma_1$, then there is a $\gamma_4 \in \Gamma$ with $\gamma_4 \geq \gamma_2$ and $\gamma_3 \rightarrow^* \gamma_4$.

$$\begin{array}{ccc} \gamma_3 & \xrightarrow{*} & \gamma_4 \\ \rotatebox{90}{$\leq$} & & \rotatebox{90}{$\leq$} \\ \gamma_1 & \xrightarrow{*} & \gamma_2 \end{array}$$

Since $\gamma_1 \rightarrow^* \gamma_2$, there are $\delta_1, \ldots, \delta_n \in \Gamma$ with $\gamma_1 = \delta_1$, $\gamma_2 = \delta_n$, and $\delta_1 \rightarrow \delta_2 \rightarrow \cdots \rightarrow \delta_n$. By compatibility of $\leq$, there are $\delta'_2, \ldots, \delta'_n$ with:

$$\begin{array}{ccccccccc} \gamma_3 & \longrightarrow & \delta'_2 & \longrightarrow & \delta'_3 & \longrightarrow & \cdots & \longrightarrow & \delta'_n \\ \rotatebox{90}{$\leq$} & & \rotatebox{90}{$\leq$} & & \rotatebox{90}{$\leq$} & & & & \rotatebox{90}{$\leq$} \\ \gamma_1 = \delta_1 & \longrightarrow & \delta_2 & \longrightarrow & \delta_3 & \longrightarrow & \cdots & \longrightarrow & \delta_n = \gamma_2 \end{array}$$

With $\gamma_4 = \delta'_n$, we have found the desired configuration. $\square$

Definition

Let $(\Gamma, \gamma_0, \rightarrow)$ be a transition system. A run

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \cdots \rightarrow \gamma_n$$

is called *self-covering* if $\gamma_i \leq \gamma_n$ for some $i \in [0, n-1]$.

Lemma

A WSTS has an infinite run if and only if it has a self-covering run.

Proof.

"$\Rightarrow$": Suppose $\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \cdots$ is an infinite run.
Since $\leq$ is a WQO, there are i, j with $i < j$ such that $\gamma_i \leq \gamma_j$.
Hence, $\gamma_0 \rightarrow \gamma_1 \rightarrow \cdots \rightarrow \gamma_j$ is a self-covering run.
"$\Leftarrow$": Suppose $\gamma_0 \rightarrow \gamma_1 \rightarrow \cdots \rightarrow \gamma_n$ be an self-covering run with $\gamma_i \leq \gamma_n$. By the previous lemma, there are configurations $\gamma_{n+1}, \gamma_{n+2}, \ldots$ with:

$$\begin{array}{ccccccccc}
 & & \gamma_n & \xrightarrow{+} & \gamma_{n+1} & \xrightarrow{+} & \gamma_{n+2} & & \cdots \\
 & & \vee\!| & & \vee\!| & & \vee\!| & & \\
\gamma_0 & \xrightarrow{*} & \gamma_i & \xrightarrow{+} & \gamma_n & \xrightarrow{+} & \gamma_{n+1} & & \cdots
\end{array}$$

Hence, the WSTS has an infinite run. $\square$

Theorem

Let $W = (\Gamma, \gamma_r, \rightarrow, \leq)$ be a finitely branching WSTS such that (i) $\leq$ is decidable and (ii) given $\gamma \in \Gamma$, we can compute $\mathrm{post}(\gamma)$*. Then, the termination problem for W is decidable.*

We work with trees whose nodes are labeled with config. from Γ.
For a node v, let $\gamma_v \in \Gamma$ be its label.
A node on a path from the root to v is called *ancestor* of v.

Checking termination

input: $\gamma_r \in \Gamma$.
Initialize t as tree with one node, labeled by γ_r.
while t has a leaf v with $\mathrm{post}(\gamma_v) \neq \emptyset$ **do**
    Take a leaf v with $\mathrm{post}(\gamma_v) \neq \emptyset$
    for $\gamma' \in \mathrm{post}(\gamma_v)$ **do**
        add a child under v with label γ'.
        if v has an ancestor $\bar{v}$ with $\gamma_{\bar{v}} \leq \gamma'$ **then**
            Return "non-terminating" and exit.
Return "terminating".

Checking termination

input: $\gamma_r \in \Gamma$.
Initialize t as tree with one node, labeled by γ_r.
while t has a leaf v with $\text{post}(\gamma_v) \neq \emptyset$ **do**
    Take a leaf v with $\text{post}(\gamma_v) \neq \emptyset$
    for $\gamma' \in \text{post}(\gamma_v)$ **do**
        add a child under v with label γ'.
        if v has an ancestor $\bar{v}$ with $\gamma_{\bar{v}} \leq \gamma'$ **then**
            Return "non-terminating" and exit.
Return "terminating".

- If the algorithm reports "non-terminating": It has found a self-covering run. Hence, the system is indeed non-terminating.
- If the algorithm reports "terminating", then the computed tree is finite and every run of $(\Gamma, \gamma_r, \rightarrow, \leq)$ is present as a branch. Hence, there is no infinite run.

Checking termination

input: $\gamma_r \in \Gamma$.
Initialize t as tree with one node, labeled by γ_r.
while t has a leaf v with $\text{post}(\gamma_v) \neq \emptyset$ **do**
    Take a leaf v with $\text{post}(\gamma_v) \neq \emptyset$
    for $\gamma' \in \text{post}(\gamma_v)$ **do**
        add a child under v with label γ'.
        if v has an ancestor $\bar{v}$ with $\gamma_{\bar{v}} \leq \gamma'$ **then**
            Return "non-terminating" and exit.
Return "terminating".

- The algorithm must terminate: Suppose not. The algorithm constructs an infinite tree that is locally finite. By Kőnig's Lemma, the tree must have an infinite branch $v_0 \to v_1 \to v_2 \to \cdots$. Since $\leq$ is a WQO, there must be nodes v_i and v_j, $i < j$, such that $\gamma_{v_i} \leq \gamma_{v_j}$.
  But then, the algorithm would have reported "non-terminating" when adding v_j. Contradiction!

Consequences: Termination is decidable for

- finite systems
- vector addition systems with states (VASS)
- reset VASS
- lossy channel systems

Question

Why are lossy channel systems finitely branching?

Part III

Well-structured transition systems

Section 5

The (configuration) reachability problem

Configuration reachability

We know now that we can "almost always" decide coverability.
What about precise reachability of configurations?

Is configuration reachability also "almost always" decidable?
No: We will see that configuration reachability is undecidable for reset VASS.

A note on uniformity

- For the coverability problem, we assumed one fixed WSTS.
- In general, we also want to consider the case where the transition system is part of the input.
- This is called a *uniform* problem, because one algorithm works for all transition systems (from a class).

Configuration reachability

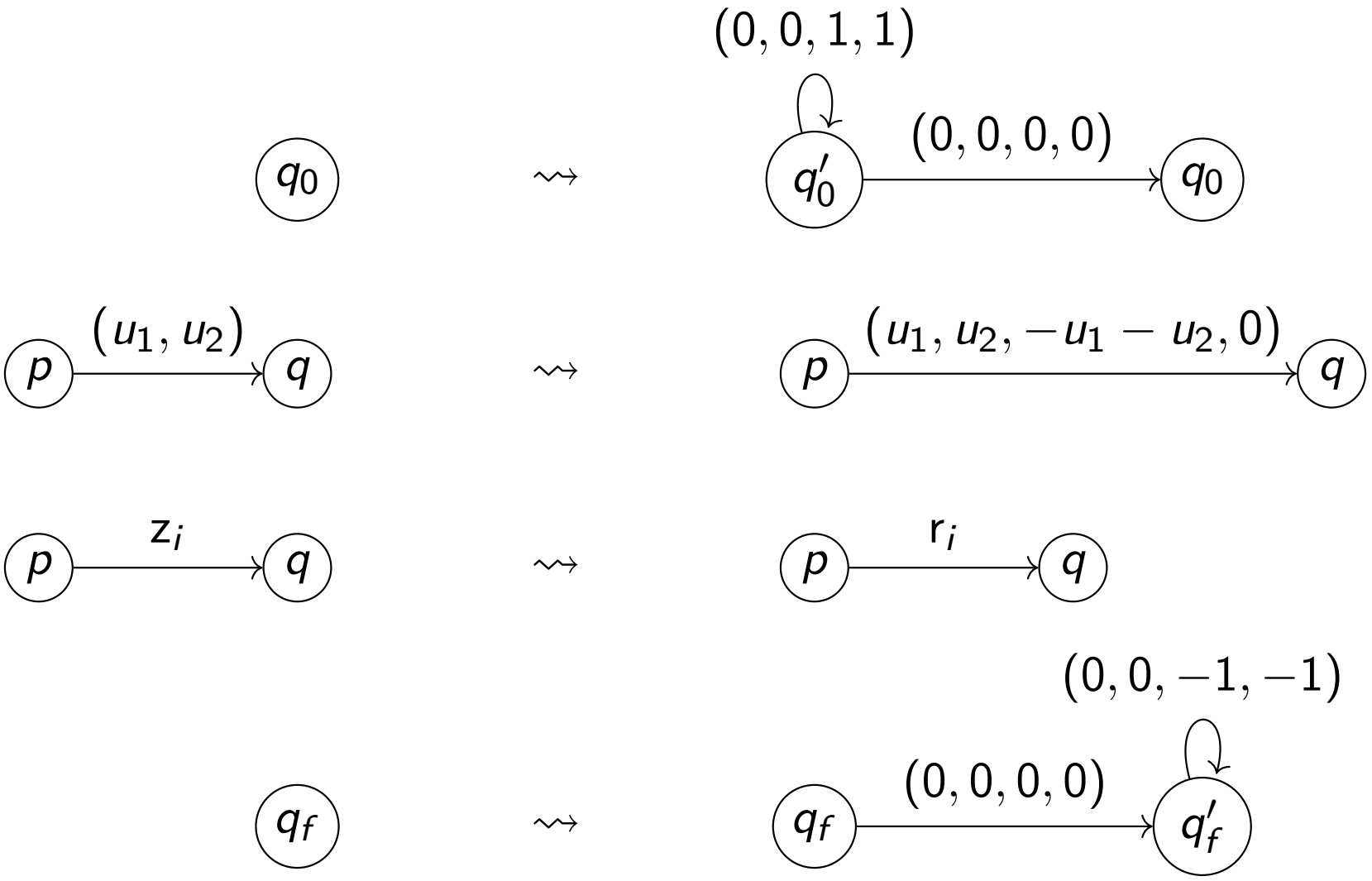
Let $\mathcal{C}$ be a class of transition systems. Examples: $\mathcal{C}$=finite systems, $\mathcal{C}$=VASS, $\mathcal{C}$=reset VASS, $\mathcal{C}$=lossy channel systems.
We assume that each transition system in $\mathcal{C}$ has a finite description.
The *reachability problem for* $\mathcal{C}$:

Given A transition system $(\Gamma, \gamma_0, \rightarrow)$ from $\mathcal{C}$, and $\gamma_1, \gamma_2 \in \Gamma$.

Question Does $\gamma_1 \rightarrow^* \gamma_2$?

Instead of configuration reachability, we also just say *reachability*.

Suppose we are given a two-counter machine (Q, T, q_0) and a state $q_f \in Q$.
We construct a reset 4-VASS by modifying the 2-CM:



Idea: third counter is a "budget", fourth counter remembers this budget.

We have to show two directions:

1. If the Minsky machine can reach $(q_f, 0, 0)$, then the reset VASS can reach $(q'_f, 0, 0, 0, 0)$:
   1.1 We first increment the two new counters to the highest value M used in the run from $(q_0, 0, 0)$ to $(q_f, 0, 0)$.
   1.2 Then we always use the new corresponding transitions: We reach $(q_f, 0, 0, M, M)$.
   1.3 Go to $(q'_f, 0, 0, M, M)$ and use the loop on q'_f to reach $(q'_f, 0, 0, 0, 0)$.
2. If the reset VASS can reach $(q'_f, 0, 0, 0, 0)$, then the Minsky machine can reach $(q_f, 0, 0)$:
   2.1 The run of the reset VASS must visit $(q_f, 0, 0, M, M)$ for some $M \in \mathbb{N}$.
   2.2 In the run to $(q_f, 0, 0, M, M)$, every transition mimics a two-counter transition, except possibly the resets.
   2.3 Suppose a reset is applied to a non-zero counter. Then the sum of the first three counters strictly decreases. However, this sum can never increase. Hence, it is impossible to reach $(q_f, 0, 0, M, M)$. Therefore, a reset r_i can only happen when the counter i is already zero.
   2.4 Hence, the run of the reset VASS corresponds to a run of the Minsky machine. $\square$

Part III

Well-structured transition systems

Section 6

The recurrent state reachability problem

The recurrent state reachability problem

We have seen that the termination problem is "almost always" decidable. What happens if we are looking for infinite runs that perform some action again and again?

Recurrent state reachability

Suppose we have a class $\mathcal{C}$ of systems as before such that all configurations have a finite-state component (as in finite systems, VASS, reset VASS, lossy channel systems).
The *recurrent state reachability problem* is the following:

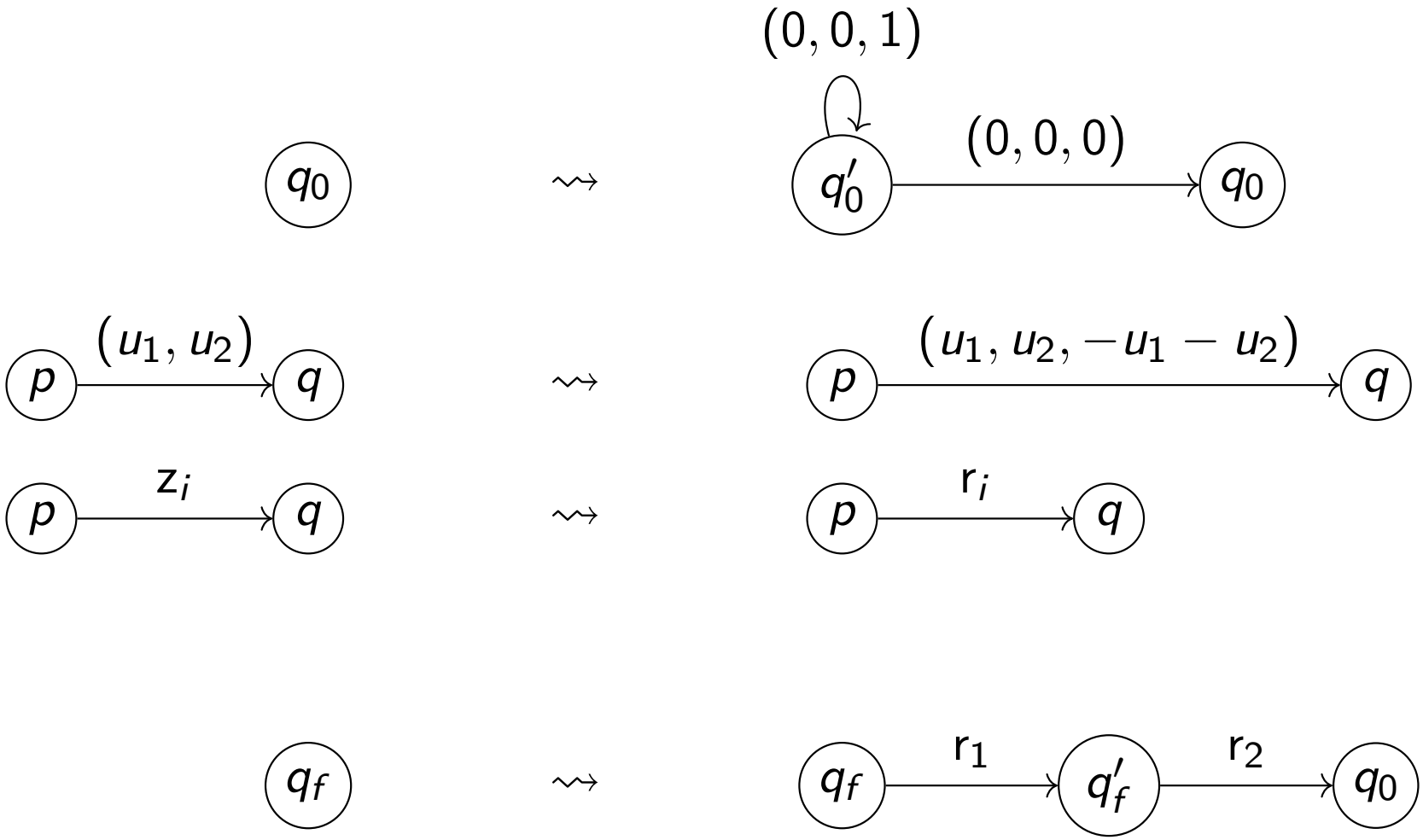
Given A system $\mathcal{S}$ from the class $\mathcal{C}$ and a state q of $\mathcal{S}$.

Question Is there an infinite run that visits q infinitely often?

Theorem

The recurrent state reachability problem is undecidable for the class of reset VASS.

Suppose we are given a Minsky machine (Q, T, q_0) with two counters, and a state $q_f \in Q$.
We construct a reset 3-VASS by modifying the Minsky machine:



We have to show two directions:

1. If the Minsky machine can reach $(q_f, 0, 0)$, then the reset VASS can reach q'_f recurrently:
   - 1.1 We first increment the new counter to the highest value M used in the run from $(q_0, 0, 0)$ to $(q_f, 0, 0)$.
   - 1.2 Then we always use the new corresponding transitions: We reach $(q_f, 0, 0, M)$.
   - 1.3 Go to $(q_0, 0, 0, M)$ and perform the same run again, etc.
2. If the reset VASS can reach q'_f recurrently, then the Minsky machine can reach $(q_f, 0, 0)$:
   - 2.1 The run of the reset VASS must visit q'_f infinitely often. Right afterwards, it enters the configuration $(q_0, 0, 0, n)$, $n \in \mathbb{N}$.
   - 2.2 Suppose a reset is applied to a non-zero counter. Then the sum of the the counters strictly decreases. However, this sum can never increase. Hence, such a mistake can occur only finitely many times during the run. After this, all resets must be performed on zero.
   - 2.3 Thus, eventually, every visit of q'_f follows a visit of $(q_f, 0, 0, n)$ for some $n \in \mathbb{N}$ and is followed by a visit of $(q_0, 0, 0, n)$.
   - 2.4 Moreover, all but finitely many of those phases are from $(q_0, 0, 0, n)$ to $(q_f, 0, 0, n)$ and must be runs of the Minsky machine from $(q_0, 0, 0)$ to $(q_f, 0, 0)$. $\square$

Part III

Well-structured transition systems

Section 7

The boundedness problem

The boundedness problem

For a vector $\mathbf{u} \in \mathbb{Z}^d$ with $\mathbf{u} = (u_1, \ldots, u_d)$, let $\|\mathbf{u}\| := |u_1| + \cdots + |u_d|$ and $\sum \mathbf{u} = u_1 + \cdots + u_d$. By $\mathbf{e}_i$, we denote the vector $(0, \ldots, 0, 1, 0, \ldots, 0)$, where the 1 is at position i.

Bounded reset VASS

A reset VASS (Q, T, q_0) is *bounded* if there is a number $k \in \mathbb{N}$ such that for every configuration $(q, \mathbf{u})$ with $(q_0, \mathbf{0}) \to^* (q, \mathbf{u})$, we have $\|\mathbf{u}\| \leq k$.

The boundedness problem for reset VASS

Given A reset VASS $\mathcal{V} = (Q, T, q_0)$.

Question Is $\mathcal{V}$ bounded?

Theorem (Dufourd, Finkel, Schnoebelen 1997)

The boundedness problem for reset VASS is undecidable.

Here: A simplified version of the proof of Mayr (2003).

S-Deterministic 2CM

An *s-deterministic* (d-)counter machine is a tuple (Q, T, q_0, q_f) where

- (Q, T, q_0) is a d-counter machine where all vectors in transitions are either $\mathbf{e}_i$ or $-\mathbf{e}_i$ for some $i \in [1, d]$,
- $q_f \in Q$ is the *final state*,
- for every $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$ with $q \neq q_f$, there is exactly one $(q', \mathbf{u}') \in Q \times \mathbb{N}^d$ with $(q, \mathbf{u}) \to (q', \mathbf{u}')$,
- for every $(q_f, \mathbf{u})$ with $\mathbf{u} \in \mathbb{N}^d$, there is no $(q', \mathbf{u}')$ with $(q_f, \mathbf{u}) \to (q', \mathbf{u}')$.

Theorem

The following problem is undecidable:

Given *An s-deterministic 2-CM* (Q, T, q_0, q_f).

Question *Is there a* $\mathbf{u} \in \mathbb{N}^d$ *with* $(q_0, \mathbf{0}) \to^* (q_f, \mathbf{u})$*?*

A classic proof of undecidability of 2-CM goes from general Turing machines to general 2-CM. Adapt this proof to use deterministic Turing machines.

Accumulating counter machines

A sequence $x_1, x_2, \ldots$ of numbers $x_i \in \mathbb{N}$ is called *unbounded* if for every $k \in \mathbb{N}$, there is an $i \in \mathbb{N}$ with $x_i \geq k$.
A CM is called *accumulating* if for every infinite run

$$(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \cdots$$

the sequence $\|\mathbf{u}_0\|, \|\mathbf{u}_1\|, \ldots$ is unbounded.

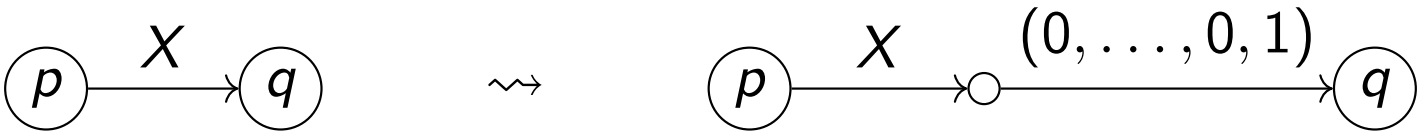
Lemma

The following problem is undecidable:

Given *An accumulating s-deterministic 3-CM* (Q, T, q_0, q_f).

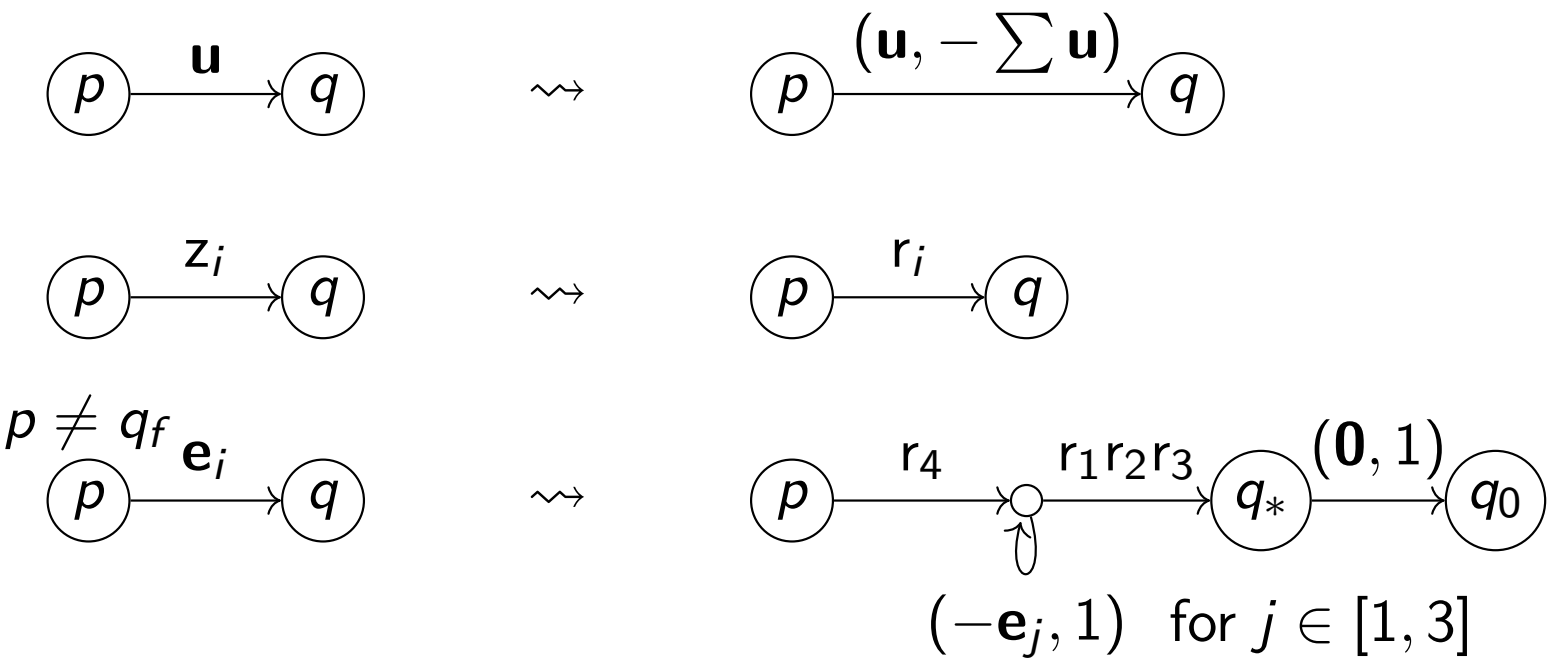
Question *Is there a* $\mathbf{u} \in \mathbb{N}^d$ *with* $(q_0, \mathbf{0}) \to^* (q_f, \mathbf{u})$*?*

Add a counter and modify as follows:



(where X is any operation).

We are given an accum. s-det. 3-CM $\mathcal{M}$.
We construct a reset 4-VASS $\mathcal{V}$ by modifying $\mathcal{M}$:



New counter is budget: increased if there is no run with the current budget.

- If $\mathcal{M}$ cannot reach q_f, it can exceed any budget without visiting q_f. Hence, $\mathcal{V}$ can exceed any bound by visiting q_* again and again.
- If $\mathcal{V}$ is unbounded, it has to visit q_* arbitrarily often. Each time we make vectors larger, all zero tests must be correct. Hence, $\mathcal{M}$ can exceed any budget without visiting q_f.

Detailed proof:

1. Suppose $\mathcal{M}$ cannot reach q_f.
   - 1.1 Since $\mathcal{M}$ is deterministic and cannot reach q_f, every configuration of $\mathcal{M}$ has exactly one successor configuration. Hence, there is an infinite run $(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \cdots$ in $\mathcal{M}$.
   - 1.2 Since $\mathcal{M}$ is accumulating, the sequence $\|\mathbf{u}_0\|, \|\mathbf{u}_1\|, \ldots$ is unbounded.
   - 1.3 We show that $\mathcal{V}$ is unbounded. Let $k \in \mathbb{N}$. Since $\|\mathbf{u}_0\|, \|\mathbf{u}_1\|, \ldots$ is unbounded, there is an $i \in \mathbb{N}$ with $\|\mathbf{u}_i\| \geq k$. We construct a run of $\mathcal{V}$ where the budget counter is $\geq k$:
     - Simulate $\mathcal{M}$ from the start until its counters sum to ≥ 1, then go to q_*, increase budget counter,
     - simulate $\mathcal{M}$ from the start until its counters sum to ≥ 2, then go to q_*, increase budget counter
     - $\ldots$
     - simulate $\mathcal{M}$ until its counters sum to $\geq k$. The budget counter is $\geq k$.
   - 1.4 Hence, $\mathcal{V}$ is unbounded.

2. Suppose $\mathcal{V}$ is unbounded. We think of the counter values as "tokens": If $\|\mathbf{u}\| = k$, then $\mathbf{u}$ has k tokens.
   2.1 We show that for every $k \in \mathbb{N}$, there is a run in $\mathcal{M}$ that ends in some $(q, \mathbf{u})$, $\|\mathbf{u}\| \geq k$. Then, q_f is not reachable: Otherwise, $\mathcal{M}$ has only one finite run, which ends in q_f.
   2.2 Consider a run of $\mathcal{V}$ that ends in some $(q, \mathbf{u})$ with $\|\mathbf{u}\| \geq k$. Note that the only time when $\mathcal{V}$ can increase the number of tokens is going through q_*. Hence, this run must go through q_* at least k times such that a token is created.
   2.3 Decompose the run into *phases* as follows:

$$
\begin{aligned}
&(q_0, \mathbf{v}_{1,0}) \to (q_{1,1}, \mathbf{v}_{1,1}) \to \cdots \to (q_*, \mathbf{v}_{1,n_1}) \to \\
&(q_0, \mathbf{v}_{2,0}) \to (q_{2,1}, \mathbf{v}_{2,1}) \to \cdots \to (q_*, \mathbf{v}_{2,n_2}) \to \\
&\qquad\vdots \\
&(q_0, \mathbf{v}_{m,0}) \to (q_{m,1}, \mathbf{v}_{m,1}) \to \cdots \to (q_*, \mathbf{v}_{m,n_m}) \to \cdots
\end{aligned}
$$

   2.4 In each phase, the number of tokens can increase by at most 1. Hence, there must be a phase where the number of tokens goes from $k-1$ to k. In this phase, all the resets must have been applied to 0: Otherwise, we would lose at least one token during a reset; thus, in the end, we can only have $< k$ tokens.

Thus, this phase corresponds to a run of $\mathcal{M}$ that arrives in a state $p \neq q_f$ with $k-1$ tokens and would have incremented some counter (using $p \xrightarrow{e_i} q$). Hence, in $\mathcal{M}$, this run would have reached configuration with $\geq k$ tokens. $\square$

Part III

Well-structured transition systems

Section 8

Undecidability in Lossy Channel Systems

(Un)decidability in Lossy Channel Systems

Observation

In lossy channel systems, configuration reachability is the same as coverability. Hence, configuration reachability is decidable for lossy channel systems.

Theorem

The recurrent state reachability problem is undecidable for lossy channel systems.

Proof: Reduce from recurrent state reachability of reset VASS. Exercise.

(Un)decidability in Lossy Channel Systems

For a lossy channel system $\mathcal{S} = (Q, C, \Sigma, T, q_0)$ and $\tau \in (\Sigma^*)^C$, we write $|\tau| = \sum_{c \in C} |\tau(c)|$, i.e. the total number of letters in all channels. Moreover, let $\varepsilon \in (\Sigma^*)^C$ be the map with $\varepsilon(c) = \varepsilon$ for every $c \in C$.

We say that $\mathcal{S}$ is *bounded* if there is a $k \in \mathbb{N}$ such that for every configuration (q, τ) with $(q_0, \varepsilon) \to^* (q, \tau)$, we have $|\tau| \leq k$.

Boundedness problem for lossy channel systems

The *boundedness problem for lossy channel systems*:

Given A lossy channel system $\mathcal{S} = (Q, C, \Sigma, T, q_0)$.

Question Is $\mathcal{S}$ bounded?

Theorem

The boundedness problem is undecidable for lossy channel systems.

Proof: Reduce from boundedness of reset VASS. Exercise.

Recap: Decidability and undecidability

1. Finite behavior:
   - 1.1 Coverability:
     decidable for "almost all" WSTS
   - 1.2 Configuration reachability:
     undecidable for reset VASS
     decidable for lossy channel systems
     we will see: decidable for VASS
2. Infinite behavior:
   - 2.1 Termination:
     decidable for "almost all" WSTS
   - 2.2 Recurrent state reachability:
     undecidable for reset VASS and for lossy channel systems
     we will see: decidable for VASS
   - 2.3 Boundedness:
     undecidable for reset VASS and for lossy channel systems
     also: decidable for VASS (exercise)

Part III

Well-structured transition systems

Section 9

Representing downward closed sets

Let $(X, \leq)$ be a WQO. A subset $D \subseteq X$ is *downward closed* if $y \in D$ and $x \leq y$ imply that $x \in D$. For $S \subseteq X$, let $S{\downarrow} = \{x \in X \mid \exists y \in S \colon x \leq y\}$.

- In algorithms that work with upward closed sets, we have represented them using a basis.
- How do we represent downward closed sets?
  - One option: To represent a downward closed D, take a basis for $X \setminus D$. We call this "obstruction sets".
  - This is cumbersome: e.g. how to check "is D infinite?"
- More useful: ideals.

Definition

A subset $S \subseteq X$ is *(upward) directed* if for every $x, y \in S$, there exists $z \in S$ with $x \leq z$ and $y \leq z$. A non-empty subset $I \subseteq X$ is an *ideal* if

1. I is downward closed and
2. I is upward directed.

A set X is *countable* if there is a sequence $x_1, x_2, \ldots \in X$ such that $X = \{x_1, x_2, \ldots\}$.

Theorem

Let $(X, \leq)$ be a WQO such that X is countable. For every subset $I \subseteq X$, the following are equivalent:

(1) *I is an ideal.*

(2) *I is downward closed, non-empty, and for any two downward closed sets $D_1, D_2 \subseteq X$, we have: $I \subseteq D_1 \cup D_2$ implies that $I \subseteq D_1$ or $I \subseteq D_2$.*

(3) *there is an ascending chain $y_1 \leq y_2 \leq \cdots \in I$ such that $I = \{y_1, y_2, \ldots\}{\downarrow}$.*

Lemma ((1) $\Rightarrow$ (2))

Let $(X, \leq)$ be a WQO and let I be an ideal. Moreover, let $D_1, D_2 \subseteq X$ be downward closed subsets such that $I \subseteq D_1 \cup D_2$. Then $I \subseteq D_1$ or $I \subseteq D_2$.

Proof.

Suppose $I \not\subseteq D_1$ and $I \not\subseteq D_2$. Then there is an $x_1 \in I \setminus D_1$ and an $x_2 \in I \setminus D_2$. Since I is upward directed, there is a $y \in I$ with $x_1 \leq y$ and $x_2 \leq y$. Since $I \subseteq D_1 \cup D_2$, we have either $y \in D_1$ or $y \in D_2$. In the former case, we have $x_1 \in D_1$; in the latter case, we have $x_2 \in D_2$. Both are impossible. Contradiction! $\square$

Lemma ((2) $\Rightarrow$ (1))

Let $(X, \leq)$ be a WQO. Suppose that $I \neq \emptyset$ is downward closed and for any downward closed sets D_1, D_2, we have that $I \subseteq D_1 \cup D_2$ implies $I \subseteq D_1$ or $I \subseteq D_2$. Then I is an ideal.

Proof.

Since I is downward closed, it remains to be shown that I is upward directed. Suppose I is not upward directed. Then there are $x_1, x_2 \in I$ such that there is no $y \in I$ with $x_1 \leq y$ and $x_2 \leq y$. In particular, $x_1 \not\leq x_2$ and $x_2 \not\leq x_1$. Consider the following subsets:

$$D_1 = \{x \in I \mid x_1 \not\leq x\}, \quad D_2 = \{x \in I \mid x_2 \not\leq x\}.$$

Then $I \subseteq D_1 \cup D_2$ because there is no y as above.
By assumption, we have $I \subseteq D_1$ or $I \subseteq D_2$.
In the first case, we have $x_1 \in D_1$, in the second case, we have $x_2 \in D_2$. Contradiction! $\square$

Lemma ((3) $\Rightarrow$ (1))

Let $(X, \leq)$ be a WQO and let $y_1 \leq y_2 \leq \cdots$ be an ascending chain in X. Then $\{y_1, y_2, \ldots\}{\downarrow}$ is an ideal.

Proof.

Clearly, the set $I = \{y_1, y_2, \ldots\}{\downarrow}$ is downward closed and non-empty. It remains to show that I is upward directed. So let $x_1, x_2 \in I$. Then there are y_i, y_j with $x_1 \leq y_i$ and $x_2 \leq y_j$. Set $m = \max\{i, j\}$. Since $y_1 \leq y_2 \leq \cdots$, we have $y_i, y_j \leq y_m$ and thus $x_1, x_2 \leq y_m$.
Hence, I is upward directed. $\square$

Lemma ((1) $\Rightarrow$ (3))

Let $(X, \leq)$ be a WQO such that X is countable. Moreover, let $I \subseteq X$ be an ideal. Then there exists an ascending chain $y_1 \leq y_2 \leq \cdots \in I$ such that $I = \{y_1, y_2, \ldots\}{\downarrow}$.

Proof.

Since X is countable, I is countable as well: There is a sequence $x_1, x_2, \ldots$ such that $I = \{x_1, x_2, \ldots\}$.
Construct $y_1, y_2, \ldots$ successively as follows:

1. $y_1 = x_1$.
2. Suppose we have $y_1, \ldots, y_i$. Since I is upward directed, there is some $y \in I$ with $y_i \leq y$ and $x_{i+1} \leq y$. Take $y_{i+1} := y$.

We claim that $I = \{y_1, y_2, \ldots\}{\downarrow}$.
Since $y_i \in I$ for every $i \in \mathbb{N}$, we clearly have "$\supseteq$".
Moreover, if $x \in I$, then $x = x_i$ for some $i \in \mathbb{N}$.
Then we have $x_i \leq y_i$ and thus $x = x_i \in \{y_1, y_2, \ldots\}{\downarrow}$. $\square$

This completes the proof of the theorem!

Ideal decompositions

Theorem

Let $(X, \leq)$ be a WQO. For every downward closed set $D \subseteq X$, there exist $n \in \mathbb{N}$ and ideals $I_1, \ldots, I_n$ such that $D = I_1 \cup \cdots \cup I_n$.

Examples:

- Consider $D = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid \mathtt{aba} \not\preccurlyeq w \text{ and } \mathtt{bab} \not\preccurlyeq w\}$.
  Note that D is not an ideal, because $\mathtt{ab} \in D$ and $\mathtt{ba} \in D$, but there is no $w \in D$ with $\mathtt{ab} \preccurlyeq w$ and $\mathtt{ba} \preccurlyeq w$.
  However, $D = \{\mathtt{a}\}^*\{\mathtt{b}\}^* \cup \{\mathtt{b}\}^*\{\mathtt{a}\}^*$.
  Notice that both $\{\mathtt{a}\}^*\{\mathtt{b}\}^*$ and $\{\mathtt{b}\}^*\{\mathtt{a}\}^*$ are ideals.
- Consider $D = \{(x, y) \in \mathbb{N}^2 \mid x = 0 \text{ or } y = 0\}$.
  Then D is not an ideal: $(0, 1) \in D$ and $(1, 0) \in D$, but there is no $(x, y) \in D$ with $(0, 1) \leq (x, y)$ and $(1, 0) \leq (x, y)$.
  However $D = \{0\} \times \mathbb{N} \ \cup \ \mathbb{N} \times \{0\}$.
  Notice that both $\{0\} \times \mathbb{N}$ and $\mathbb{N} \times \{0\}$ are ideals.

The first step to proving the theorem is the following lemma:

Lemma

Let $(X, \leq)$ be a WQO and $D \subseteq X$ be a non-empty downward closed set that is not an ideal. Then there are downward closed $D_1, D_2 \subseteq X$ with $D = D_1 \cup D_2$ and $D_i \subsetneq D$ for $i = 1, 2$.

Proof.

Since D is non-empty and not an ideal, it is not upward directed. Hence, there are $x_1, x_2 \in D$ that have no common upper bound in D. Consider

$$D_1 = \{y \in D \mid x_1 \not\leq y\}, \quad D_2 = \{y \in D \mid x_2 \not\leq y\}$$

Since x_1, x_2 have no common upper bound in D, we have $D = D_1 \cup D_2$. Moreover, D_1, D_2 are downward closed and $D_i \subsetneq D$ (because $x_i \notin D_i$) for $i = 1, 2$. $\square$

Proof of the theorem.

If $D = \emptyset$, we are done. Hence, let $D \subseteq X$ be non-empty.
We construct a tree whose nodes are labeled with non-empty downward closed sets:

1. The root node is labeled with D.
2. We distinguish two cases:
   - If all leaves are labeled with ideals: stop.
   - If there is a leaf v labeled by some $E \subseteq X$ that is not an ideal, use the lemma to write $E = E_1 \cup E_2$ with $E_i \subsetneq E$ for $i = 1, 2$. Add two children labeled E_1 and E_2 to v.

Observe:

- In each step, if $E_1, \ldots, E_n$ are the labels of all leaves, then $D = E_1 \cup \cdots \cup E_n$.
- The process has to terminate: Otherwise, Kőnig's Lemma guarantees an infinite branch. This branch contains a sequence $F_1 \supsetneq F_2 \supsetneq \cdots$ of downward closed sets, which cannot exist.
- If the process stops, all leaves are labeled with ideals. $\square$

Unique ideal decompositions

- Now, we know that each downward closed set can be written as the union of ideals.
- To compute with these representations, we want them to be unique.
- To get uniqueness, we need a particular kind of ideal:

Definition

Let $D \subseteq X$ be downward closed. An ideal $I \subseteq D$ is called *maximal in D* if for every ideal J with $I \subseteq J \subseteq D$, we have $I = J$.

With maximal ideals, we have unique ideal decompositions:

Corollary

For every downward closed set $D \subseteq X$, we can write $D = I_1 \cup \cdots \cup I_n$, where $I_1, \ldots, I_n$ are exactly the maximal ideals in D. In particular, this decomposition is unique up to re-ordering of the ideals.

Proof of the corollary.

By the theorem, we have $D = J_1 \cup \cdots \cup J_m$ for some ideals $J_1, \ldots, J_m$.
Of the ideals $J_1, \ldots, J_m$, pick those that are maximal among $J_1, \ldots, J_m$: Keep J_i if there is no J_k with $J_i \subsetneq J_k$.
Let the picked set of ideals be $I_1, \ldots, I_n$.
We claim that $I_1, \ldots, I_n$ are exactly the maximal ideals in D.
Exercise! $\square$

Ideals in $\mathbb{N}^k$

How do we represent ideals? Define

$$\mathbb{N}_\omega = \mathbb{N} \cup \{\omega\}.$$

The ω stands for "above all natural numbers". Write $\mathbb{N}_\omega^k = (\mathbb{N}_\omega)^k$.
We will see that ideals in $\mathbb{N}^k$ can be represented by vectors in $\mathbb{N}_\omega^k$.
We extend the ordering $\leq$ from natural numbers to $\mathbb{N}_\omega$:

$$n \leq \omega \quad \text{for every } n \in \mathbb{N}, \qquad \omega \leq \omega.$$

We also extend $\leq$ to $\mathbb{N}_\omega^k$ component-wise.
Given $\mathbf{u} \in \mathbb{N}_\omega^k$, define

$$\mathsf{Idl}(\mathbf{u}) = \{\mathbf{v} \in \mathbb{N}^k \mid \mathbf{v} \leq \mathbf{u}\}$$

Proposition

The ideals in $\mathbb{N}^k$ are exactly the sets of the form $\mathsf{Idl}(\mathbf{u})$ for $\mathbf{u} \in \mathbb{N}_\omega^k$.

Proof of the proposition.

- Every set $\mathrm{Idl}(\mathbf{u})$ for $\mathbf{u} \in \mathbb{N}_\omega^k$ is an ideal: They are clearly downward closed. To show upward directedness, consider $\mathbf{v}', \mathbf{v}'' \in \mathrm{Idl}(\mathbf{u})$. We construct $\mathbf{v} \in \mathrm{Idl}(\mathbf{u})$, $\mathbf{v} = (v_1, \ldots, v_k)$, with $\mathbf{v}' \leq \mathbf{v}$ and $\mathbf{v}'' \leq \mathbf{v}$. For each component $i = 1, \ldots, k$ set v_i to the maximum of $\mathbf{v}'$'s and $\mathbf{v}''$'s i-th component. Then clearly, $\mathbf{v}' \leq \mathbf{v}, \mathbf{v}'' \leq \mathbf{v}$ and $\mathbf{v} \in \mathrm{Idl}(\mathbf{u})$.
- Every ideal is of the form $\mathrm{Idl}(\mathbf{u})$: Consider an ideal $I \subseteq \mathbb{N}^k$. Then there is a sequence $\mathbf{v}_1 \leq \mathbf{v}_2 \leq \cdots \in I$ such that $I = \{\mathbf{v}_1, \mathbf{v}_2, \ldots\}\downarrow$. For $i = 1, \ldots, k$, construct $\mathbf{u} \in \mathbb{N}_\omega^k$, $\mathbf{u} = (u_1, \ldots, u_k)$, as follows:
  - If the i-th component of $\mathbf{v}_1, \mathbf{v}_2, \ldots$ is unbounded, then set $u_i = \omega$.
  - If the i-th component of $\mathbf{v}_1, \mathbf{v}_2, \ldots$ has a maximal value $m \in \mathbb{N}$, then set $u_i = m$.

  Then clearly, $\{\mathbf{v}_1, \mathbf{v}_2, \ldots\}\downarrow = \mathrm{Idl}(\mathbf{u})$. $\square$

Ideals in Σ^*

Let Σ be an alphabet.
We want to understand the ideals of $(\Sigma^*, \preccurlyeq)$.

- An *atom* is either
  - a set $\{a, \varepsilon\}$ with some $a \in \Sigma$
  - or Δ^* for some $\Delta \subseteq \Sigma$.
- A *product* is a concatenation $A_1 \cdots A_n$, where each A_i is an atom.

Proposition

The ideals in $(\Sigma^*, \preccurlyeq)$ are exactly the products.

Easy to see: Every product is an ideal.
We will now see that every ideal is a product.

Definition

A *(non-deterministic) finite automaton (NFA)* is a tuple $\mathcal{A} = (Q, \Sigma, T, q_0, F)$, where

- Q is a finite set of *states*,
- Σ is a finite *alphabet*,
- $T \subseteq Q \times \Sigma \times Q$ is a finite set of *transitions*,
- $q_0 \in Q$ is its *initial state*,
- $F \subseteq Q$ is a set of *final states*.

We write $p \xrightarrow{w} q$ if there are states $p_0, \ldots, p_n$ with $p = p_0$, $q = p_n$ and letters $a_1, \ldots, a_n$ with $w = a_1 \cdots a_n$ and a transition $(p_{i-1}, a_i, p_i) \in T$ for each $i = 1, \ldots, n$. We define

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q \text{ for some } q \in F\}.$$

Definition

An NFA (Q, Σ, T, q_0, F) is called *partially ordered* if there is a partial order $(Q, \leq)$ such that for every transition $(p, a, q) \in T$, we have $p \leq q$.

Lemma 1

If $D \subseteq \Sigma^*$ is downward closed, then there is a partially ordered NFA $\mathcal{A}$ with $L(\mathcal{A}) = D$.

Proof of Lemma 1

Let $D \subseteq \Sigma^*$ be downward closed and $D \neq \emptyset$.
Then there are $\{w_1, \ldots, w_n\}$ such that $D = \Sigma^* \setminus \{w_1, \ldots, w_n\}\uparrow$.
Construct $\mathcal{A}$ as follows:

- $Q = P_1 \times \cdots \times P_n$, $P_i = \{w \in \Sigma^* \mid w \text{ is a prefix of } w_i\}$.
- $q_0 = (\varepsilon, \ldots, \varepsilon)$.
- For $q = (p_1, \ldots, p_n) \in Q$ and $a \in \Sigma$, introduce transition (q, a, q'), where $q' = (p'_1, \ldots, p'_n)$ such that for each $i = 1, \ldots, n$:

$$p'_i = \begin{cases} p_i a & \text{if } p_i a \text{ is a prefix of } w_i \\ p_i & \text{otherwise} \end{cases}$$

- Observe: if $q_0 \xrightarrow{w} (p_1, \ldots, p_n)$, then for each i: p_i is the longest prefix p of w_i such that $p \preccurlyeq w$.
- $F = \{(p_1, \ldots, p_n) \in Q \mid |p_i| < |w_i| \text{ for all } i\}$.
- $L(\mathcal{A}) = D$.
- As partial order, take

$$(p_1, \ldots, p_n) \leq (p'_1, \ldots, p'_n) \iff p_i \text{ is a prefix of } p'_i \text{ for every } i \qquad \square$$

Lemma 2

Let $\mathcal{A} = (Q, \Sigma, T, q_0, F)$ be a partially ordered NFA. Then $L(\mathcal{A})$ is a finite union of sets of the form $Y_0^* x_1 Y_1^* x_2 Y_2^* \cdots x_n Y_n^*$ for letters $x_1, \ldots, x_n \in \Sigma$ and alphabets $Y_0, \ldots, Y_n \subseteq \Sigma$.

Let $(Q, \leq)$ be the partial order. Induction on $|Q|$: Suppose the statement is true for every $\mathcal{A} = (Q, \Sigma, T, q_0, F)$ with $|Q| \leq k$.

- Without loss of generality, assume that q_0 is minimal w.r.t. $\leq$.
- Let $(q_0, a_1, q_0), \ldots, (q_0, a_\ell, q_0)$ be the self-loops in q_0.
- Let $(q_0, b_1, r_1), \ldots, (q_0, b_m, r_m)$ be the transitions starting in q_0 that are not self-loops.
- For $i = 1, \ldots, m$, let $\mathcal{A}_i$ be the automaton obtained from $\mathcal{A}$ by removing q_0 and making r_i initial. Then $\mathcal{A}_i$ is again partially ordered and has k states.
- By induction, $L(\mathcal{A}_i)$ is a finite union $L(\mathcal{A}_i) = \bigcup_{j=1}^{s_i} K_{i,j}$ of languages $K_{i,j}$ as above. Observe:
$$L(\mathcal{A}) = \bigcup_{i=1}^{m} Y^* b_i L(\mathcal{A}_i) = \bigcup_{i=1}^{m} \bigcup_{j=1}^{s_i} Y^* b_i K_{i,j} \quad \text{with } Y = \{a_1, \ldots, a_\ell\}.$$
- If $q_0 \in F$ then we have to add Y^* to this union. $\square$

We are now ready to prove:

Proposition

The ideals in $(\Sigma^*, \preccurlyeq)$ are exactly the products.

It remains to be shown: Every ideal is a product.
Let $I \subseteq \Sigma^*$ be an ideal. Since I is downward closed, Lemma 1 says that $I = L(\mathcal{A})$ for some partially ordered NFA $\mathcal{A}$.
By Lemma 2, we then have

$$I = L(\mathcal{A}) = \bigcup_{i=1}^{m} Y_{i,0}^* x_{i,1} Y_{i,1}^* \cdots x_{i,s_i} Y_{i,s_i}^*$$

for some letters $x_{i,j} \in \Sigma$ and alphabets $Y_{i,j} \subseteq \Sigma$. Since I is downward closed, we have

$$I = I{\downarrow} = \bigcup_{i=1}^{m} \underbrace{Y_{i,0}^* \{x_{i,1}, \varepsilon\} Y_{i,1}^* \cdots \{x_{i,s_i}, \varepsilon\} Y_{i,s_i}^*}_{\text{downward closed}}.$$

Since I is an ideal, we have $I \subseteq Y_{i,0}^* \{x_{i,1}, \varepsilon\} Y_{i,1}^* \cdots \{x_{i,s_i}, \varepsilon\} Y_{i,s_i}^*$ for some i. But then $I = Y_{i,0}^* \{x_{i,1}, \varepsilon\} Y_{i,1}^* \cdots \{x_{i,s_i}, \varepsilon\} Y_{i,s_i}^*$. $\square$

We have seen two proofs for ideal representations:

- Ideals in $\mathbb{N}^k$ are of the form $\mathrm{Idl}(\mathbf{u})$ for $\mathbf{u} \in \mathbb{N}^k_\omega$.
- Ideals in $(\Sigma^*, \preccurlyeq)$ are products of atoms.

Note:

- The proof for $(\Sigma^*, \preccurlyeq)$ is constructive: Given a set $\{w_1, \ldots, w_m\}$, it allows to construct a union of products that represents $\Sigma^* \setminus \{w_1, \ldots, w_m\}\uparrow$.
- The first was non-constructive: Not clear how to compute with infinite sequence.

However, we can use the algorithm for the second proof to compute ideal representations for $\mathbb{N}^k$.

Conclusion

For $(X, \leq) = (\mathbb{N}^k, \leq)$ or $(X, \leq) = (\Sigma^*, \preccurlyeq)$, downward closed sets can be represented either

- by finite unions of ideals
- by obstruction sets

And the two representations can be translated into each other.

From Obstruction sets to Ideals in $\mathbb{N}^k$

$$O = \{(2,3),(4,2)\}$$

$$(x,y) \in D \iff \neg((x \geq 2 \wedge y \geq 3) \vee (x \geq 4 \wedge y \geq 2))$$
$$(x,y) \in D \iff (x \leq 1 \vee y \leq 2) \wedge (x \leq 3 \vee y \leq 1)$$
$$(x,y) \in D \iff (x \leq 1 \vee (y \leq 2 \wedge x \leq 3) \vee y \leq 1)$$
$$D = (1,\omega)\downarrow \cup (3,2)\downarrow \cup (\omega,1)\downarrow$$

From Ideals to Obstruction sets in $\mathbb{N}^k$

$$D = (2, \omega, \omega)\downarrow \cup (\omega, 2, 1)\downarrow$$

O consists of the minimal elements in the set
$\{(x, y, z) \mid (x \geq 3 \wedge (y \geq 3 \vee z \geq 2))\}$
$O = \{(3, 3, 0), (3, 0, 2)\}$

Part III

Well-structured transition systems

Section 10

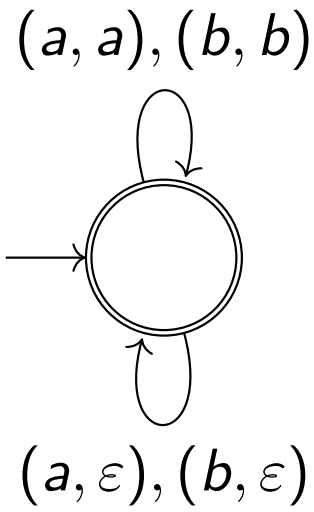
Downclosures for Full Trios

Computing downclosures

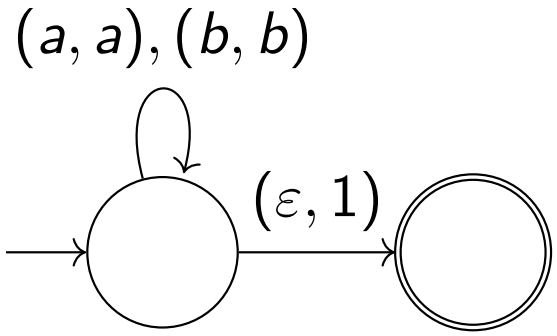
- We have seen that the downclosure of **any** language under the subword order is regular.
- Thus $L{\downarrow}$ forms an overapproximation of L and is a simpler language.
- In fact, one can show that no information is lost by using $L{\downarrow}$ instead of L for *safety verification*.
- We will see that under some mild assumptions of closure on a language class $\mathcal{C}$, computing downclosures can be reduced to deciding a simpler problem.

Rational Transductions

An *asychronous* transducer is a machine $T = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where:
Q is a finite state set, Σ the finite input alphabet, Γ the finite output alphabet, $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ a set of transitions, $q_0 \in Q$ the initial state and $F \subseteq Q$ a set of final states.
The transducer accepts a subset of $\Sigma^* \times \Gamma^*$, denoted $L(T)$.



$L(T_1) = \{(u, v) \mid v \preccurlyeq u\}$



$L(T_2) = \{(u, v) \mid v = u1\}$

Transduction of a language

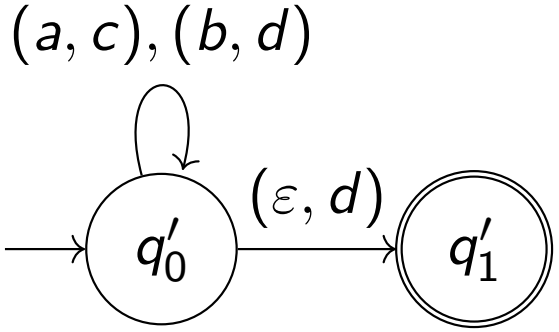
Given a language $K \subseteq \Sigma^*$ and a transducer T, $TK \subseteq \Gamma^*$ is defined to be the language obtained by application of T on K.
$TK = \{v \in \Gamma^* \mid \exists u \in K \colon (u, v) \in L(T)\}$.
Clearly, when $\Sigma = \{a, b\}$ and $T = T_1$ in our first example, then $T_1 K = K{\downarrow}$.

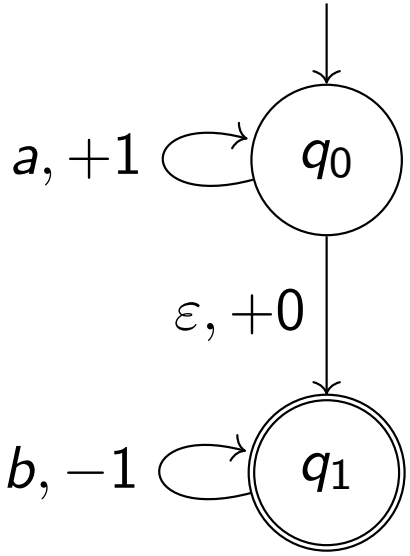
Definition

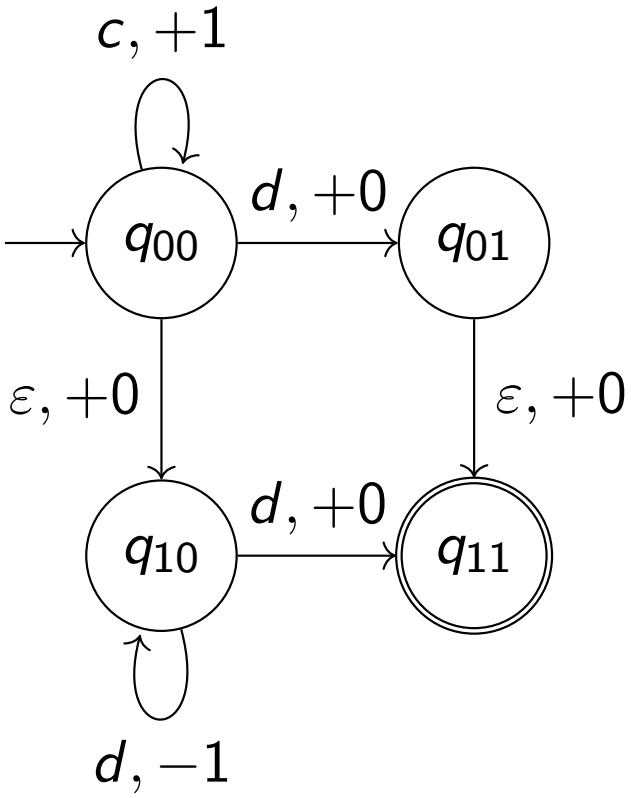
A language class $\mathcal{C}$ is said to be *closed under rational transductions* if for every $K \in \mathcal{C}$ and every asynchronous transducer T, we have $TK \in \mathcal{C}$.

A language class closed under rational transductions is also called a **full trio**. Additionally, we will also assume that it is *effectively* closed, i.e., given T and $K \in \mathcal{C}$, we can compute a representation for TK.
Language classes which are defined by machine models using a finite control and a nondeterministic data structure(s) are usually full trios, e.g., VASS coverability languages.

Example







$L(\mathcal{V}) = \{a^m b^n \mid m \geq n\}$

$L(\mathcal{V}') = \{c^m d^{n+1} \mid m \geq n\} = TL(\mathcal{V})$

Nivat's Theorem

A homomorphism $h\colon \Sigma^* \to \Gamma^*$ is a map that satisfies $h(uv) = h(u)h(v)$.
E.g. let $h\colon \{a,b\}^* \to \{a,b\}^*$ be defined by $h(a) = bb$, $h(b) = \varepsilon$.
Let $K = a^*b^*$, then $h(K) = \{b^{2n} \mid n \in \mathbb{N}\}$.
$h^{-1}(bb) = b^*ab^*$.

Theorem (Nivat, 1968)

A class of languages $\mathcal{C}$ is closed under rational transductions (i.e., $\mathcal{C}$ is a full trio) if, and only if, it is closed under each of the following operations:

1. *intersection with a regular language*
2. *homomorphism*
3. *inverse homomorphism*

Note: $\mathcal{C}$ is effectively closed under rational transductions iff the listed closure properties are effective.

Simultaneous Unboundedness Problem

SUP for language class $\mathcal{C}$

Given Language $K \in \mathcal{C}, K \subseteq a_1^* a_2^* \dots a_n^*$, where each a_i is a letter.

Question Is $K{\downarrow} = a_1^* a_2^* \dots a_n^*$?

The name of the problem comes from the fact that $K{\downarrow} = a_1^* a_2^* \dots a_n^*$ iff for any $k \in \mathbb{N}$, there exists a word $a_1^{x_1} a_2^{x_2} \dots a_n^{x_n} \in K$ such that each $x_i \geq k$.

Theorem (Equivalence of downclosures and SUP)

For any effective full trio $\mathcal{C}$, the following two statements are equivalent:

(1) *From $K \in \mathcal{C}$, we can compute an NFA accepting $K{\downarrow}$.*

(2) *SUP is decidable for $\mathcal{C}$.*

Proof

(1) $\Rightarrow$ (2): Suppose downclosures are computable for $\mathcal{C}$, then clearly, we can take the NFA $\mathcal{A}$ produced by the algorithm and check if $L(\mathcal{A}) = a_1^* a_2^* \dots a_n^*$.
(2) $\Rightarrow$ (1): First, we show emptiness of a language can be reduced to SUP.

Emptiness problem for $\mathcal{C}$

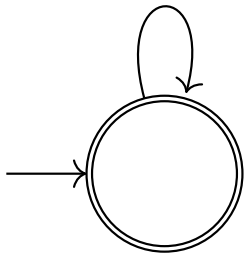
Given	Language $K \in \mathcal{C}$.
Question	Is $K = \emptyset$?

Let $K \subseteq \Sigma^*$ and define T with $L(T) = \Sigma^* \times \{a\}^*$. By assumption, TK can be computed and is a subset of a^*. Observe that $K = \emptyset$ iff $TK \neq a^*$.
Now, let K be a language of class $\mathcal{C}$. The language $K{\downarrow}$ is described by a finite union of products (UOP) as we have seen. For each UOP r, check whether $L(r) = K{\downarrow}$ as described on the following slides until we find the correct r.

Checking "$K{\downarrow} \subseteq L(r)$"

Recall the following transducer T:

$$(a, a), (a, \varepsilon) \text{ f.a. } a \in \Sigma$$



We have $L(T) = \{(u, v) \mid v \preccurlyeq u\}$. Then $TK = K{\downarrow}$, i.e., $K{\downarrow}$ is in $\mathcal{C}$. Since $\Sigma^* \setminus L(r)$ is regular, the language $K{\downarrow} \cap (\Sigma^* \setminus L(r))$ is also in $\mathcal{C}$ (Nivat's Theorem). Since the emptiness problem is decidable for $\mathcal{C}$, we can finally check $K{\downarrow} \cap (\Sigma^* \setminus L(r)) = \emptyset$ which is equivalent to $K{\downarrow} \subseteq L(r)$.

Checking "$L(r) \subseteq K{\downarrow}$" (1/2)

$L(r)$ is a finite union of languages of the form
$\{w_0\}{\downarrow} \cdot Y_1^* \cdot \{w_1\}{\downarrow} \cdot Y_2^* \cdots Y_n^* \cdot \{w_n\}{\downarrow}$.
We check if each $\{w_0\}{\downarrow} \cdot Y_1^* \cdot \{w_1\}{\downarrow} \cdot Y_2^* \cdots Y_n^* \cdot \{w_n\}{\downarrow} \subseteq K{\downarrow}$.
To decide this inclusion, we use the following observation:

Observation

Let $\Gamma = \{y_1, y_2, \ldots, y_n\} \subseteq \Sigma$ and $L \subseteq \Sigma^*$ be downward closed.
Then $\Gamma^* \subseteq L$ if, and only if, for each $k \in \mathbb{N}$ there is $\ell \geq k$ with $(y_1 y_2 \ldots y_n)^\ell \in L$.

Let $u_i = y_1 y_2 \ldots y_m$, where $Y_i = \{y_1, y_2, \ldots, y_m\}$.
Due to the observation it suffices to check whether for any $k \in \mathbb{N}$, there are numbers $x_1, x_2, \ldots, x_n \in \mathbb{N}$ with each $x_i \geq k$ such that

$$w_0 u_1^{x_1} w_1 u_2^{x_2} w_2 \ldots u_n^{x_n} w_n \in K{\downarrow}.$$

Checking "$L(r) \subseteq K{\downarrow}$" (2/2)

Define a transducer T with

$$L(T) = \{(w_0 u_1^{x_1} w_1 u_2^{x_2} w_2 \dots u_n^{x_n} w_n, a_1^{x_1} a_2^{x_2} \dots a_n^{x_n}) \mid \forall i\ x_i \geq 0\}.$$

The following properties hold:

- $T(K{\downarrow}) = \{a_1^{x_1} a_2^{x_2} \dots a_n^{x_n} \mid w_0 u_1^{x_1} w_1 u_2^{x_2} w_2 \dots u_n^{x_n} w_n \in K{\downarrow}\}$
- $T(K{\downarrow}){\downarrow} = a_1^* a_2^* \dots a_n^*$ iff $w_0 Y_1^* w_1 Y_2^* w_2 \dots Y_n^* w_n \subseteq K{\downarrow}$.

Since SUP is decidable for $\mathcal{C}$, we can decide $T(K{\downarrow}){\downarrow} = a_1^* a_2^* \dots a_n^*$ and therefore $w_0 Y_1^* w_1 Y_2^* w_2 \dots Y_n^* w_n \subseteq K{\downarrow}$. $\square$

Part IV

Vector addition systems

Section 11

Karp-Miller trees

Coverability in vector addition systems

In the following, by "computing a downward closed set" we always mean computing its unique ideal decomposition.

- Coverability is decidable "almost always" for WSTS
- The set of coverable configurations is downward closed
- Does this mean we can always compute this set?

Observation

Boundedness is undecidable for reset VASS. Therefore, we cannot compute the set of coverable configurations.

Otherwise, we could decide boundedness by checking if ω appears in the ideals.

Karp-Miller trees: Motivation

For a d-VASS $\mathcal{V} = (Q, T, q_0)$, define

$$\mathrm{Reach}(\mathcal{V}) = \{(q, \mathbf{v}) \in Q \times \mathbb{N}^d \mid (q_0, \mathbf{0}) \to^* (q, \mathbf{v})\}$$

This is called the *reachability set* of $\mathcal{V}$. The set

$$\mathrm{Cover}(\mathcal{V}) = \mathrm{Reach}(\mathcal{V})\downarrow$$

is called the *coverability set* of $\mathcal{V}$.

We will now see

For VASS, the coverability set can be computed.

- We will use *Karp-Miller trees* to find an ideal decomposition of $\mathrm{Cover}(\mathcal{V})$.
- We also obtain a second algorithm for the coverability problem in VASS: for a given VASS $\mathcal{V} = (Q, T, q_0)$ and a configuration $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$, compute $\mathrm{Cover}(\mathcal{V})$ and check afterwards $(q, \mathbf{u}) \in \mathrm{Cover}(\mathcal{V})$.

Addition with ω-vectors

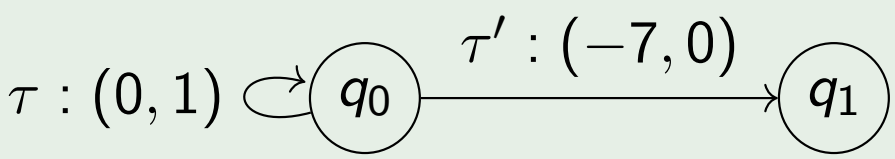
Suppose $\mathcal{V} = (Q, T, q_0)$ is a d-VASS. Then its configurations belong to $\mathbb{N}^d$. In the following algorithm, it will be useful, to consider $\mathcal{V}$ as operating with markings in $\mathbb{N}_\omega^d$:

- If we have $(q, \mathbf{u}) \to (q', \mathbf{u}')$ with a transition $\tau = (q, \mathbf{v}, q')$, then we write $(q, \mathbf{u}) \xrightarrow{\tau} (q', \mathbf{u}')$.
- For $n \in \mathbb{N}$, define $\omega + n := \omega$ and $\omega - n := \omega$.
- Analogously, for $\mathbf{u} \in \mathbb{N}_\omega^d$ and $\mathbf{v} \in \mathbb{N}^d$, we define $\mathbf{u} + \mathbf{v} \in \mathbb{N}_\omega^d$ with component-wise addition.

Firing transitions on ω-vectors

With this, it makes sense to write $(q, \mathbf{u}) \xrightarrow{\tau} (q', \mathbf{u}')$ with $\mathbf{u}, \mathbf{u}' \in \mathbb{N}_\omega^d$ for transitions $\tau \in T$.

Example



We have $(q_0, (\omega, 0)) \xrightarrow{\tau} (q_0, (\omega, 1))$ and $(q_0, (\omega, 0)) \xrightarrow{\tau'} (q_1, (\omega, 0))$, but $(q_0, (5, 0)) \not\xrightarrow{\tau'} (q_1, (5, 0))$.

$\tau \in T$ is *activated in* $(q, \mathbf{u}) \in Q \times \mathbb{N}_\omega^d$ if there is $(q', \mathbf{u}') \in Q \times \mathbb{N}_\omega^d$ with $(q, \mathbf{u}) \xrightarrow{\tau} (q', \mathbf{u}')$.

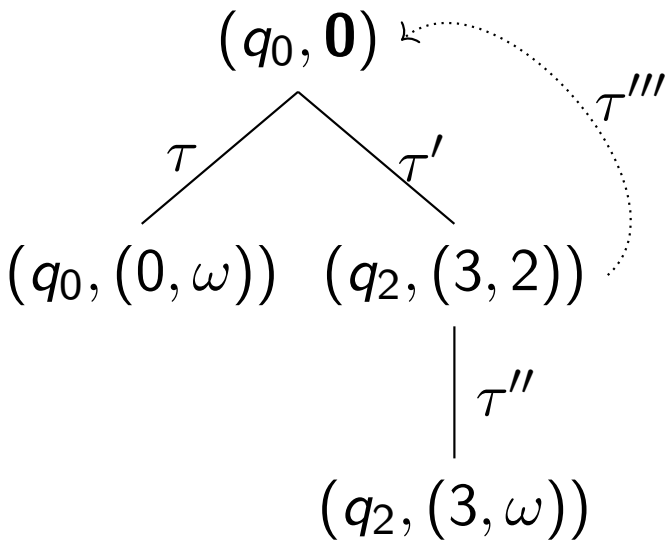
Karp-Miller tree: data structure

We will see how to construct the Karp-Miller tree of a VASS. This will require a data structure. Let (Q, T, q_0) be a d-VASS.

Data structure

Tree with:

- node labels in $Q \times \mathbb{N}_\omega^d$
- edge labels in T
- additional *upward edges* pointing from nodes to ancestors



- Let the label of node v be γ_v. We call v *complete* if v has an outgoing τ-edge for each $\tau \in T$ activated in γ_v.
- Suppose $\gamma = (q, \mathbf{u}), \gamma' = (q, \mathbf{u}') \in Q \times \mathbb{N}_\omega^d$ with $\mathbf{u} \leq \mathbf{u}'$. Then $\Omega(\gamma, \gamma') := (q, \mathbf{v})$ with:

$$\mathbf{v}(i) = \begin{cases} \mathbf{u}(i) & \text{if } \mathbf{u}(i) = \mathbf{u}'(i) \\ \omega & \text{if } \mathbf{u}(i) < \mathbf{u}'(i) \end{cases} \qquad \text{for } i \in \{1, \ldots, d\}$$

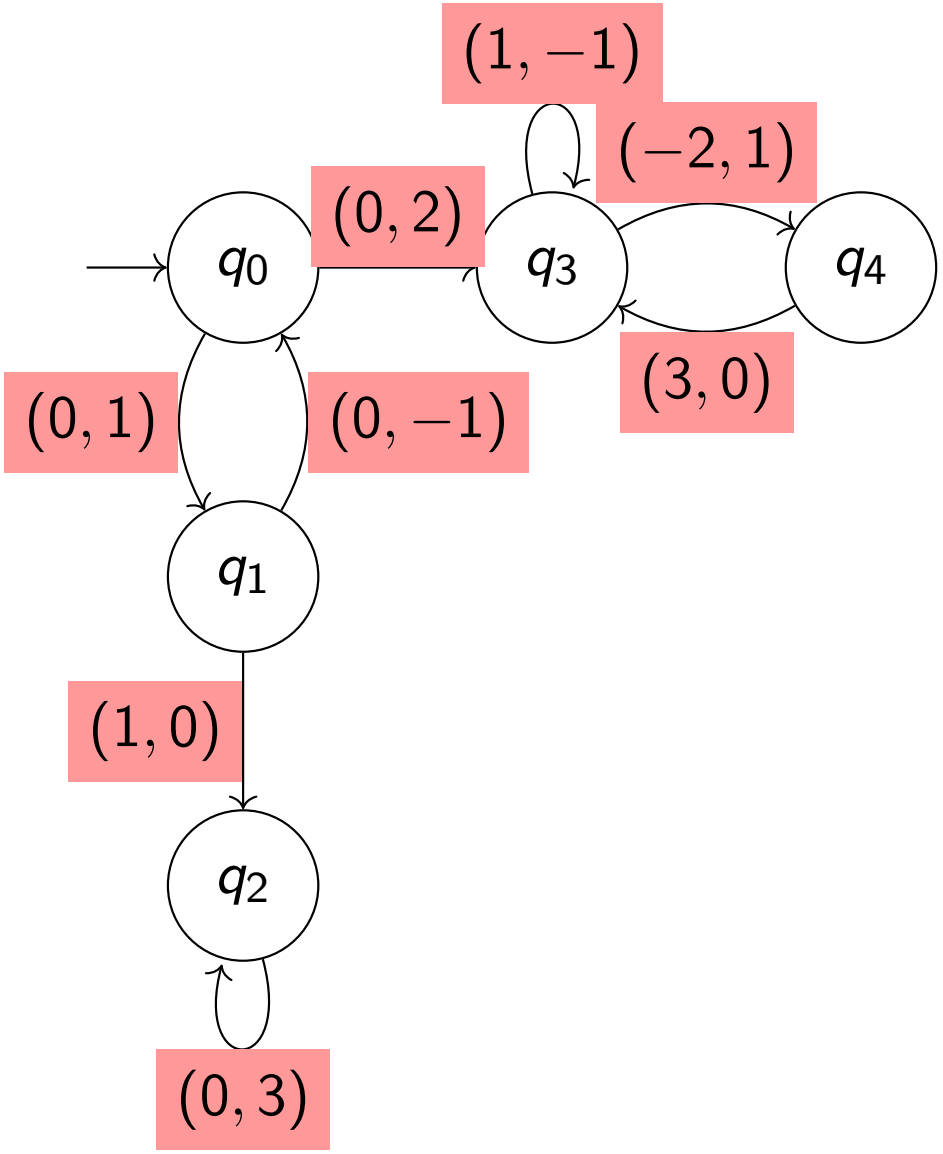
Karp-Miller trees

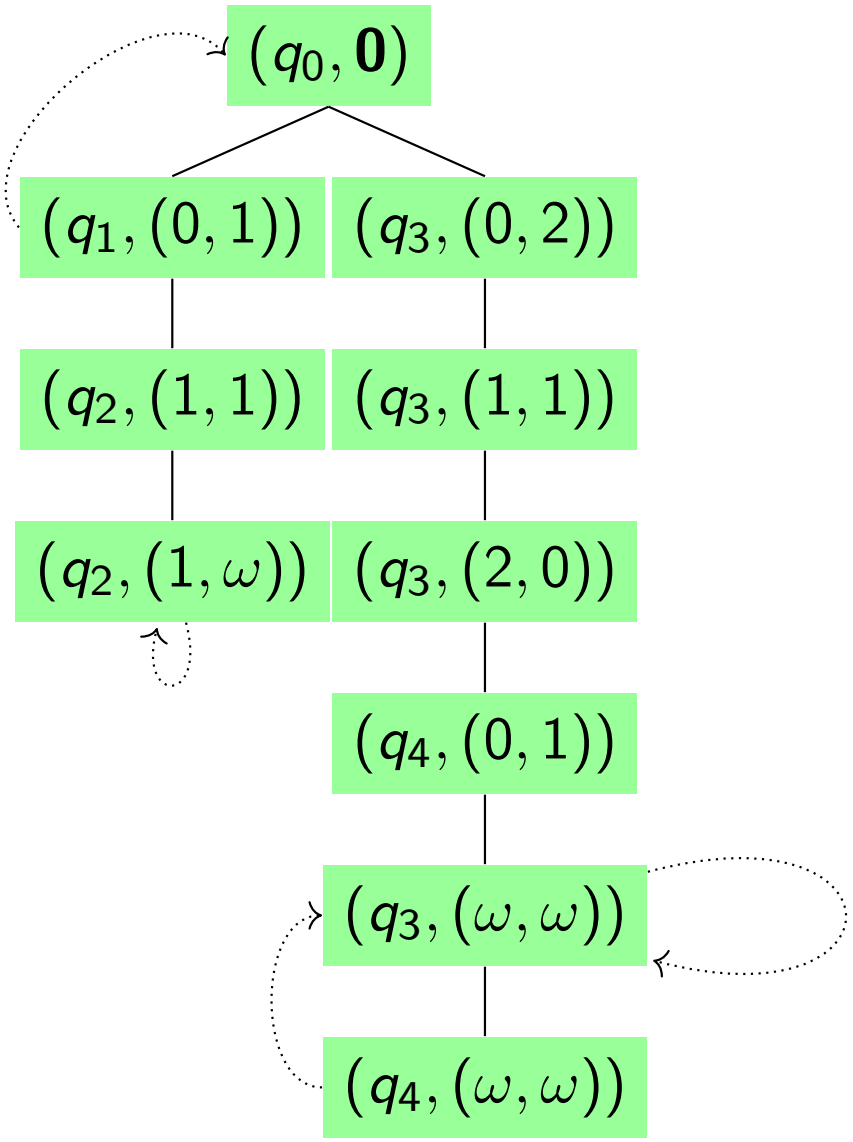
Let $\mathcal{V} = (Q, T, q_0)$ be a d-VASS.
t: tree with node labels in $Q \times \mathbb{N}_\omega^d$ and edge labels in T

Constructing the Karp-Miller tree

input: VASS $\mathcal{V} = (Q, T, q_0)$.
Initialize t as tree with one node, labeled by $(q_0, \mathbf{0})$.
while t has an incomplete leaf v **do**
  for $\tau \in T$ activated in γ_v **do**
    Compute $\gamma \in Q \times \mathbb{N}_\omega^d$ with $\gamma_v \xrightarrow{\tau} \gamma$
    if there is an ancestor $\bar{v}$ of v with $\gamma_{\bar{v}} \leq \gamma$ **then**
      Pick such $\bar{v}$ with maximal distance from the root
      if $\gamma_{\bar{v}} = \gamma$ **then**
        Add an upward edge from v to $\bar{v}$ labeled τ
      else if $\gamma_{\bar{v}} < \gamma$ **then**
        Add child v' labeled $\Omega(\gamma_{\bar{v}}, \gamma)$ under v, edge label: τ
    else
      Add child v' labeled γ under v, edge label: τ

An example


$(1,-1)$
$(-2,1)$
$(0,2)$
q_0
q_3
q_4
$(3,0)$
$(0,1)$
$(0,-1)$
q_1
$(1,0)$
q_2
$(0,3)$


$(q_0, \mathbf{0})$
$(q_1,(0,1))$
$(q_3,(0,2))$
$(q_2,(1,1))$
$(q_3,(1,1))$
$(q_2,(1,\omega))$
$(q_3,(2,0))$
$(q_4,(0,1))$
$(q_3,(\omega,\omega))$
$(q_4,(\omega,\omega))$

Karp-Miller trees are finite

Proposition

The algorithm constructing the Karp-Miller tree terminates.

Proof.

Suppose the algorithm does not terminate.
Then the constructed tree is infinite. It is finitely branching, so by Kőnig's Lemma, there is an infinite branch.
Let $(q_1, \mathbf{u}_1), (q_2, \mathbf{u}_2), \ldots \in Q \times \mathbb{N}_\omega^d$ be the labels on the branch.
Since $\leq$ is a WQO on $Q \times \mathbb{N}_\omega^d$, there is an ascending subsequence $(q_1', \mathbf{u}_1') \leq (q_2', \mathbf{u}_2') \leq \cdots$.
The branch cannot contain a label twice: Otherwise, we would have introduced an upward edge. Hence $(q_1', \mathbf{u}_1') < (q_2', \mathbf{u}_2') < \cdots$.
This means, the algorithm introduces an ω when it adds $(q_i', \mathbf{u}_i')$, for every $i \in \mathbb{N}$, $i \geq 2$.
Along a branch, the set of ω-coordinates can never get smaller.
Thus, we can introduce an ω at most d times in a branch.
Contradiction! $\square$

The Karp-Miller graph

Definition

The *Karp-Miller graph* is obtained from the Karp-Miller tree by turning upward edges into normal edges.

(In the literature, one sometimes also merges nodes with the same label. But the difference is not important for our purposes.)
For $\mathbf{u} \in \mathbb{N}_\omega^d$ and $k \in \mathbb{N}$, let $\mathbf{u}^{(k)} \in \mathbb{N}^d$ denote the vector obtained by replacing each ω by k.

Proposition

For every node labeled $(q, \mathbf{u}) \in Q \times \mathbb{N}_\omega^d$ in the Karp-Miller graph and every $k \in \mathbb{N}$, there is a reachable configuration $(q, \mathbf{v}) \in Q \times \mathbb{N}^d$ in the VASS with $(q, \mathbf{u}^{(k)}) \leq (q, \mathbf{v})$.

Proof.

It suffices to show the proposition for the Karp-Miller tree.
Proceed by induction on the distance from the root.
Let v be a node in the Karp-Miller tree.

1. If v was added without introducing ω:
   Apply induction to its parent node.
2. If v was added with introduction of ω:
   Let $\bar{v}$ be the ancestor used to construct v; let $(q, \bar{\mathbf{u}})$ and $(q, \mathbf{u})$ be the labels of $\bar{v}$ and v. Consider the transition sequence $\sigma = \tau_1 \cdots \tau_m$ on the branch from $\bar{v}$ to v. In those components where σ has negative effect, $\bar{\mathbf{u}}$ must have ω. In every new ω-component, σ must have a positive effect. By induction hypothesis, for each number $\ell \in \mathbb{N}$ there is a $\mathbf{y} \in \mathbb{N}^d$ with $(q_0, \mathbf{0}) \to^* (q, \mathbf{y}) \geq (q, \bar{\mathbf{u}}^{(\ell)})$. Then there is a (large) $\ell \in \mathbb{N}$ and $\mathbf{y}, \mathbf{u}' \in \mathbb{N}^d$ such that $(q_0, \mathbf{0}) \to^* (q, \mathbf{y}) \geq (q, \bar{\mathbf{u}}^{(\ell)})$ and $(q, \bar{\mathbf{u}}^{(\ell)}) \xrightarrow{\sigma^k} (q, \mathbf{u}') \geq (q, \mathbf{u}^{(k)})$. Since VASS are WSTS there is also a $\mathbf{u}'' \in \mathbb{N}^d$ with $(q_0, \mathbf{0}) \to^* (q, \mathbf{y}) \xrightarrow{\sigma^k} (q, \mathbf{u}'')$ and $\mathbf{u}'' \geq \mathbf{u}' \geq \mathbf{u}^{(k)}$. $\square$

Proposition

For every run

$$(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \cdots \to (q_m, \mathbf{u}_m)$$

in $\mathcal{V}$*, there is a path*

$$(q_0, \mathbf{u}'_0) \to (q_1, \mathbf{u}'_1) \to \cdots \to (q_m, \mathbf{u}'_m)$$

in the Karp-Miller graph such that $(q_i, \mathbf{u}_i) \leq (q_i, \mathbf{u}'_i)$ *for every* $i \in \{1, \ldots, m\}$*.*

Proof.

Straightforward induction on m. □

The two previous propositions imply:

Corollary

Let $(q_1, \mathbf{v}_1), \ldots, (q_n, \mathbf{v}_n) \in Q \times \mathbb{N}_\omega^d$ *be the labels of the nodes in the Karp-Miller graph. Then*

$$\mathrm{Cover}(\mathcal{V}) = \{q_1\} \times \mathrm{Idl}(\mathbf{v}_1) \ \cup \ \cdots \ \cup \ \{q_n\} \times \mathrm{Idl}(\mathbf{v}_n).$$

In particular: Given a VASS, one can effectively compute the ideal decomposition of its coverability set.

Part IV

Vector addition systems

Section 12

The complexity of the coverability problem

Coverability in VASS

- We have seen:
  Coverability is decidable for "almost all" WSTS.
- The general algorithm has extremely high running time (Ackermann function)
- For VASS, we can do better:

Theorem (Rackoff 1978)

The coverability problem for VASS can be solved in EXPSPACE.

Encoding VASS

- When we analyze the complexity of an algorithm, we study the time or space it uses depending on the *size of the input*.
- Strictly speaking, this should be the number of bits used to write down the input instance.

Binary vs. unary encodings

There is, however, one important aspect of encoding VASS:

- Are transition (numbers) encoded in unary or binary?
- This sometimes leads to very different complexities:
  - For unary encodings, all numbers are linear in the size of the input.
  - For binary encodings, all numbers are exponential in the size of the input.

In this lecture, we will assume that transition updates are encoded in unary. For the problems that we consider, there is no difference in complexity compared to binary encoding.

The size of a VASS

To simplify analysis, one usually defines the size slightly differently; but it will be clear that this will lead to the same complexity classes.

Definition

Let $\mathcal{V} = (Q, T, q_0)$ be a d-VASS. Recall that for $\mathbf{u} \in \mathbb{Z}^d$, we have $\|\mathbf{u}\| := |\mathbf{u}(1)| + \cdots + |\mathbf{u}(d)|$. We define

$$\|T\| = \max\{\|\mathbf{v}\| \mid (q, \mathbf{v}, q') \in T\}.$$

The *size of* $\mathcal{V}$ is defined as $d \cdot \|T\| \cdot |T| \cdot |Q|$.

Normalized VASS

We work with vectors in $\mathbb{Z}^d$. Recall that $\mathbf{e}_i$ denotes the vector $(0, \ldots, 0, 1, 0, \ldots, 0)$, where the 1 is in coordinate i.

Definition

Let $\mathcal{V} = (Q, T, q_0)$ be a d-VASS. We say that $\mathcal{V}$ is *normalized* if every $(q, \mathbf{v}, q') \in T$ satisfies $\mathbf{v} = \mathbf{e}_i$ or $\mathbf{v} = -\mathbf{e}_i$ with $i \in \{1, \ldots, d\}$.

The size of normalized VASS

If $\mathcal{V} = (Q, T, q_0)$ is a normalized VASS, then its size is $d \cdot |T| \cdot |Q|$, because $\|T\| = 1$.

Observation

Given a VASS $\mathcal{V}$ and a config. $(q, \mathbf{u})$, we can construct in polynomial time a normalized VASS $\mathcal{V}'$ with a config. $(q', \mathbf{u}')$ s.t.:

$$(q, \mathbf{u}) \text{ coverable in } \mathcal{V} \iff (q', \mathbf{u}') \text{ coverable in } \mathcal{V}'.$$

Idea: turn each transition of $\mathcal{V}$ into a sequence of transitions in $\mathcal{V}'$.
Hence: Show that coverability in EXPSPACE for normalized VASS.

Coverability in VASS

Let $\mathcal{V} = (Q, T, q_0)$. Recall that a state $q \in Q$ is called *reachable* if there is a run

$$(q_0, \mathbf{0}) \to (q_1, \mathbf{u}_1) \to \cdots \to (q_m, \mathbf{u}_m)$$

in $\mathcal{V}$ with $q_m = q$.
The coverability problem in VASS can be reduced to the following: How?

State reachability problem in VASS

Given A VASS $\mathcal{V} = (Q, T, q_0)$ and a state $q_f \in Q$.

Question Is q_f reachable?

The following is the key fact for the exponential space algorithm:

Theorem (Rackoff 1978)

Let $\mathcal{V} = (Q, T, q_0)$ be a normalized d-VASS and $q \in Q$. If q is reachable, then q is reachable with a run of length $\leq (2|Q|)^{(d+1)!}$.

Let $\mathcal{V} = (Q, T, q_0)$ be a VASS with a given target state $q_f \in Q$.

Key concept: i-runs

We define a new step relation $\to^i$ on $Q \times \mathbb{Z}^d$ for $i \in [0, d]$:
For $(q, \mathbf{u}), (q', \mathbf{u}') \in Q \times \mathbb{Z}^d$, let $(q, \mathbf{u}) \to^i (q', \mathbf{u}')$ if

1. there is a transition $(q, \mathbf{v}, q') \in T$ with $\mathbf{u}' = \mathbf{u} + \mathbf{v}$ and
2. $\mathbf{u}(j) \geq 0$ and $\mathbf{u}'(j) \geq 0$ for $j \in [1, i]$.

For $i \in [0, d]$, an *i-run from* $(q, \mathbf{u})$ is a run

$$(q_1, \mathbf{u}_1) \to^i (q_2, \mathbf{u}_2) \to^i \cdots \to^i (q_m, \mathbf{u}_m)$$

with $(q_1, \mathbf{u}_1) = (q, \mathbf{u})$ and $q_m = q_f$. Its *length* is $m - 1$.

Clearly, q_f is reachable if and only if there is a d-run from $(q_0, \mathbf{0})$.
For $i \in [0, d]$ and $(q, \mathbf{u}) \in Q \times \mathbb{Z}^d$, let

$$m(i, (q, \mathbf{u})) := \min\{\ell{+}1 \mid \text{there is an } i\text{-run from } (q, \mathbf{u}) \text{ of length } \ell\}.$$

if there is an i-run from $(q, \mathbf{u})$. Otherwise, set $m(i, (q, \mathbf{u})) = 0$.
For $i \in [0, d]$, let

$$f(i) := \max\{m(i, (q, \mathbf{u})) \mid (q, \mathbf{u}) \in Q \times \mathbb{Z}^d\}.$$

Observe: $f(0) \leq |Q|$.

Lemma

Let $n = |Q|$. For every $i \in [0, d-1]$:
$f(i+1) \leq n \cdot f(i)^{i+1} + f(i) - 1$.

Let $(q, \mathbf{u}) \in Q \times \mathbb{Z}^d$ and suppose there is an $(i+1)$-run ρ from $(q, \mathbf{u})$.
We have to show that there is one of length $\leq n \cdot f(i)^{i+1} + f(i) - 1$.

1. Case 1: For every configuration $(p, \mathbf{v}) \in Q \times \mathbb{Z}^d$ in ρ, we have $\mathbf{v}(j) < f(i)$ for $j \in [1, i+1]$.
   If two configurations $(p, \mathbf{v})$ and $(p', \mathbf{v}')$ satisfy $p = p'$ and $\mathbf{v}(j) = \mathbf{v}'(j)$ for all $j \in [1, i+1]$, then we can cut out all steps between $(p, \mathbf{v})$ and $(p', \mathbf{v}')$ and obtain a shorter $(i+1)$-run.
   There are at most $n \cdot f(i)^{i+1}$ possible choices for $p \in Q$ and the values $\mathbf{v}(j)$ for $j \in [1, i+1]$. Therefore, after the cutting, we have a run of length $\leq n \cdot f(i)^{i+1} - 1$.
2. Case 2: There is a configuration $(p, \mathbf{v}) \in Q \times \mathbb{Z}^d$ in ρ with $\mathbf{v}(j) \geq f(i)$ for some $j \in [1, i+1]$. Suppose $(p, \mathbf{v})$ is the earliest such configuration. Without loss of generality, suppose $\mathbf{v}(i+1) \geq f(i)$.

Let $(p', \mathbf{v}')$ be the configuration right before $(p, \mathbf{v})$. We can decompose ρ into three parts:

$$\rho = \underbrace{\rho_1}_{(i+1)\text{-run from } (q,\mathbf{u}) \text{ to } (p',\mathbf{v}')} \underbrace{\rho_2}_{\text{transition } (p',\mathbf{v}') \to^i (p,\mathbf{v})} \underbrace{\rho_3}_{\text{rest}}$$

- ρ_1 has the property of Case 1. Hence, by the same argument, it can be replaced by an $(i+1)$-run ρ_1' of length $\leq n \cdot f(i)^{i+1} - 1$.
- ρ_2 has length 1.
- We now replace ρ_3:
  Let ρ_3' be a minimal-length i-run from $(p, \mathbf{v})$.
  Since $\mathbf{v}(i+1) \geq f(i)$ and ρ_3' is of length $\leq f(i) - 1$, the i-run ρ_3' must even be an $(i+1)$-run:
  A run of length $f(i) - 1$ can decrease the coordinate $i+1$ by at most $f(i) - 1$, thus ρ_3' cannot reach a negative value there.

This implies that $\rho' = \rho_1' \rho_2 \rho_3'$ is an $(i+1)$-run from $(q, \mathbf{u})$. It has length:

$$\leq (n \cdot f(i)^{i+1} - 1) + 1 + (f(i) - 1) = n \cdot f(i)^{i+1} + f(i) - 1. \quad \square$$

Lemma

$f(i) \leq (2n)^{(i+1)!}$ for every $i \in [0, d]$.

Induction on i: $f(0) \leq n \leq 2n = (2n)^{1!}$. Now suppose $i \geq 0$ and $f(i) \leq (2n)^{(i+1)!}$.
Then we have:

$$\begin{aligned} f(i+1) &\leq n \cdot f(i)^{i+1} + f(i) \leq n \cdot \left((2n)^{(i+1)!}\right)^{i+1} + (2n)^{(i+1)!} \\ &\leq 2n \cdot \left((2n)^{(i+1)!}\right)^{i+1} = (2n)^{1+(i+1)!\cdot(i+1)} \\ &\leq (2n)^{(i+1)!\cdot(i+2)} = (2n)^{(i+2)!}. \quad \square \end{aligned}$$

We are now ready to prove:

Theorem (Rackoff 1978)

Let $\mathcal{V} = (Q, T, q_0)$ be a normalized d-VASS and $q \in Q$. If q is reachable, then q is reachable with a run of length $\leq (2|Q|)^{(d+1)!}$.

Suppose q is reachable. Then there is a d-run from $(q_0, \mathbf{0})$.
It has length at most $f(d) \leq (2n)^{(d+1)!} = (2|Q|)^{(d+1)!}$. $\square$

Theorem (Rackoff 1978)

The coverability problem for VASS can be solved in EXPSPACE.

We first reduce coverability to state reachability for normalized VASS. Hence, we are given a normalized VASS $\mathcal{V} = (Q, T, q_0)$ and a state $q_f \in Q$. We know that if q_f is reachable, then with a run of length $(2|Q|)^{(d+1)!}$. In such a run, every configuration $(q, \mathbf{u})$ must satisfy $\|\mathbf{u}\| \leq (2|Q|)^{(d+1)!}$. A non-deterministic algorithm:

input: VASS $\mathcal{V} = (Q, T, q_0)$ and state $q_f \in Q$.
Initialize $(q, \mathbf{u}) = (q_0, \mathbf{0}), c = 0$.
while $c \leq (2|Q|)^{(d+1)!}$ **do**
  Guess $\tau = (q, \mathbf{v}, q')$ with $\mathbf{u} + \mathbf{v} \in \mathbb{N}^d$, $\|\mathbf{u} + \mathbf{v}\| \leq (2|Q|)^{(d+1)!}$.
  Set $(q, \mathbf{u}) := (q', \mathbf{u} + \mathbf{v}), c := c + 1$.
  if $q = q_f$ **then**
    Report: "State q_f is reachable".
Report: "State q_f is not reachable".

We encode all numbers in binary. Then the algorithm needs space

$$|Q| \cdot d \cdot \log\left((2|Q|)^{(d+1)!}\right).$$

Let $s = |Q| \cdot |T| \cdot d$ denote the size of the VASS. Then the used space is bounded by:

$$\begin{aligned} |Q| \cdot d \cdot \log\left((2|Q|)^{(d+1)!}\right) &\leq s \cdot \log\left((2s)^{(s+1)!}\right) \\ &= s \cdot \log\left(2^{\log(2s)\cdot(s+1)!}\right) \\ &= s \cdot \log(2s) \cdot (s+1)! \\ &\leq (s+3)! \end{aligned}$$

Note that for every number $m \in \mathbb{N}$, we have

$$m! \leq m^m = 2^{m \cdot \log(m)} \leq 2^{m^2}.$$

Therefore, the algorithm uses space at most

$$2^{(s+3)^2}$$

which is exponential in the size of the VASS. $\square$

Part IV

Vector addition systems

Section 13

EXPSPACE-hardness of the coverability problem

Lipton's lower bound

We have seen that, if a state in VASS can be reached, then with a run with doubly exponentially bounded counters. In particular, VASS coverability can be solved using exponential space.

In this section we prove a matching lower bound.

Theorem (Lipton 1976)

The coverability problem for VASS is EXPSPACE-*hard.*

In particular, also the reachability problem is EXPSPACE-hard. (Decidability of reachability was only proved in 1981!)

We follow a presentation of Lipton's proof by Esparza (1996). In the proof we will construct a VASS that can count up *precisely* to 2^{2^n}.

We will reduce from Turing machines, via Minsky machines, to VASS.

To simplify notation we replace Minsky machines and VASS by simple programming languages:

- Minsky machines $\leftrightarrow$ *counter programs*.
- VASS $\leftrightarrow$ *net programs*.

Definition

A *counter program* (or GOTO program) C is a sequence of *labelled instructions*:

ℓ : x++

ℓ : x−−

ℓ : **goto** ℓ ▷ unconditional jump

ℓ : **if** $x = 0$ **goto** ℓ_1 **else goto** ℓ_2 ▷ conditional jump

ℓ : **halt**

All variables initialized to 0. Counter programs are *deterministic*. The *size* of a counter program is the number of instructions.

The unique run of a counter program is *k-bounded* if all variables are bounded by k throughout the run.

Lemma

The following problem is EXPSPACE*-hard:*

Given *A counter program C and a unary encoded number n.*

Question *Does C halt via a 2^{2^n}-bounded run?*

Proof.

Let $L \in$ EXPSPACE and M be a Turing machine for L using space $2^{p(n)}$ for some polynomial p. Given an input word $w = a_1 \dots a_n$ construct a Turing machine M_w which writes w on its (empty) tape and simulates M. Then $w \in L$ iff M_w halts (on ε) via a run that uses at most $2^{p(n)}$ tape cells.

It remains to simulate an s-space bounded Turing machine by a $2^{O(s)}$-bounded counter program, see next lemma.

From TMs to Counter Programs

Lemma

An s-space bounded Turing machine M can be converted in polynomial-time into a $2^{O(s)}$-bounded counter program C.

Proof.

Step 1. Convert M into a machine P with two pushdowns (stacks). A tape with content $a_1 a_2 \cdots a_n$ is split into two pushdowns with contents $a_k a_{k-1} \cdots a_1$ and $a_{k+1} \cdots a_n$ if the head is at position k. The pushdown heights are bounded by s.

Step 2. Convert P into a counter program C with three counters. Assume that the stack alphabet is $\{1, \ldots, b-1\}$, read each pushdown as the b-ary representation of a number bounded by $b^s \leq 2^{O(s)}$. Pushdown operations can be simulated:

- **push**(a): multiply by b, add a.
- **pop**(a): subtract a, divide by b.

Implement multiplication/division by b using a third counter. □

Net programs

Definition

A *net program* N is a sequence of *labelled instructions*:

ℓ : x++

ℓ : x−−

ℓ : **goto** ℓ ▷ unconditional jump

ℓ : **goto** ℓ_1 **or goto** ℓ_2 ▷ nondeterministic jump

ℓ : **call** ℓ ▷ subroutine call

ℓ : **return** ▷ end of subroutine

ℓ : **halt**

Again, all variables initialized to 0. Net programs are *nondeterministic*. We ask the question whether there *exists* a run which reaches **halt**.

The *size* of a net program is the number of instructions.

Net programs

We only work with *well-structured* net programs:

- Net program is decomposed into k subprograms.
- Jumps only inside a subprogram.
- Subroutine calls only from subprogram $i+1$ to subprogram i.

We transform a well-structured net program N into a VASS V with a state f such that:

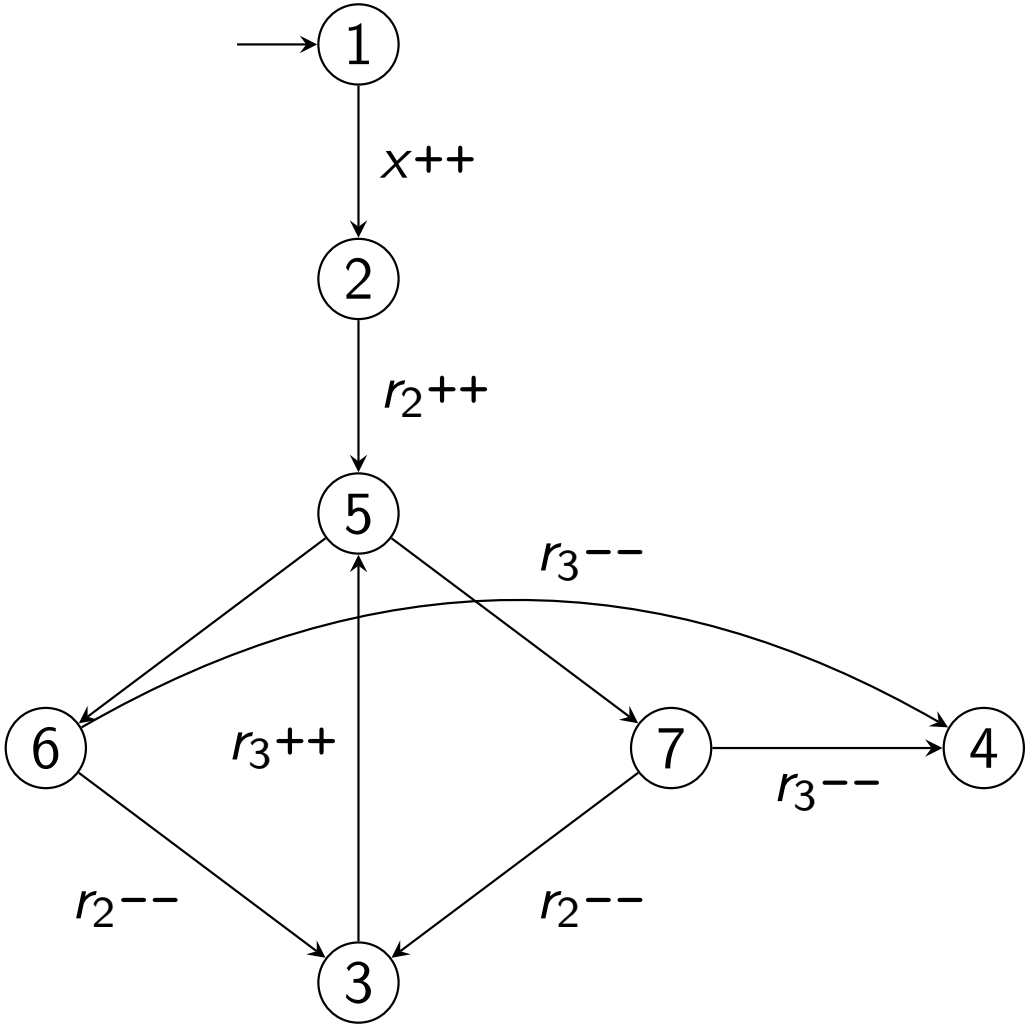
$$N \text{ halts} \iff \text{state } f \text{ is reachable in } V.$$

V contains

- a state for each instruction label,
- a counter for each variable,
- a counter for each subroutine call.

Example: From net programs to VASS

1: x++
2: **call** 5
3: **call** 5
4: **halt**
5: **goto** 6 **or goto** 7
6: **return**
7: **return**



Check whether state 4 can be reached.

Question: Where do we use well-structuredness?

Proof idea

It remains to reduce the 2^{2^n}-bounded halting problem for counter programs to the halting problem for net programs.

Clearly we can simulate counter updates in a VASS.
How to simulate zero tests on 2^{2^n}-bounded counters?

Idea 1

Every variable s has a copy $\bar{s}$ such that $s + \bar{s} = 2^{2^n}$.

- The net program guesses whether $s > 0$ or $s = 0$.
- To test $s > 0$ we first decrement and then increment s.
- To test $s = 0$ we add 2^{2^n} to s and subtract 2^{2^n} from $\bar{s}$.
  $\rightarrow$ Side effect: s and $\bar{s}$ are swapped.

Proof idea

How to increment/decrement by 2^{2^n}?
Induction by n: Assume that

- we can increment/decrement by $2^{2^{n-1}}$
- we have a zero-test on $2^{2^{n-1}}$-bounded counters.

Then the following procedure computes s -= 2^{2^n} and $\bar{s}$ += 2^{2^n}:

Idea 2

assume $y = z = 2^{2^{n-1}}$, $\bar{y} = \bar{z} = 0$
repeat
  y−−; $\bar{y}$++;
  repeat
    z−−; $\bar{z}$++;
    s−−; $\bar{s}$++;
  until $z = 0$ ▷ side effect: swaps z and $\bar{z}$ if $z = 0$
until $y = 0$ ▷ side effect: swaps y and $\bar{y}$ if $y = 0$

The simulation

Given a counter program C and a number n.
We transform C into a net program N, which contains all variables from C and further auxiliary variables. Each variable x in N has a complementary variable $\bar{x}$.
N consists of two phases:

- $N_{\text{init}}(C)$ initializes $\bar{x} = 2^{2^n}$ for all variables x.
- $N_{\text{sim}}(C)$ simulates C while maintaining $x + \bar{x} = 2^{2^n}$.

We start with the simulating phase $N_{\text{sim}}(C)$.

Easy: Increments, decrements, unconditional jumps are simulated directly.

We replace a conditional jump

if $x = 0$ **goto** $\ell_{=0}$ **else goto** $\ell_{\neq 0}$

by a program $\text{Test}_n(x, \ell_{=0}, \ell_{\neq 0})$.

Since net programs are nondeterministic and can abort at any point, we need to carefully specify how a net program *simulates* a conditional jump.

Specification of $\mathsf{Test}_n(x, \ell_{=0}, \ell_{\neq 0})$

If $x = 0$ ($x \neq 0$) then some run of the program leads to $\ell_{=0}$ ($\ell_{\neq 0}$), and no run leads to $\ell_{\neq 0}$ ($\ell_{=0}$).
Moreover, the program has no *side-effects*: After any run leading to $\ell_{=0}$ or $\ell_{\neq 0}$, no variable has changed its value.

We will design a program $\mathsf{Test}'_n(x, \ell_{=0}, \ell_{\neq 0})$ with the same specification but a side-effect: Any run leading to $\ell_{=0}$ swaps the values of x and $\bar{x}$.

Program $\mathsf{Test}_n(x, \ell_{=0}, \ell_{\neq 0})$

$$\begin{array}{rl} & \textbf{call } \mathsf{Test}'_n(x, \ell_{\mathsf{cont}}, \ell_{\neq 0}) \\ \ell_{\mathsf{cont}} : & \textbf{call } \mathsf{Test}'_n(\bar{x}, \ell_{=0}, \ell_{\neq 0}) \end{array}$$

$\mathrm{Test}'_i(x, \ell_{=0}, \ell_{\neq 0})$ (zero test with side-effect)

$$
\begin{aligned}
& \text{assume } s_i = 0 \text{ and } \bar{s}_i = 2^{2^i} \\
& \textbf{goto } \ell_{\mathrm{nz}} \textbf{ or } \ell_{\mathrm{z}} \\
\ell_{\mathrm{nz}} : \; & x{-}{-}, x{+}{+} \\
& \textbf{goto } \ell_{\neq 0} \\
\ell_{\mathrm{z}} : \; & x{+}{+}, \bar{x}{-}{-}, s_i{+}{+}, \bar{s}_i{-}{-} \\
& \textbf{goto } \ell_{\mathrm{z}} \textbf{ or } \ell_{\mathrm{brk}} \\
\ell_{\mathrm{brk}} : \; & \textbf{call } \mathrm{Dec}_i \\
& \textbf{goto } \ell_{=0}
\end{aligned}
$$

Specification of Dec_i

- If $s_i \geq 2^{2^i}$ then all runs to **return** have the effect $s_i := s_i - 2^{2^i}$ and $\bar{s}_i := \bar{s}_i + 2^{2^i}$, and no other effects. Furthermore, there exists such a run.
- If $s_i < 2^{2^i}$ then every run fails.

Decrement subroutines

subroutine Dec_0 $\quad\triangleright$ decrements by $2^{2^0} = 2$

$s_0{--}, \bar{s}_0{++}$

$s_0{--}, \bar{s}_0{++}$

return

subroutine Dec_{i+1} $\quad\triangleright$ assume $y_i = z_i = 2^{2^i}$, $\bar{y}_i = \bar{z}_i = 0$

ℓ_{out} : $y_i{--}, \bar{y}_i{++}$

ℓ_{in} : $z_i{--}, \bar{z}_i{++}$

$s_i{--}, \bar{s}_i{++}$

call $\mathrm{Test}'_i(z_i, \ell_{\mathrm{in}'}, \ell_{\mathrm{in}})$

$\ell_{\mathrm{in}'}$: **call** $\mathrm{Test}'_i(y_i, \ell_{\mathrm{out}'}, \ell_{\mathrm{out}})$

$\ell_{\mathrm{out}'}$: **return**

Initialization

The program $N_{\text{init}}(C)$ initializes the variables of N as follows:

- All variables x from C are set to 0 and their complementary variables $\bar{x}$ to 2^{2^n}.
- All variables s_i are set to 0 and $\bar{s}_i$ to 2^{2^i}.
- For all $0 \leq i \leq n-1$ initialize y_i, z_i to 2^{2^i} and $\bar{y}_i, \bar{z}_i$ to 0.

Initialization $N_{\text{init}}(C)$

call $\text{Inc}_0(\bar{s}_0, y_0, z_0)$
call $\text{Inc}_1(\bar{s}_1, y_1, z_1)$
...
call $\text{Inc}_{n-1}(\bar{s}_{n-1}, y_{n-1}, z_{n-1})$
call $\text{Inc}_n(\bar{s}_n, \bar{x}_1, \ldots, \bar{x}_l)$ $\triangleright$ x_i are the variables from C

Here, Inc_i increases the given variables by 2^{2^i} without any side-effects.

Initialization

$\mathrm{Inc}_i(v_1, \ldots, v_m)$ increases all variables $v_1, \ldots, v_m$ by 2^{2^i} without any side-effects.

Programs Inc_i

subroutine $\mathrm{Inc}_0(v_1, \ldots, v_m)$ ▷ increments by $2^{2^0} = 2$

$v_1{+}{+}, v_1{+}{+}, \ldots, v_m{+}{+}, v_m{+}{+}$

return

subroutine $\mathrm{Inc}_{i+1}(v_1, \ldots, v_m)$ ▷ assume $y_i = z_i = 2^{2^i}$ and $\bar{y}_i = \bar{z}_i = 0$

ℓ_{out} : $y_i{-}{-}, \bar{y}_i{+}{+}$

ℓ_{in} : $z_i{-}{-}, \bar{z}_i{+}{+}$

$v_1{+}{+}, \ldots, v_m{+}{+}$

call $\mathrm{Test}'_i(z_i, \ell_{\mathrm{in}'}, \ell_{\mathrm{in}})$

$\ell_{\mathrm{in}'}$: **call** $\mathrm{Test}'_i(y_i, \ell_{\mathrm{out}'}, \ell_{\mathrm{out}})$

$\ell_{\mathrm{out}'}$: **return**

Summary

We claim that the size of N is linear in the size of C and n.

- Each counter update (increment/decrement) on x is simulated by two updates on x and $\bar{x}$.
- Each unconditional jump is simulated by an unconditional jump.
- Each conditional jump is simulated by a different instance of Test_n.
- The subroutines $\mathsf{Dec}_n, \ldots, \mathsf{Dec}_0$ and programs $\mathsf{Inc}_n, \ldots, \mathsf{Inc}_0$ have total size $O(n)$.

It is easy to see that N can be constructed in polynomial-time. This completes the reduction.

Part IV

Vector addition systems

Section 14

Non-primitive-recursive behavior of VASS

- We have seen that covering runs are "short" (in the size of the VASS).
- Now we want to know:
  Suppose the reachability set of a VASS is finite.
  How large can it be?
- Given Rackoff's result, one might be tempted to think: at most doubly exponential.
- However, finite reachability sets can be much, much larger.
- To explain how much, we have to make an excursion to the theory of computable functions.

Primitive recursive functions

The primitive recursive functions were an early attempt to describe all intuitively computable functions.

The primitive recursive functions

The primitive recursive functions are the smallest class PR of functions $f\colon \mathbb{N}^k \to \mathbb{N}$ for $k \in \mathbb{N}$ that contains:

1. the zero function $\mathsf{z}\colon \mathbb{N} \to \mathbb{N}$, $n \mapsto 0$ for $n \in \mathbb{N}$
2. the successor function $\mathsf{s}\colon \mathbb{N} \to \mathbb{N}$, $n \mapsto n+1$ for $n \in \mathbb{N}$
3. the projections $\pi_\ell^k\colon \mathbb{N}^k \to \mathbb{N}$, $(x_1, \ldots, x_k) \mapsto x_\ell$

and is closed under:

1. Composition: Given $g_1, \ldots, g_\ell\colon \mathbb{N}^m \to \mathbb{N}$ and $h\colon \mathbb{N}^\ell \to \mathbb{N}$, we have the function $f\colon \mathbb{N}^m \to \mathbb{N}$ with $f(\mathbf{u}) = h(g_1(\mathbf{u}), \ldots, g_\ell(\mathbf{u}))$.
2. Primitive recursion: Given $g\colon \mathbb{N}^k \to \mathbb{N}$ and $h\colon \mathbb{N}^{k+2} \to \mathbb{N}$, we have $f\colon \mathbb{N}^{k+1} \to \mathbb{N}$ with
$$f(\mathbf{u}, 0) = g(\mathbf{u}), \quad f(\mathbf{u}, n+1) = h(\mathbf{u}, n, f(\mathbf{u}, n)).$$

Primitive recursive functions: Loop programs

Loop programs

1. Simple assignments: $x := 0$, $x := y + 1$, $x := y$
2. Sequential composition: $p; q$
3. Conditional test: `if` $x < y$ `then` p `else` q
   Instead of $<$, one can also use $>$, $\geq$, $\leq$, $=$, $\neq$.
4. Loop: `loop` y do p

Note: When executing a `loop`, first the value of y is evaluated, then the loop is executed y times. Thus, the body of the loop cannot influence the number of times it is executed.

Intuition: Programs with loops where number of executions is known beforehand.

Theorem

A function is primitive recursive if and only if it is computed by some loop program.

Examples

Here are some examples of primitive recursive functions:

$n \mapsto n+1$	
$n \mapsto 2n$	1 recursion/loop
$n \mapsto 2^n$	2 recursions/nested loops
$n \mapsto \underbrace{2^{2^{2^{\cdot^{\cdot^{\cdot^{2}}}}}}}_{n} = \mathrm{TOWER}(n)$	3 recursions/nested loops
$n \mapsto \underbrace{\mathrm{TOWER}(\cdots \mathrm{TOWER}(1) \cdots)}_{n \text{ nestings}}$	4 recursions/nested loops
$= \mathrm{TOWER}^n(1)$	

- And so on... Here: Each function "iterates" the last. Primitive recursive functions can grow fast!
- They are quite powerful in general $\rightarrow$ were conjectured to contain all intuitively computable functions.
- However, there are computable functions that grow faster than every primitive recursive function.

The Ackermann function

The Ackermann function

The Ackermann function $A\colon \mathbb{N}^2 \to \mathbb{N}$ is defined as follows:

$$A(0,n) = n+1$$
$$A(m+1,0) = A(m,1) \qquad A(m+1,n+1) = A(m,A(m+1,n))$$

We think of it as a function $A(m,\cdot)\colon \mathbb{N} \to \mathbb{N}$ for each $m \in \mathbb{N}$. Note:

$$\begin{aligned} A(m+1,3) &= A(m,A(m+1,2)) = A(m,A(m,A(m+1,1))) \\ &= A(m,A(m,A(m,A(m+1,0)))) \\ &= A(m,A(m,A(m,A(m,1)))) \end{aligned}$$

More generally:

$$\begin{aligned} A(0,n) &= n+1 \\ A(m+1,n) &= \underbrace{A(m,A(m,A(m,\ldots A(m,1)\cdots)}_{n+1\text{-fold nesting}} = A_m^{n+1}(1). \end{aligned}$$

where we write $A_m(n) := A(m,n)$ for $m,n \in \mathbb{N}$.
Thus: A_{m+1} is the iterated application of A_m to 1.

Ackermann function: properties

Examples:

$$
\begin{aligned}
A_0(n) &= n+1 \\
A_1(n) &= n+2 \\
A_2(n) &= 2n+3 \\
A_3(n) &= 2^{n+3}-3 \\
A_4(n) &= \underbrace{2^{2^{2^{\cdot^{\cdot^{\cdot^{2}}}}}}}_{n+3} - 3
\end{aligned}
$$

1. Easy to imagine, not very difficult to show:
   1.1 If $n < k$, then $A_m(n) < A_m(k)$.
   1.2 If $m < \ell$, then $A_m(n) < A_\ell(n)$.
   1.3 $A_m(n+1) \leq A_{m+1}(n)$.
2. $A_m(A_\ell(n)) < A_{m+\ell+2}(n)$ for all $n \in \mathbb{N}$:

$$
\begin{aligned}
A_m(A_\ell(n)) \leq A_{m+\ell}(A_\ell(n)) &< A_{m+\ell}(A_{m+\ell+1}(n)) \\
&= A_{m+\ell+1}(n+1) \leq A_{m+\ell+2}(n).
\end{aligned}
$$

The Ackermann function is not primitive recursive

Theorem

The Ackermann function is not primitive recursive.

Intuition:

- To construct primitive recursive functions that grow fast, we can take a function $f : \mathbb{N} \to \mathbb{N}$ and iterate it in $f' : \mathbb{N} \to \mathbb{N}$ with $f'(n) = f^n(1)$.
- For each primitive recursive function, this can be repeated a bounded number of times, e.g. s, s′, s″, s‴, etc.
  - The number of times we apply primitive recursion
  - Nesting depth of loops in loop programs
- The Ackermann function repeats the "iteration process" as often as the input specifies! $A_m(n) = A(m, n)$ would need m of such iteration nestings.

We will show the theorem by proving that the Ackermann functions "grows faster" than any primitive recursive function. Formally:

Majorization

For $\mathbf{u} \in \mathbb{N}^k$, $\mathbf{u} = (u_1, \ldots, u_k)$, let $|\mathbf{u}| = \max\{u_1, \ldots, u_k\}$.
We say that a function $f\colon \mathbb{N}^k \to \mathbb{N}$ is *majorized* by $g\colon \mathbb{N} \to \mathbb{N}$ if $f(\mathbf{u}) < g(|\mathbf{u}|)$ for every $\mathbf{u} \in \mathbb{N}^k$.

Lemma

For every primitive recursive function f, there exists an $m \in \mathbb{N}$ such that f is majorized by A_m.

Intuition: The m is the "nesting depth" mentioned above.
This clearly implies that A cannot be primitive recursive:
Otherwise, A would be majorized by some A_m. But then $A(m, m) < A_m(m) = A(m, m)$, a contradiction.

1. Zero: $\mathsf{z}(n) = 0 < n + 1 = A_0(n)$.
2. Successor: $\mathsf{s}(n) = n + 1 < n + 2 = A_1(n)$.
3. Projections: for $\mathbf{u} = (u_1, \ldots, u_k)$, we have: $\pi_\ell^k(\mathbf{u}) = u_\ell < |\mathbf{u}| + 1 = A_0(|\mathbf{u}|)$.

Composition

Suppose $g_1, \ldots, g_\ell \colon \mathbb{N}^m \to \mathbb{N}$ and $h \colon \mathbb{N}^\ell \to \mathbb{N}$ are majorized by $A_{r_1}, \ldots, A_{r_\ell}$ and A_s. Consider $f(\mathbf{u}) = h(g_1(\mathbf{u}), \ldots, g_\ell(\mathbf{u}))$. For $\mathbf{u} \in \mathbb{N}^m$, choose $j \in [1, \ell]$ so that $g_j(\mathbf{u}) = \max\{g_i(\mathbf{u}) \mid i \in [1, \ell]\}$. Then we have:

$$f(\mathbf{u}) < A_s(g_j(\mathbf{u})) < A_s(A_{r_j}(|\mathbf{u}|)) < A_{s+r_j+2}(|\mathbf{u}|) \leq A_{s+r+2}(|\mathbf{u}|).$$

where $r = \max\{r_1, \ldots, r_\ell\}$. Thus, f is majorized by A_{s+r+2}.

Primitive recursion

Suppose $g\colon \mathbb{N}^k \to \mathbb{N}$ and $h\colon \mathbb{N}^{k+2} \to \mathbb{N}$ and consider $f\colon \mathbb{N}^{k+1} \to \mathbb{N}$ with $f(\mathbf{u}, 0) = g(\mathbf{u})$ and $f(\mathbf{u}, n+1) = h(\mathbf{u}, n, f(\mathbf{u}, n))$. Moreover, let g and h be majorized by A_r and A_s, respectively. Take $q = 1 + \max\{r, s\}$. We claim that

$$f(\mathbf{u}, n) < A_q(n + |\mathbf{u}|) \quad \text{for all } \mathbf{u},\ n$$

We proceed by induction on n. First, $f(\mathbf{u}, 0) = g(\mathbf{u}) < A_r(|\mathbf{u}|) < A_q(|\mathbf{u}|)$. Moreover:

$$f(\mathbf{u}, n+1) = h(\mathbf{u}, n, f(\mathbf{u}, n)) < A_s(\underbrace{\max\{|\mathbf{u}|, n, f(\mathbf{u}, n)\}}_{=:M}).$$

Let us bound M. Induction yields $f(\mathbf{u}, n) < A_q(n + |\mathbf{u}|)$ and since $\max\{|\mathbf{u}|, n\} \leq n + |\mathbf{u}| < A_q(n + |\mathbf{u}|)$, we have $M < A_q(n + |\mathbf{u}|)$. Therefore,

$$\begin{aligned} f(\mathbf{u}, n+1) < A_s(M) &< A_s(A_q(n + |\mathbf{u}|)) \\ &\leq A_{q-1}(A_q(n + |\mathbf{u}|)) = A_q(n + 1 + |\mathbf{u}|). \end{aligned}$$

This proves the claim. Therefore, with $t := \max\{|\mathbf{u}|, n\}$, we have: $f(\mathbf{u}, n) < A_q(n+|\mathbf{u}|) \leq A_q(2t) < A_q(2t+3) = A_q(A_2(t)) < A_{q+4}(t)$.

This establishes the lemma and thus the theorem.

The following is the main result of this section:

Theorem (Ernst Mayr & Albert Meyer 1981)

For each $m \in \mathbb{N}$, there is a VASS $\mathcal{V} = (Q, T, q_0)$ of size polynomial in m such that $\mathrm{Reach}(\mathcal{V})$ *is finite, but* $|\mathrm{Reach}(\mathcal{V})| \geq A(m, m)$.

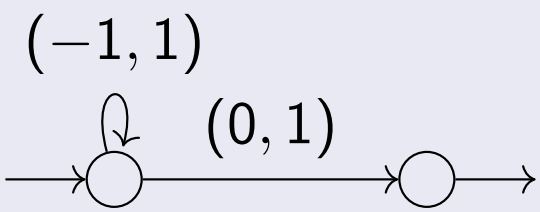
Given m, we construct a d-VASS $\mathcal{V}_m$ for some $d \in \mathbb{N}$ of size polynomial in m with distinguished states q_0 and q_1 such that:

- for any $\mathbf{u} \in \mathbb{N}^d$, with $(q_0, \mathbf{u})$ as its initial configuration, $\mathcal{V}_m$ is terminating, and
- for every $n \in \mathbb{N}$, we have $(q_0, (n, \mathbf{0})) \to^* (q_1, (\mathbf{0}, A_m(n)))$.

Here, $(n, \mathbf{0})$ stands for $(n, 0, \ldots, 0) \in \mathbb{N}^d$ and $(\mathbf{0}, A_m(n))$ stands for $(0, \ldots, 0, A_m(n)) \in \mathbb{N}^d$.
Graphically, the distinguished states q_0 and q_1 are denoted by

respectively.

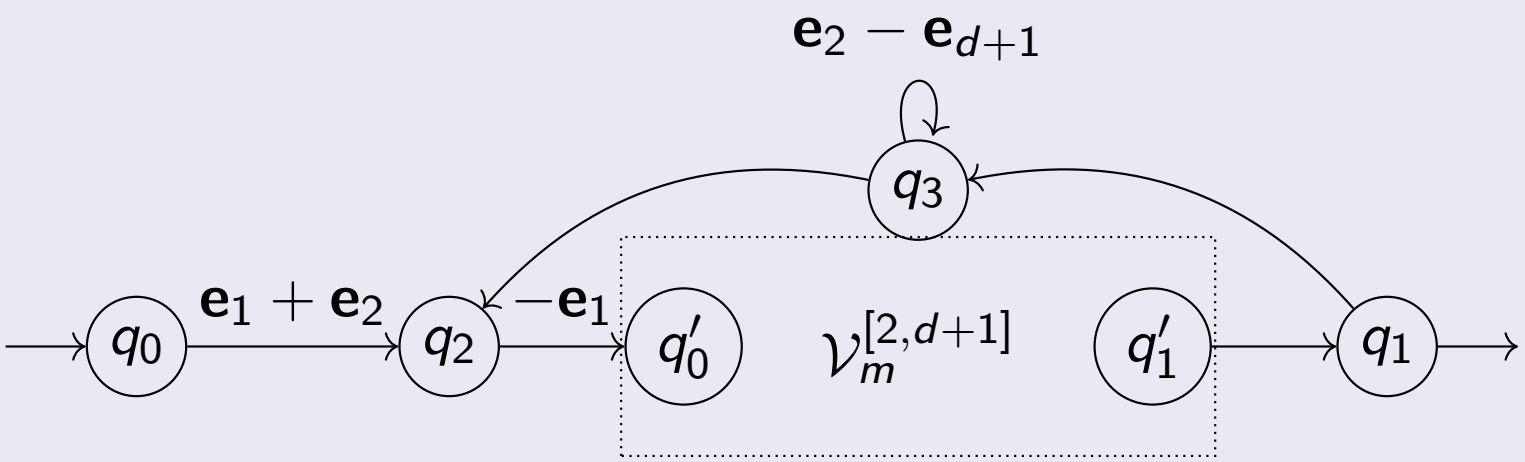
$\mathcal{V}_0$



- $\mathcal{V}_0$ is terminating for every $(q_0, \mathbf{u})$ as initial configuration.
- Also $A_0(n) = n + 1$ and $(q_0, (n, 0)) \to^* (q_1, (0, n + 1))$.

Suppose we have constructed a d-VASS $\mathcal{V}_m$. Let $\mathcal{V}_m^{[2,d+1]}$ be $\mathcal{V}_m$, but using counters $\{2, \ldots, d+1\}$ instead of $\{1, \ldots, d\}$.

$\mathcal{V}_{m+1}$



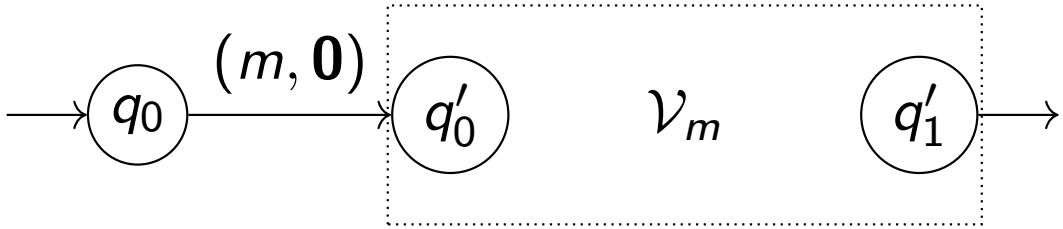
- $\mathcal{V}_{m+1}$ is terminating: Every time we take the loop from q_2 to q_2, the first counter is decremented, but it is never incremented. Thus, an infinite run would have to happen inside $\mathcal{V}_m^{[2,d+1]}$, which is impossible by assumption.
- Because of $\mathcal{V}_m$, for $k \geq 0$, we have $(q_2, (n, k, \mathbf{0})) \to^* (q_1, (n-1, \mathbf{0}, A_m(k))) \to^* (q_2, (n-1, A_m(k), \mathbf{0}))$. Hence: $(q_0, (n, \mathbf{0})) \to (q_2, (n+1, 1, \mathbf{0}))) \to^* (q_2, (n, A_m(1), \mathbf{0})) \to^* (q_2, (1, A_m^n(1), \mathbf{0})) \to^* (q_1, (\mathbf{0}, A_m^{n+1}(1))) = (q_1, (\mathbf{0}, A_{m+1}(n)))$.

We are now ready to prove:

Theorem (Ernst Mayr & Albert Meyer 1981)

For each $m \in \mathbb{N}$, there is a VASS $\mathcal{V} = (Q, T, q_0)$ of size polynomial in m such that $\mathrm{Reach}(\mathcal{V})$ *is finite, but* $|\mathrm{Reach}(\mathcal{V})| \geq A(m, m)$.

- $\mathcal{V}_m = (Q, T, q_0)$ is an $(m+2)$-VASS with $4m+2$ states and $6m+2$ transitions and $\|T\| = 2$.
- We use this to construct $\bar{\mathcal{V}}_m$:



- $\bar{\mathcal{V}}_m = (\bar{Q}, \bar{T}, \bar{q}_0)$ is a $(m+2)$-VASS with $|\bar{Q}| = 4m+3$ and $|\bar{T}| = 6m+3$, and $\|\bar{T}\| \leq m+2$. Hence, the size of $\bar{\mathcal{V}}_m$ is polynomial in m.
- Since $(q_0, \mathbf{0}) \to^* (q_1', (\mathbf{0}, A_m(m))) = (q_1', (\mathbf{0}, A(m, m)))$, we have $|\mathrm{Reach}(\bar{\mathcal{V}}_m)| \geq A(m, m)$.
- $\bar{\mathcal{V}}_m$ is clearly terminating by construction of $\mathcal{V}_m$. Hence $\mathrm{Reach}(\bar{\mathcal{V}}_m)$ is finite. $\square$

Part IV

Vector addition systems

Section 15

Equivalence of VASS

Deciding equivalence of programs

- Given two programs, when can we say they behave the same way?
- In the case of VASS, given two VASS as input, do they have the same reachability set?
- Since reachability is decidable, one may be tempted to think this problem is decidable.
- Unfortunately not.

Undecidability of VASS reachability set equality

Theorem (Rabin, unpublished)

The following problem is undecidable: Given two VASS $\mathcal{V}_1$ and $\mathcal{V}_2$, does $\mathrm{Reach}(\mathcal{V}_1) = \mathrm{Reach}(\mathcal{V}_2)$ *hold?*

We will follow a simpler proof by Jančar.
We reduce from the global state reachability of **deterministic two counter machines** (D2CM). Recall that if there is a run of such a machine reaching the target state, it is *unique*.
Easy to construct VASS $\mathcal{V}$ which simulates a D2CM $\mathcal{M}$ when allowed to make a mistake on the zero test.
We construct $\mathcal{V}_1$ and $\mathcal{V}_2$ from $\mathcal{V}$, by addressing the following issues:

1. If a mistake is made on a zero test, how can it be reflected in the set Reach?
2. How can we ensure that $\mathrm{Reach}(\mathcal{V}_1) = \mathrm{Reach}(\mathcal{V}_2)$ iff $\mathcal{M}$ cannot reach q_f?

VASS $\mathcal{V}$ weakly simulating $\mathcal{M}$

Let $\mathcal{M} = (Q, T, q_0)$ with a designated final state q_f where T has transitions of the following forms:

(t1) $q \xrightarrow{+e_k} q'$

(t2) $q \xrightarrow{z_k} q'$, $q \xrightarrow{-e_k} q''$.

In the second case, the initial state q is the same in both transitions.

We will write $c{+}{+}, c{-}{-}$ on a transition with the assumption that there is some intermediate state which is not of importance.

Define VASS $\mathcal{V} = (Q', T', q_0')$ as follows:

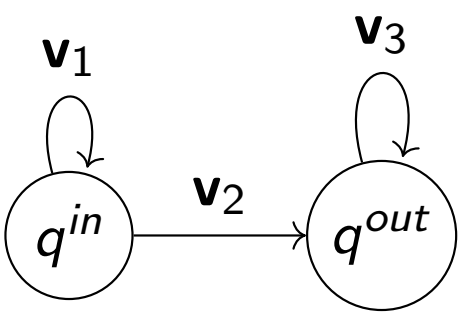
- Retain each transition of type $t1$.
- For transitions of type $t2$, add $t_i^Z = (q \xrightarrow{0} q')$, $t_i^{NZ} = (q \xrightarrow{-e_k} q'')$ and $t_i' = (q \xrightarrow{-e_k, +e_k} q')$.

The transitions t_i' are said to be *definitely cheating*.

An intermediate VASS $\mathcal{V}'$

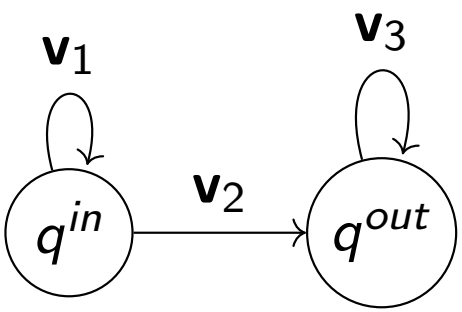
We construct an VASS $\mathcal{V}'$ from $\mathcal{V}$, which we will then modify to obtain $\mathcal{V}_1$ and $\mathcal{V}_2$.

- Add 5 additional counters $r_1, r_2, STEPS, TNZ, DOUBLE$.
- Initialize r_1 to 1, other four counters to 0.
- Modify each transition in $\mathcal{V}$ to increment $r_2, STEPS$ and decrement r_1. If the transition is a t_i^{NZ}, then also increment TNZ.
- For each state q of $\mathcal{V}$ do the following: Replace the state with two copies q^{in} and q^{out}, modify each transition such that incoming transitions come into q^{in} and outgoing transitions go out of q^{out}, and add the following transitions:



$\mathbf{v}_1 = r_2{--}, TNZ{--}, r_2{++}, DOUBLE{+}= 2,$
$\mathbf{v}_2 = r_2{--}, r_1{++}$, and
$\mathbf{v}_3 = r_1{--}, DOUBLE{--}, r_1{++}, TNZ{++}.$

Understanding the gadget



$\mathbf{v}_1 = r_2{--}, TNZ{--}, r_2{++}, DOUBLE{+}= 2,$
$\mathbf{v}_2 = r_2{--}, r_1{++}$, and
$\mathbf{v}_3 = r_1{--}, DOUBLE{--}, r_1{++}, TNZ{++}.$

- The gadget enables us to double the number of tokens in counter TNZ after each transition of $\mathcal{V}$.
- The number of tokens in r_1 and r_2 always adds up to 1.
- While executing a transition in $\mathcal{V}$, we move the token from r_1 to r_2 in $\mathcal{V}'$ while incrementing $STEPS$. If it is a t_i^{NZ} transition, we also increment TNZ.
- The loop on q^{in} verifies that r_2 contains a token, then puts two tokens in $DOUBLE$ for each token in TNZ.
- We then shift the token back to r_1 from r_2 and move to q^{out}.
- At q^{out}, we are allowed to shift tokens from $DOUBLE$ to TNZ.

Gadget lemma

Let t_1, t_2, t_3 be the transitions corresponding to $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ respectively. Let $\sigma_u = t_1 t_1 \ldots t_1 t_2 t_3 t_3 \ldots t_3$.

Lemma

Suppose the unique computation in $\mathcal{M}$ reaching q_f takes k steps. Then $\sigma = t^1 \sigma_u^1 t^2 \sigma_u^2 \ldots t^k \sigma_u^k$ is allowed in $\mathcal{V}'$, where the t^i are counted transitions (which lead to incrementing $STEPS$) which are noncheating and the σ_u^i are instances of σ_u which cause the maximum possible increase of TNZ.
Let C be the configuration reached at the end of σ and C' be the configuration reached at the end of any other run σ' which contains k counted transitions. Then $C'(TNZ) < C(TNZ)$.

Proof of gadget lemma

It suffices to consider σ' which use instances of σ_u which maximally increase TNZ.
Hence we can assume that $\sigma' = z^1\sigma_u^1 z^2\sigma_u^2 \ldots z^k\sigma_u^k$.
Observe that if the bit representation of $C'(TNZ) = b_1 b_2 \ldots b_{k+1}$, then $b_i = 1$ iff z^i is some t_j^{NZ}.
Furthermore, σ' can only differ from σ on a t_j^{NZ} transition at the first place where they differ: instead it uses t_j^Z or t_j'.
In this case, the corresponding bit in $C'(TNZ)$ is a 0 instead of a 1 in $C(TNZ)$.
Since this bit can never be changed later in the computation, the lemma follows. $\square$

Constructing $\mathcal{V}_1$ and $\mathcal{V}_2$

- To $\mathcal{V}'$, add two more counters p, p' and initialize them with $1, 0$ tokens respectively.
- Modify each definitely cheating transition t'_i to also shift the token from p to p'.
- Add a new state q'_f and a transition $t_a = (q_f^{out} \xrightarrow{0} q'_f)$. At this point, we have constructed $\mathcal{V}_1$.
- To $\mathcal{V}_1$ add the transition $t_b = (q_f^{out} \xrightarrow{p--,p'++} q'_f)$ to get $\mathcal{V}_2$.

We will now show that $\mathcal{M}$ halts iff $\text{Reach}(\mathcal{V}_1) \neq \text{Reach}(\mathcal{V}_2)$.

Proof of theorem

<u>$\mathcal{M}$ halts $\Rightarrow$ Reach($\mathcal{V}_1$) $\neq$ Reach($\mathcal{V}_2$)</u>: Consider the correct simulation $\sigma = t^1\sigma_u^1 t^2\sigma_u^2 \dots t^k\sigma_u^k t^{k+1}$ where $t^{k+1} = t_b$ in $\mathcal{V}_2$. Let C be the configuration at the end. Consider any σ' in $\mathcal{V}_1$ which ends in the same configuration C: it has to contain exactly k counted transitions since $C(STEPS) = k$. Furthermore, σ' has to attain $C(TNZ)$ and by the gadget lemma, it has to have a prefix $t^1\sigma_u^1 t^2\sigma_u^2 \dots t^k\sigma_u^k$. But only t_a can be applied in $\mathcal{V}_1$, which implies that the token cannot be shifted from p to p'. Contradiction!

<u>$\mathcal{M}$ does not halt $\Rightarrow$ Reach($\mathcal{V}_1$) = Reach($\mathcal{V}_2$)</u>: Note that t_b can be used at most once in $\mathcal{V}_2$. Let σ' be any sequence ending with t_b: it cannot have any definitely cheating transitions because this moves the token from p to p'. However, since there is no run in $\mathcal{M}$ to q_f, this means σ' must have cheated on a t_i^Z transition. The same configuration can be reached in $\mathcal{V}_1$ by replacing t_i^Z by t_i' and the final t_b by t_a. $\square$

Part V

Integer VASS

Section 16

Integer VASS

The reachability problem for VASS has Ackermannian complexity. In the following we consider simpler variants for which we can decide reachability (and other problems) more efficiently.

- counters which range over $\mathbb{Z}$ instead of $\mathbb{N}$
- counters where the number of reversals is bounded
- add a stack

The reachability problem for $\mathcal{M}$

Given A system $M \in \mathcal{M}$ and a configuration γ.

Question Is $\gamma_0 \rightarrow_M^* \gamma$ where γ_0 is the initial configuration?

Integer VASS

Definition

A *d-dimensional integer VASS (or $\mathbb{Z}$-VASS)* $V = (Q, T, q_0)$ is syntactically the same as a d-dimensional VASS.
Configurations in V are pairs $(q, \mathbf{u}) \in Q \times \mathbb{Z}^d$, and we define $(q, \mathbf{u}) \to (q', \mathbf{u}')$ if there exists $(q, \mathbf{v}, q') \in T$ such that $\mathbf{u}' = \mathbf{u} + \mathbf{v}$.

$\mathbb{Z}$-VASS are an *overapproximation* of VASS: The reachability set of a $\mathbb{Z}$-VASS V contains the reachability set of V viewed as a VASS.

We will prove the following result:

Theorem

The reachability problem for $\mathbb{Z}$-VASS is in NP.

For the proof we will introduce two important tools from logic and formal language theory:

- Presburger arithmetic
- Parikh's theorem

Part V

Integer VASS

Section 17

Presburger arithmetic

Presburger arithmetic

Definition

Presburger arithmetic (PA) is the first-order theory of $\mathcal{N} = (\mathbb{N}, +, <, 0, 1)$, i.e. the set of true first-order sentences in $\mathcal{N}$.

Recall: First-order logic, also known as *predicate logic*, uses variables which range over elements of the domain, in contrast to second-order logic which can quantify over sets and relations.

Example

- $\exists x \forall y (x \neq y + y)$
- $\forall z (z > 1 \rightarrow \exists x \exists y (z = 2x + 3y))$
- $<$, 0, 1 can be defined using $+$ only.

Sometimes PA is also defined as $\mathcal{Z} = (\mathbb{Z}, +, <, 0, 1)$. Equivalence:

- $\mathcal{N} \rightarrow \mathcal{Z}$: impose $x \geq 0$ on all variables
- $\mathcal{Z} \rightarrow \mathcal{N}$: replace every integer variable x by $x_+ - x_-$.

Recap (logic)

- By $\varphi(x_1, \ldots, x_d)$ we denote a formula whose free variables are amongst the variables $x_1, \ldots, x_d$.
- A formula without free variables is a *sentence*.
- We write tuples in boldface $\mathbf{x} = (x_1, \ldots, x_d)$.
- A formula $\varphi(x_1, \ldots, x_d)$ *defines* the relation

$$R = \{(n_1, \ldots, n_d) \in \mathbb{N}^d \mid \varphi(n_1, \ldots, n_d) \text{ is true}\}.$$

We then call R *Presburger definable*.

Theorem (Presburger 1929)

Presburger arithmetic is decidable.

Presburger's proof uses quantifier elimination. We first present the proof by Büchi (1960) using finite automata.
Given words $w_1 = a_{1,1} \dots a_{1,n}, \dots, w_d = a_{d,1} \dots a_{d,n}$ over Σ we define their *convolution*

$$w_1 \otimes \cdots \otimes w_d = \begin{bmatrix} a_{1,1} \\ \vdots \\ a_{d,1} \end{bmatrix} \dots \begin{bmatrix} a_{1,n} \\ \vdots \\ a_{d,n} \end{bmatrix} \in (\Sigma^d)^*.$$

A *(synchronous) d-tape automaton* is an NFA $\mathcal{A}$ over the alphabet $\{0,1\}^d$. It defines a d-ary relation $R(\mathcal{A}) \subseteq \mathbb{N}^d$ by

$$R(\mathcal{A}) = \{(\mathrm{val}(w_1), \dots, \mathrm{val}(w_d)) \mid \mathcal{A} \text{ accepts } w_1 \otimes \cdots \otimes w_d\}$$

where $\mathrm{val}(a_0 \dots a_n) = \sum_{i=0}^{n} a_i 2^i$.

From Presburger arithmetic to automata

Proposition

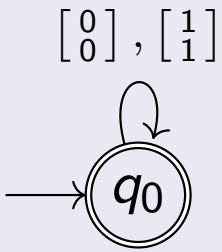
Given a Presburger formula $\varphi(x_1, \ldots, x_d)$, one can construct a d-tape automaton $\mathcal{A}_\varphi$ such that

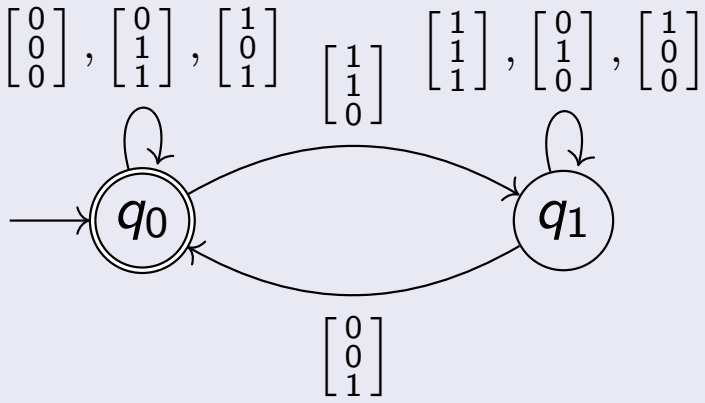
$$R(\mathcal{A}_\varphi) = \{\mathbf{n} \in \mathbb{N}^d \mid \varphi(\mathbf{n}) \text{ is true}\}.$$

Proof.

Induction on the structure of φ.

Induction base: $x_i = x_j$ and $x_i + x_j = x_k$





From Presburger arithmetic to automata

Proposition

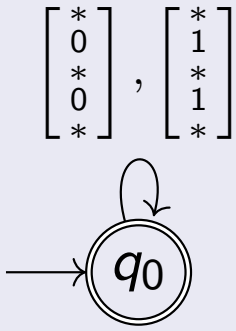
Given a Presburger formula $\varphi(x_1, \ldots, x_d)$, one can construct a d-tape automaton $\mathcal{A}_\varphi$ such that

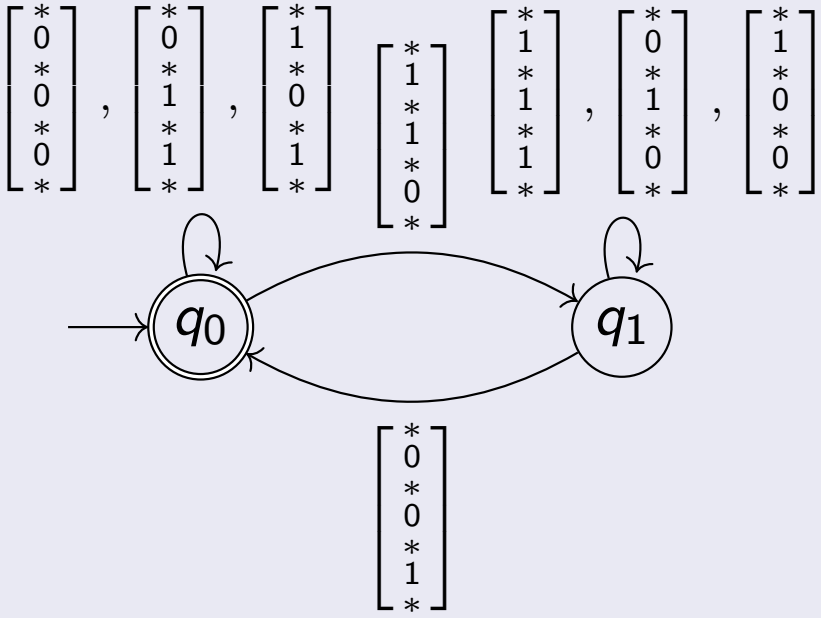
$$R(\mathcal{A}_\varphi) = \{\mathbf{n} \in \mathbb{N}^d \mid \varphi(\mathbf{n}) \text{ is true}\}.$$

Proof.

Induction on the structure of φ.

Induction base: $x_i = x_j$ and $x_i + x_j = x_k$





Proof.

Induction step: $\vee$, $\neg$, $\exists$

- $R(\mathcal{A}_{\varphi\vee\psi}) = R(\mathcal{A}_\varphi) \cup R(\mathcal{A}_\psi)$. Take the union of $\mathcal{A}_\varphi$ and $\mathcal{A}_\psi$.
- For formulas $\exists x_i \varphi$ take $\mathcal{A}_\varphi$ and remove all i-th components in the transition labels.
- $R(\mathcal{A}_{\neg\varphi}) = \mathbb{N}^d \setminus R(\mathcal{A}_\varphi)$.
  Problem: A vector $\mathbf{n} \in \mathbb{N}^d$ has many representations in $(\{0,1\}^d)^*$ caused by trailing $\mathbf{0} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$.

  Let $L = \{w \in (\{0,1\}^d)^* \mid w \text{ does not end with } \mathbf{0}\}$.

  Ensure that $L(\mathcal{A}_\varphi) \subseteq L$: Make all states in $\mathcal{A}_\varphi$ final from which there is a path to a final state labelled by $\mathbf{0}^*$, and then intersect $\mathcal{A}_\varphi$ with L.

  Construct an automaton $\mathcal{A}_{\neg\varphi}$ for $L \setminus L(\mathcal{A}_\varphi)$. $\square$

For Presburger sentences φ we can construct an automaton $\mathcal{A}_\varphi$ which accepts the empty tuple if and only if φ is true. This proves the decidability of Presburger arithmetic.

Remark

Automatic structures are automata-presented logical structures with decidable first-order theories.
Examples: $(\mathbb{N}, +)$, $(\mathbb{R}, +)$, infinite binary tree

Complexity: Every negation (or quantifier alternation) causes an exponential blowup in the automaton size. Hence the total size can be *non-elementary* in the formula length.

Theorem (Oppen 1978)

Presburger arithmetic can be decided in time $2^{2^{2^{O(n)}}}$.

Quantifier elimination

In the following we will work in $\mathcal{Z} = (\mathbb{Z}, +, <, 0, 1)$.

A logic $\mathcal{L}$ *admits quantifier elimination* if every $\mathcal{L}$-formula is equivalent to a quantifier-free $\mathcal{L}$-formula.
Clearly, $\varphi(x) = \exists y(x = y + y)$ cannot be expressed without quantifiers. Hence Presburger arithmetic does *not* admit quantifier elimination.

We need to enrich $\mathcal{Z}$ by *divisibility predicates*. For every constant $c \in \mathbb{Z} \setminus \{0\}$ we have a predicate

$$c \mid x \iff \text{there exists } y \in \mathbb{Z} \text{ such that } x = cy.$$

- We allow linear (in)equalities with *integer* coefficients, e.g. $2x - 3y - 5z > -6$, which is equivalent to $2x + 6 > 3y + 5z$.
- We also allow integer coefficients in the divisibility terms, e.g. $3 \mid (x - 7y - 5z - 13)$ is equivalent to $3 \mid (x + 2y + z + 2)$.

Theorem

First-order logic over $\mathcal{Z}_{mod} = (\mathbb{Z}, +, <, 0, 1, \{c \mid \cdot\}_{c \neq 0})$ admits effective quantifier elimination.

Effective means that the transformation can be computed by an algorithm.
We claim that every $\mathcal{Z}_{\text{mod}}$-formula φ is effectively equivalent to a quantifier-free formula φ^* which is built from $\vee$, $\wedge$, linear inequalities $s < t$ and divisibility constraints $c \mid s$.

Consider a $\mathcal{Z}_{mod}$-formula φ.

- If φ is a linear inequality or a divisibility constraint then we can set $\varphi^* = \varphi$.
- If φ is an equality $s = t$ we set $\varphi^* = (s < t+1) \wedge (t < s+1)$.
- If $\varphi = \varphi_1 \vee \varphi_2$ we set $\varphi^* = \varphi_1^* \vee \varphi_2^*$.
- If $\varphi = \varphi_1 \wedge \varphi_2$ we set $\varphi^* = \varphi_1^* \wedge \varphi_2^*$.
- If $\varphi = \neg\psi$ then φ is equivalent to $\neg\psi^*$. In $\neg\psi^*$ we push the negations inwards and replace

$$\neg(s < t) \quad \text{by} \quad s > t - 1$$

and

$$\neg(c \mid s) \quad \text{by} \quad \bigvee_{r \in \{1,\ldots,c-1\}} c \mid (s + r).$$

It remains to show how to eliminate existential quantifiers.

Consider a formula $\varphi = \exists y \psi(\mathbf{x}, y)$. We construct the formula ψ^* and bring it into disjunctive normal form. We can write:

$$\psi^*(\mathbf{x}, y) = \bigvee_{i \in I} (\psi_{i,1} \wedge \cdots \wedge \psi_{i,k_i} \wedge \theta_i)$$

where

- the formulas $\psi_{i,1}, \ldots, \psi_{i,k_i}$ are linear inequalities and divisibility constraints in which y occurs with a nonzero coefficient,
- the formulas θ_i do not contain y.

Therefore $\varphi = \exists y \psi$ is equivalent to

$$\bigvee_{i \in I} (\exists y (\psi_{i,1} \wedge \cdots \wedge \psi_{i,k_i}) \wedge \theta_i),$$

and it remains to eliminate the quantifier in $\exists y (\psi_{i,1} \wedge \cdots \wedge \psi_{i,k_i})$.

Consider a formula of the form

$$\exists y \big(\bigwedge_{i \in I} p_i(\mathbf{x}, y) < b_i \ \wedge \ \bigwedge_{k \in K} c_k \mid r_k(\mathbf{x}, y) \big)$$

where y has a nonzero coefficient in every p_i and r_k.
Rewrite

$$\exists y \big(\bigwedge_{i \in I} p'_i(\mathbf{x}) < a_i y \ \wedge \ \bigwedge_{k \in K} c_k \mid (a_k y + r'_k(\mathbf{x})) \big)$$

where $a_i \in \mathbb{Z} \setminus \{0\}$ and p'_i, r'_k are linear terms.

Next we ensure that the coefficients of y are positive.
Let $I_+ = \{i \in I \mid a_i > 0\}$ and $I_- = I \setminus I_+$. Then we can rewrite:

$$\exists y \big(\bigwedge_{i \in I_+} p'_i(\mathbf{x}) < a_i y \wedge \bigwedge_{i \in I_-} -a_i y < -p'_i(\mathbf{x}) \wedge \bigwedge_{k \in K} c_k \mid (\epsilon_k a_k y + \epsilon_k r'_k(\mathbf{x})) \big)$$

where $\epsilon_k = 1$ if $a_k > 0$ and $\epsilon_k = -1$ otherwise.

Hence, we obtain the following form (after renaming the terms):

$$\exists y \big(\bigwedge_{i \in I} p_i(\mathbf{x}) < a_i y \ \wedge \ \bigwedge_{j \in J} a_j y < q_j(\mathbf{x}) \ \wedge \ \bigwedge_{k \in K} c_k \mid (a_k y + r_k(\mathbf{x})) \big)$$

where $a_i > 0$ for all $i \in I \cup J \cup K$.
Ensure that y has coefficient $A = \prod_{i \in I \cup J \cup K} a_i$ everywhere:

- Multiply both sides of $p_i(\mathbf{x}) < a_i y$ by A/a_i.
- Multiply both sides of $a_j y < q_j(\mathbf{x})$ by A/a_j.
- Multiply both sides of $c_k \mid (a_k y + r_k(\mathbf{x}))$ by A/a_k.

$$\exists y \big(\bigwedge_{i \in I} \frac{A}{a_i} p_i(\mathbf{x}) < Ay \wedge \bigwedge_{j \in J} Ay < \frac{A}{a_j} q_j(\mathbf{x}) \wedge \bigwedge_{k \in K} \frac{A}{a_k} c_k \mid (Ay + \frac{A}{a_k} r_k(\mathbf{x})) \big)$$

$$\exists y\big(\bigwedge_{i\in I}\frac{A}{a_i}p_i(\mathbf{x}) < Ay \wedge \bigwedge_{j\in J} Ay < \frac{A}{a_j}q_j(\mathbf{x}) \wedge \bigwedge_{k\in K}\frac{A}{a_k}c_k \mid (Ay+\frac{A}{a_k}r_k(\mathbf{x}))\big)$$

Now replace Ay by z and add the condition $A \mid z$:

$$\begin{aligned}\exists z\big(A \mid z \ \wedge \ \bigwedge_{i\in I}\frac{A}{a_i}p_i(\mathbf{x}) < z \ \wedge \ \bigwedge_{j\in J} z < \frac{A}{a_j}q_j(\mathbf{x}) \\ \wedge \ \bigwedge_{k\in K}\frac{A}{a_k}c_k \mid (z + \frac{A}{a_k}r_k(\mathbf{x}))\big)\end{aligned}$$

Observe: $(\mathbf{x}, y)$ satisfies the original formula iff $(\mathbf{x}, Ay)$ satisfies the new formula.

Now z has coefficient 1 everywhere:

$$\Phi(\mathbf{x}, z) = \bigwedge_{i \in I} p_i(\mathbf{x}) < z \ \wedge \ \bigwedge_{j \in J} z < q_j(\mathbf{x}) \ \wedge \ \bigwedge_{k \in K} c_k \mid (z + r_k(\mathbf{x}))$$

for some (new) linear terms p_i, q_j, r_k.

$\rightarrow$ Is there an integer $z \in \mathbb{Z}$

- which lies strictly between $\max_{i \in I} p_i(\mathbf{x})$ and $\min_{j \in J} q_j(\mathbf{x})$,
- and satisfies the divisibility constraints?

Intuitively: Look for solutions "close" to $\max_{i \in I} p_i(\mathbf{x})$ or $\min_{j \in J} q_j(\mathbf{x})$.

$$\Phi(\mathbf{x}, z) = \bigwedge_{i \in I} p_i(\mathbf{x}) < z \ \wedge \ \bigwedge_{j \in J} z < q_j(\mathbf{x}) \ \wedge \ \bigwedge_{k \in K} c_k \mid (z + r_k(\mathbf{x}))$$

The divisibility constraints can be written as a system of congruences:

$$z \equiv -r_k(\mathbf{x}) \mod c_k \quad \text{for } k \in K$$

By the Chinese remainder theorem: If the congruence system has a solution y_0 then the solution set is $\{y_0 + \lambda C \mid \lambda \in \mathbb{Z}\}$ where $C = \mathrm{lcm}\{c_k \mid k \in K\}$.

Lemma

One can construct a finite set of linear terms S over $\mathbf{x}$ such that $\exists z \Phi(\mathbf{x}, z)$ is equivalent to $\bigvee_{s \in S} \Phi(\mathbf{x}, s(\mathbf{x}))$.

Proof.

$$\Phi(\mathbf{x}, z) = \bigwedge_{i \in I} p_i(\mathbf{x}) < z \ \wedge \ \bigwedge_{j \in J} z < q_j(\mathbf{x}) \ \wedge \ \bigwedge_{k \in K} c_k \mid (z + r_k(\mathbf{x}))$$

Case 1. $I = J = \emptyset$: Set $S = \{0, \ldots, C-1\}$.

Case 2. $I \neq \emptyset$: Set $S = \{p_i(\mathbf{x}) + c \mid i \in I,\ 1 \leq c \leq C\}$. Suppose that $(\mathbf{u}, v)$ is a solution of Φ such that $v = \min\{v' \in \mathbb{Z} \mid \Phi(\mathbf{u}, v') \text{ is true}\}$. Let $i \in I$ where $p_i(\mathbf{u})$ is maximal. Then $v > p_i(\mathbf{u})$ by the inequality in Φ. Furthermore $v \leq p_i(\mathbf{u}) + C$; otherwise $(\mathbf{u}, v - C)$ would also be a solution, contradicting the minimality of v. Therefore $v = p_i + c$ for some $1 \leq c \leq C$.

Case 3. $I = \emptyset, J \neq \emptyset$: Set $S = \{q_j(\mathbf{x}) - c \mid j \in J,\ 1 \leq c \leq C\}$. Correctness can be shown similar to case 2. $\square$

Then $\bigvee_{s \in S} \Phi(\mathbf{x}, s(\mathbf{x}))$ is the desired quantifier-free formula equivalent to $\exists z \Phi(\mathbf{x}, z)$. This concludes the proof of the quantifier elimination.

Quantifier elimination implies decidability of Presburger arithmetic since a quantifier-free variable-free formula φ can be easily evaluated.

Furthermore, it shows that Presburger definable sets are Boolean combinations of half-spaces (defined by linear (in)equalities) and divisibility constraints.
$\rightarrow$ Later: These are precisely the *semilinear sets*.

Existential Presburger arithmetic

Existential Presburger arithmetic is the restriction of PA to *existential formulas* of the form

$$\exists x_1 \ldots \exists x_d \varphi(x_1, \ldots, x_d) \quad \text{where } \varphi \text{ is quantifier-free.}$$

Theorem (von zur Gathen, Sieveking 1978)

Let $A \in \mathbb{Z}^{m \times n}$, $\mathbf{b} \in \mathbb{Z}^{m \times 1}$, $C \in \mathbb{Z}^{p \times n}$, $\mathbf{d} \in \mathbb{Z}^{p \times 1}$. Let M be the maximum absolute value of an entry in $A, \mathbf{b}, C, \mathbf{d}$. If $A\mathbf{x} = \mathbf{b}$ and $C\mathbf{x} \geq \mathbf{d}$ has a solution $\mathbf{x} \in \mathbb{Z}^{n \times 1}$ then there exists such a solution whose absolute values are bounded by $M^{n+1} 2^{O(n \log n)}$.

Theorem

Existential Presburger arithmetic is NP*-complete.*

Proof.

Given a formula $\varphi = \exists \mathbf{x} \psi(\mathbf{x})$ where ψ is a Boolean combination of linear equalities and inequalities over $\mathbf{x}$.

1. Bring ψ into negation normal form.
2. Replace $s \neq t$ by $(s < t) \vee (s > t)$ and $s \not< t$ by $(s = t) \vee (s > t)$.
3. Replace any disjunction $\psi_1 \vee \psi_2$ in ψ nondeterministically by ψ_1 or ψ_2.
4. Guess and verify a nonnegative solution $\mathbf{x}$ whose entries are bounded exponentially in the size of the formula.

Remark: NP membership even holds for binary encoded coefficients.

Proof.

For NP-hardness we reduce from the satisfiability problem of propositional formulas in CNF.

Given a propositional formula $\varphi = \bigwedge_{i=1}^{m} C_i$ where C_i is a disjunction of literals.

- Each propositional variable x_j becomes a variable $x_j \in \{0, 1\}$.
- Each C_i is turned into a linear inequality.

Example: $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_2 \vee x_3 \vee \bar{x}_4)$ is satisfiable iff the following existential Presburger formula holds:

$$\exists x_1 \exists x_2 \exists x_3 \exists x_4 \bigwedge_{i=1}^{4} (0 \leq x_i \leq 1) \wedge x_1 + (1 - x_2) + x_3 \geq 1$$
$$\wedge\, (1 - x_2) + x_3 + (1 - x_4) \geq 1.$$

□

Part V

Integer VASS

Section 18

Semilinear sets

Semilinear sets

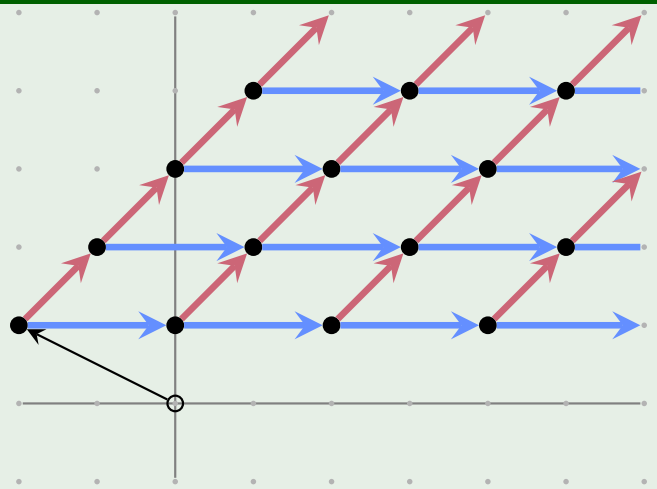
Definition

A set $L \subseteq \mathbb{Z}^d$ is *linear* if there is a *base vector* $\mathbf{b} \in \mathbb{Z}^d$ and a finite set of period vectors $P = \{\mathbf{p}_1, \ldots, \mathbf{p}_k\} \subseteq \mathbb{Z}^d$ such that

$$L = L(\mathbf{b}, P) := \{\mathbf{b} + \lambda_1 \mathbf{p}_1 + \cdots + \lambda_k \mathbf{p}_k \mid \lambda_1, \ldots, \lambda_k \in \mathbb{N}\}.$$

A set $S \subseteq \mathbb{Z}^d$ is *semilinear* if it is a finite union of linear sets.

Example



Semilinear sets

We represent linear sets and semilinear sets by a list of the defining base and period vectors (not unique).

Let $\mathbf{b} \in \mathbb{Z}^d$ and view $P = \{\mathbf{p}_1, \ldots, \mathbf{p}_k\} \subseteq \mathbb{Z}^d$ as a matrix in $\mathbb{Z}^{d \times k}$. Then we can write

$$L(\mathbf{b}, P) = \{\mathbf{b} + P\boldsymbol{\lambda} \mid \boldsymbol{\lambda} \in \mathbb{N}^{k \times 1}\}.$$

Theorem (Ginsburg, Spanier 1964)

A set $S \subseteq \mathbb{Z}^d$ is Presburger definable if and only if it is semilinear. Furthermore, the equivalence is effective.

That means: There is an algorithm that converts a Presburger formula into a semilinear representation, and vice versa.

Semilinear $\rightarrow$ Presburger

Proposition

Every semilinear set is effectively Presburger definable.

Proof.

Consider a linear set $L(\mathbf{b}, P)$ where $\mathbf{b} \in \mathbb{Z}^d$ and $P = \{\mathbf{p}_1, \ldots, \mathbf{p}_k\} \subseteq \mathbb{Z}^d$. Then $L(\mathbf{b}, P)$ is defined by the Presburger formula

$$\varphi(\mathbf{x}) = \exists(\lambda_1, \ldots, \lambda_k) \in \mathbb{N}^k : \mathbf{x} = \mathbf{b} + \sum_{i=1}^{k} \lambda_i \mathbf{p}_i$$

where $\mathbf{x} = \mathbf{b} + \sum_{i=1}^{k} \lambda_i \mathbf{p}_i$ is a conjunction of d linear equations. Hence all semilinear sets are Presburger definable. $\square$

Presburger $\rightarrow$ Semilinear

Proposition

Every Presburger definable set is effectively semilinear.

Here: proof for $(\mathbb{N}, +, <, 0, 1)$. Exercise: proof for $(\mathbb{Z}, +, <, 0, 1)$.

Presburger $\to$ Semilinear

Lemma

Let $A \in \mathbb{Z}^{k \times d}$, $\mathbf{c} \in \mathbb{Z}^k$. Then $S = \{\mathbf{x} \in \mathbb{N}^d \mid A\mathbf{x} = \mathbf{c}\}$ is semilinear.

Define $S_0 = \{\mathbf{x} \in \mathbb{N}^d \setminus \{\mathbf{0}\} \mid Ax = \mathbf{0}\}$. Let B be the set of minimal elements of S. Let P be the set of minimal elements of S_0. Then P and B are finite since $(\mathbb{N}^d, \leq)$ is a WQO.

Claim

$S_0 = L(\mathbf{0}, P)$.

"$\supseteq$": trivial. For "$\subseteq$", let $\mathbf{s} \in S_0$. Induction w.r.t. $(\mathbb{N}^d, \leq)$: Write $\mathbf{s} = \mathbf{s}_0 + \mathbf{x}$ for some $\mathbf{s}_0 \in P$. Then $A\mathbf{x} = A\mathbf{s}_0 - A\mathbf{s} = \mathbf{0}$, hence $\mathbf{x} \in S_0$. By induction, $\mathbf{x} \in L(\mathbf{0}, P)$, hence $\mathbf{s} = \mathbf{s}_0 + \mathbf{x} \in L(\mathbf{0}, P)$.

Claim

$S = \bigcup_{\mathbf{b} \in B} L(b, P)$.

"$\supseteq$": clear. For $\mathbf{s} \in S$, write $\mathbf{s} = \mathbf{b} + \mathbf{x}$ for some $\mathbf{b} \in B$. Then $A\mathbf{x} = A\mathbf{s} - A\mathbf{b} = \mathbf{0}$, thus $\mathbf{x} \in S_0 = L(0, P)$ and $\mathbf{s} = \mathbf{b} + \mathbf{x} \in L(\mathbf{b}, P)$.

Effectiveness

This proves the lemma, except:

How to compute P and B?

- Simple: Define P and B in Presburger arithmetic and compute automata for them.
- Efficient: The largest absolute value of vectors in P and B is at most $(d+1)(\|A\| + \|\mathbf{c}\| + 1)^k$ where $\|\cdot\|$ returns the maximum absolute value (Pottier 1991)

We are ready to prove:

Proposition

Every Presburger definable set is effectively semilinear.

Let $\varphi(\mathbf{x})$ be a PA formula. We may assume that $\varphi(\mathbf{x})$ is quantifier-free. Since semilinear sets are closed under finite union, we may assume that φ is a conjunction of atoms.
Observe that $\varphi(\mathbf{x})$ is equivalent to a formula of the form $\exists \mathbf{y} \colon A(\mathbf{x}, \mathbf{y}) = \mathbf{c}$ for some matrix $A \in \mathbb{Z}^{k \times d}$ and $\mathbf{c} \in \mathbb{Z}^k$:

- Replace $\langle \mathbf{u}, \mathbf{x} \rangle \geq a$ with $\exists y \colon \langle \mathbf{u}, \mathbf{x} \rangle - y = a$.
- Replace $a \mid \langle \mathbf{u}, \mathbf{x} \rangle$ with $\exists y, z \colon \langle \mathbf{u}, \mathbf{x} \rangle - ay + az = 0$.

The projection of a semilinear set is effectively semilinear.
Hence, by our lemma: The formula $\exists \mathbf{y} \colon A(\mathbf{x}, \mathbf{y}) = \mathbf{c}$ defines an effectively semilinear set. $\square$

Part V

Integer VASS

Section 19

Parikh's Theorem

Parikh images

We will introduce a useful *abstraction* of formal languages.
For $w \in \Sigma^*$ we define

- $|w|_a$: the number of occurrences of $a \in \Sigma$ in w.
- $\mathrm{alph}(w) = \{a \in \Sigma \mid |w|_a > 0\}$

For an alphabet $\Sigma = \{a_1, \ldots, a_k\}$ the *Parikh map* is defined as

$$\Psi \colon \Sigma^* \to \mathbb{N}^k, \ \Psi(w) = (|w|_{a_1}, \ldots, |w|_{a_k}).$$

The *Parikh image* of $L \subseteq \Sigma^*$ is $\Psi(L) = \{\Psi(w) \mid w \in L\}$.

Example

- $L = (aba)^* c^* \ \rightarrow \ \Psi(L) = \{(2i, i, j) \mid i, j \in \mathbb{N}\}$
- $K = \{a^n c^m (ab)^n \mid m, n \in \mathbb{N}\} \ \rightarrow \ \Psi(K) = \Psi(L)$

Parikh's theorem

Theorem (Parikh 1966)

The Parikh image of a context-free language is effectively semilinear.

We will show that the Parikh image of a context-free language is definable by an existential Presburger formula whose length is linear in the grammar size.

As a warm up we will start with a simpler statement:

Theorem

Given an NFA $\mathcal{A}$ one can compute in linear time an existential Presburger formula defining the Parikh image of $L(\mathcal{A})$.

Let $\mathcal{A} = (Q, \Sigma, E, q_{\text{in}}, F)$ be an NFA where $E \subseteq Q \times \Sigma \times Q$. For simplicity we assume that F contains a single final state q_f.

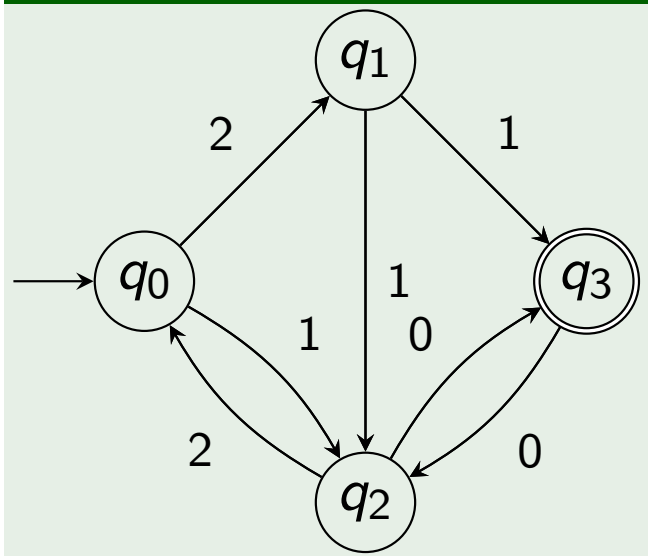
- We write $q \xrightarrow{\tau} q'$ if $\tau = (q, a, q')$ for some $a \in \Sigma$.
- A run π is a sequence $q_0 \xrightarrow{\tau_1} q_1 \xrightarrow{\tau_2} \dots \xrightarrow{\tau_n} q_n$.
- π is *accepting* if $q_0 = q_{\text{in}}$ and $q_n = q_f$.

Instead of letters we will (first) count transitions in runs:

Definition

The Parikh image of a run π is the vector $\mathbf{x} = (x_\tau)_{\tau \in E} \in \mathbb{N}^E$ where x_τ is the number of occurrences of τ in π.

Example



$q_0 \to q_1 \to q_2 \to q_0 \to q_2 \to q_0 \to$
$q_1 \to q_3$

Notice that the Parikh vector $\mathbf{x}$ satisfies a "flow condition":
In every state the number of times an incoming transition is used is equal to the number of times an outgoing transition is used (except for the initial and the final state).

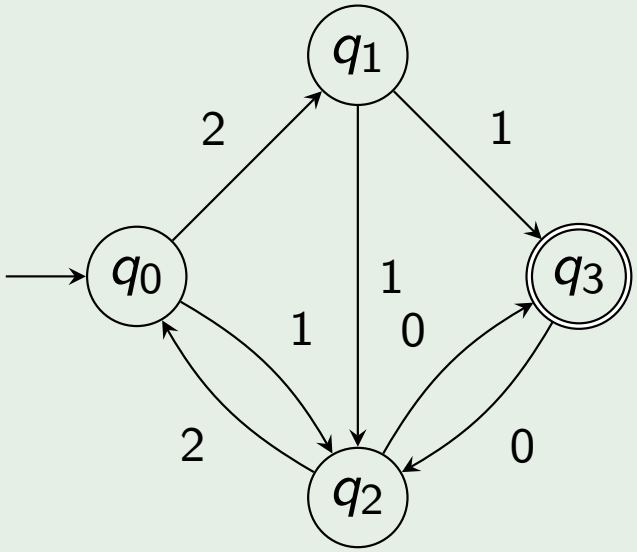
Notation:

- $\text{out}(q) = \{\tau \in E \mid \tau \text{ starts in } q\}$
- $\text{in}(q) = \{\tau \in E \mid \tau \text{ ends in } q\}$

Given a Parikh vector $\mathbf{x} \in \mathbb{N}^E$ we obtain an automaton/graph by removing from $\mathcal{A}$ all transitions and states which are not used in $\mathbf{x}$. Formally, we define the graph $N[\mathbf{x}] = (Q[\mathbf{x}], E[\mathbf{x}])$ where

- $E[\mathbf{x}] = \{\tau \in E \mid x_\tau > 0\}$
- $Q[\mathbf{x}] = \{q \in Q \mid q \text{ occurs in some } \tau \in E[\mathbf{x}]\}$

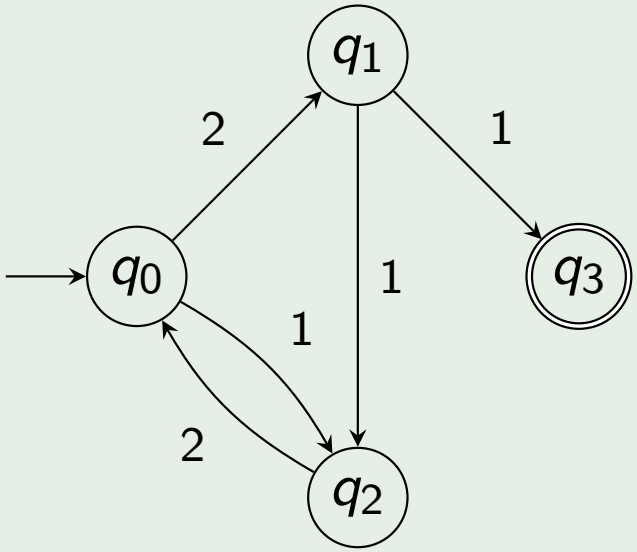
Example



Given a Parikh vector $\mathbf{x} \in \mathbb{N}^E$ we obtain an automaton/graph by removing from $\mathcal{A}$ all transitions and states which are not used in $\mathbf{x}$. Formally, we define the graph $N[\mathbf{x}] = (Q[\mathbf{x}], E[\mathbf{x}])$ where

- $E[\mathbf{x}] = \{\tau \in E \mid x_\tau > 0\}$
- $Q[\mathbf{x}] = \{q \in Q \mid q \text{ occurs in some } \tau \in E[\mathbf{x}]\}$

Example



Define $\delta_{a,b} = 1$ if $a = b$ and $\delta_{a,b} = 0$ otherwise.

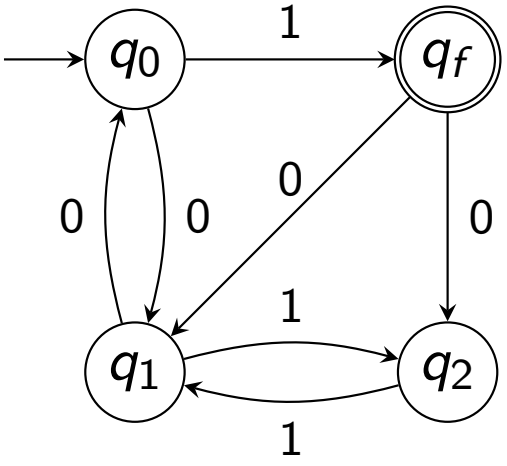
Proposition

There exists an accepting run with Parikh image $\mathbf{x} \in \mathbb{N}^E$ *in* $\mathcal{A}$ *if and only if for all* $q \in Q$ *we have*

$$\sum_{\tau \in \mathsf{out}(q)} x_\tau - \sum_{\tau \in \mathsf{in}(q)} x_\tau = \delta_{q,q_{\mathsf{in}}} - \delta_{q,q_f}$$

and all states in $N[\mathbf{x}]$ *are reachable from* q_{in}.

The reachability condition excludes Parikh vectors of the form:



Define $\delta_{a,b} = 1$ if $a = b$ and $\delta_{a,b} = 0$ otherwise.

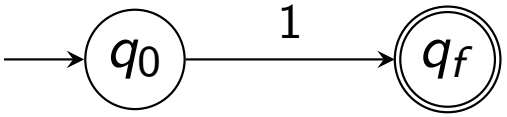
Proposition

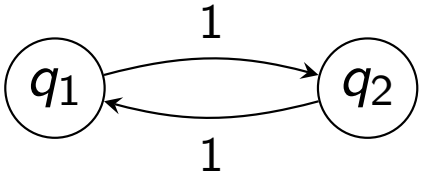
There exists an accepting run with Parikh image $\mathbf{x} \in \mathbb{N}^E$ *in* $\mathcal{A}$ *if and only if for all* $q \in Q$ *we have*

$$\sum_{\tau \in \mathsf{out}(q)} x_\tau - \sum_{\tau \in \mathsf{in}(q)} x_\tau = \delta_{q,q_{\mathsf{in}}} - \delta_{q,q_f}$$

and all states in $N[\mathbf{x}]$ *are reachable from* q_{in}.

The reachability condition excludes Parikh vectors of the form:





Proposition

There exists an accepting run with Parikh image $\mathbf{x} \in \mathbb{N}^E$ *in* $\mathcal{A}$ *if and only if for all* $q \in Q$ *we have*

$$\sum_{\tau \in \mathsf{out}(q)} x_\tau - \sum_{\tau \in \mathsf{in}(q)} x_\tau = \delta_{q,q_{\mathsf{in}}} - \delta_{q,q_f}$$

and all states in $N[\mathbf{x}]$ *are reachable from* q_{in}.

Proof.

$(\Rightarrow)$: Consider an accepting run $q_0 \to q_1 \to \cdots \to q_n$. If q occurs k times amongst $q_1, \ldots, q_{n-1}$ then

- q occurs k times as a source state of a transition plus another occurrence if $q = q_{\mathsf{in}}$,
- q occurs k times as a target state of a transition plus another occurrence if $q = q_f$.

Clearly, all states in $N[\mathbf{x}]$ are reachable from q_{in} via the run.

Proposition

There exists an accepting run with Parikh image $\mathbf{x} \in \mathbb{N}^E$ *in* $\mathcal{A}$ *if and only if for all* $q \in Q$ *we have*

$$\sum_{\tau \in \mathsf{out}(q)} x_\tau - \sum_{\tau \in \mathsf{in}(q)} x_\tau = \delta_{q,q_{\mathsf{in}}} - \delta_{q,q_f}$$

and all states in $N[\mathbf{x}]$ *are reachable from* q_{in}.

Proof.

($\Leftarrow$) : By induction on $\sum_{\tau \in E} x_\tau$. If $\mathbf{x} = \mathbf{0}$ then $q_{\mathsf{in}} = q_f$. The empty run is accepting and has Parikh image $\mathbf{0}$. Now assume $\mathbf{x} \neq \mathbf{0}$. We distinguish two cases:

- Assume that q_{in} is contained in a cycle C of $N[\mathbf{x}]$. If $\tau = (q_{\mathsf{in}}, a, p)$ lies on C then all states in $N[\mathbf{x} - \mathbf{e}_\tau]$ are reachable from p.
- Otherwise q_{in} has no incoming transition in $N[\mathbf{x}]$ and exactly one outgoing transition $\tau = (q_{\mathsf{in}}, a, p)$ in $N[\mathbf{x}]$. Again, all states in $N[\mathbf{x} - \mathbf{e}_\tau]$ are reachable from p.

Proof.

In any case we find a transition $\tau = (q_{\mathsf{in}}, a, p)$ such that all states in $N[\mathbf{x} - \mathbf{e}_\tau]$ are reachable from p. Setting $\mathbf{x}' = \mathbf{x} - \mathbf{e}_\tau$ we obtain:

$$\begin{aligned}\sum_{\tau \in \mathsf{out}(q)} x'_\tau - \sum_{\tau \in \mathsf{in}(q)} x'_\tau &= \sum_{\tau \in \mathsf{out}(q)} x_\tau - \sum_{\tau \in \mathsf{in}(q)} x_\tau - \delta_{q,q_{\mathsf{in}}} + \delta_{q,p} \\ &= \delta_{q,q_{\mathsf{in}}} - \delta_{q,q_f} - \delta_{q,q_{\mathsf{in}}} + \delta_{q,p} \\ &= \delta_{q,p} - \delta_{q,q_f}\end{aligned}$$

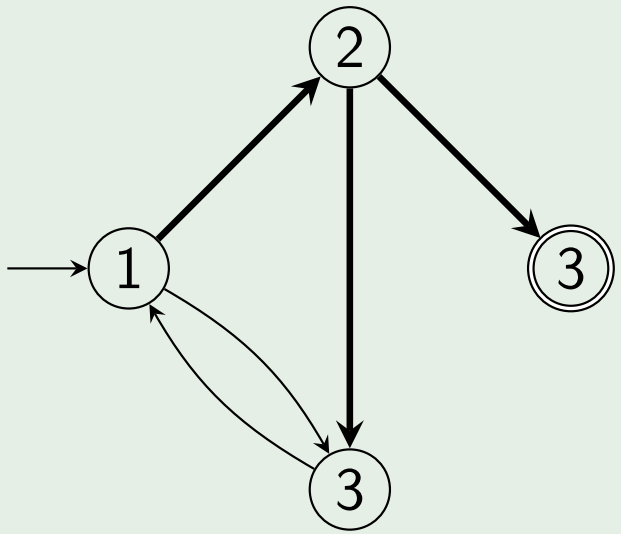
Therefore, by induction there is a run from p to q_f with Parikh image $\mathbf{x}'$. Adding the transition τ we obtain the desired run from q_{in} to q_f. □

Theorem

Given an NFA $\mathcal{A}$ one can compute in linear time an existential Presburger formula defining the Parikh image of $L(\mathcal{A})$.

To express the reachability condition we describe the existence of a spanning tree of $N[\mathbf{x}]$ rooted in q_{in}. To every state $q \in Q$ we assign a distance $y_q \in \mathbb{N}$ to q_{in}.

Example



Proof (Parikh's theorem for automata).

We use variables $\mathbf{x} = (x_\tau)_{\tau \in E}$, and $\mathbf{y} = (y_q)_{q \in Q}$, $\mathbf{z} = (z_a)_{a \in \Sigma}$. Then

$$\varphi(\mathbf{z}) := \exists \mathbf{x} \exists \mathbf{y} \colon \bigwedge_{q \in Q} (\alpha_q \wedge \beta_q) \wedge \bigwedge_{a \in \Sigma} \gamma_a.$$

defines the Parikh image of $L(\mathcal{A})$ where the subformulas are

$$\alpha_q := \sum_{\tau \in \mathrm{out}(q)} x_\tau - \sum_{\tau \in \mathrm{in}(q)} x_\tau = \delta_{q,q_{\mathrm{in}}} - \delta_{q,q_f}$$

$$\beta_{q_{\mathrm{in}}} := y_{q_{\mathrm{in}}} = 1$$

$$\beta_q := \bigvee_{(p,a,q) \in \mathrm{in}(q)} (x_{(p,a,q)} > 0 \wedge y_p > 0 \wedge y_q = y_p + 1)$$

$$\vee \sum_{\tau \in \mathrm{in}(q)} x_\tau = 0 \rightarrow y_q = 0$$

$$\gamma_a := z_a = \sum_{\tau \in E \text{ reads } a} x_\tau.$$

□

Parikh's theorem

Next we show the efficient version of Parikh's theorem for context-free grammars:

Theorem (Verma, Seidl, Schwentick 2005, Esparza 1997)

Given a context-free grammar G one can compute in linear time an existential Presburger formula defining the Parikh image of $L(G)$.

We follow a presentation by Kufleitner (2022).

Recap

Let $G = (V, \Sigma, P, S)$ be a context-free grammar where

- V is the finite set of *variables* or *nonterminals*,
- Σ is the finite terminal alphabet,
- $P \subseteq V \times (V \cup \Sigma)^*$ is a finite set of *productions*,
- $S \in V$ is the *start variable*.

The language of G is denoted $L(G) \subseteq \Sigma^*$.

The formula for $\Psi(L(G))$ will use the following variables:

- x_p for each $p \in P$,
- y_X for each $X \in V$,
- z_a for each $a \in \Sigma$.

The formula

For $X \in V$ and $p = (Y \to u) \in P$ we set $X(p) = 1$ if $X = Y$ and $X(p) = 0$, otherwise.
Furthermore, we write $|p|_X = |u|_X$.

Flow Condition

For each $X \in V \setminus \{S\}$ we define

$$\alpha_X := \sum_{p \in P} X(p) \cdot x_p = \sum_{p \in P} |p|_X \cdot x_p$$

and

$$\alpha_S := \sum_{p \in P} S(p) \cdot x_p = 1 + \sum_{p \in P} |p|_S \cdot x_p.$$

The formula says that a variable X is produced as often as it is consumed (exception: S).

The formula

Reachability Condition

We define

$$\beta_S := y_S = 1$$

and for each $X \in V \setminus \{S\}$ we define

$$\beta_X := \left(y_X = 0 \wedge \sum_{p \in P} |p|_X \cdot x_p = 0 \right) \vee \bigvee_{\substack{(Y \to u) \in P \\ |u|_X > 0}} \left(x_{Y \to u} > 0 \wedge y_Y > 0 \wedge y_X = y_Y + 1 \right).$$

We number the nonterminals X occurring in the derivation tree by an index y_X. If X does not occur then $y_X = 0$.

The formula

Counting Letters

For each $a \in \Sigma$ we define

$$\gamma_a := \left(z_a = \sum_{p \in P} |p|_a \cdot x_p \right).$$

The formula γ_s counts how often the letter a is produced in productions.

The final formula for the Parikh image is:

$$\varphi(\mathbf{z}) := \exists \mathbf{x} \exists \mathbf{y} \colon \bigwedge_{X \in V} (\alpha_X \wedge \beta_X) \wedge \bigwedge_{a \in \Sigma} \gamma_a.$$

Correctness of the formula

We claim that φ defines $\Psi(L(G))$.

Suppose that $w \in L(G)$, i.e. there exists a derivation tree T for w. Define x_p the number of times production p is applied in T and define $z_a = |w|_a$. Clearly, $(\mathbf{x}, \mathbf{z})$ satisfies $\bigwedge_{X \in V} \alpha_X \wedge \bigwedge_{a \in \Sigma} \gamma_a$.

The indices y_X are defined inductively.

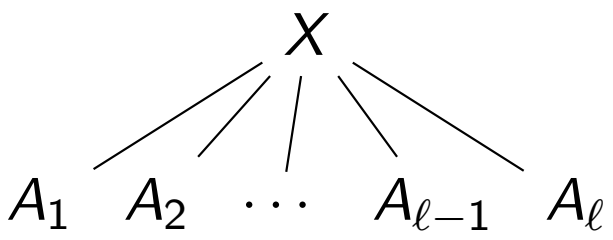
- Set $y_S = 1$.
- If y_X is undefined but y_Y is defined for some parent Y of X in T, set $y_X = y_Y + 1$.

Then y_X is defined if and only if X occurs in T (why?). Furthermore, an index y_X is defined because T uses a production $Y \to u$ with $|u|_X > 0$ and y_Y is already defined. Hence $(\mathbf{x}, \mathbf{y})$ satisfies $\bigwedge_{X \in V} \beta_X$.

Correctness of the formula

For the converse, suppose that $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ satisfies $\bigwedge_{X \in V}(\alpha_X \wedge \beta_X) \wedge \bigwedge_{a \in \Sigma} \gamma_a$.

Represent a production $p = (X \to A_1 \cdots A_\ell)$ by a tree:



Consider the following forest containing x_p copies of the production p for each $p \in P$.
The formula α_X with $X \neq S$ states that the number of X-roots is equal to the number of X-leaves.
The formula α_S states that the number of S-roots is equal to the number of S-leaves, plus one.

Correctness of the formula

Repeat the following process as long as possible:

- Take a tree t_1 with a X-leaf and another tree t_2 with a X-root.
- Replace t_1 and t_2 by a new tree obtained from t_1 where a X-leaf is replaced by t_2.

Observe that this process does not change the balance between X-leaves and X-roots.
Also, the number of a-leaves ($a \in \Sigma$) remains unchanged.

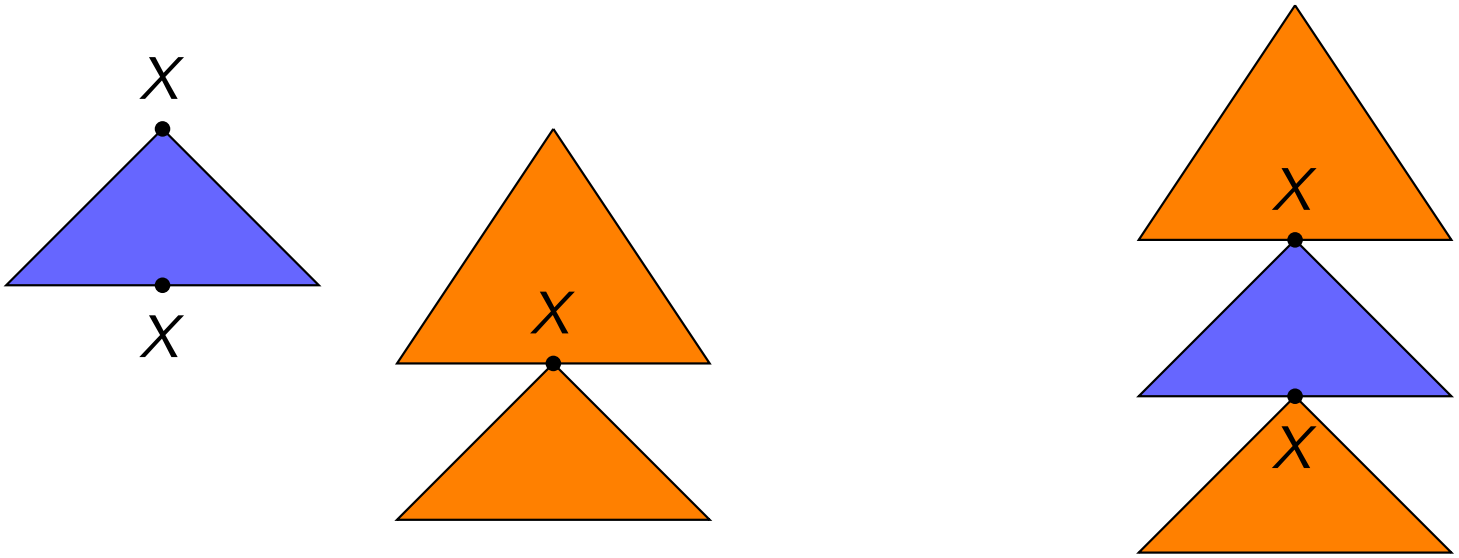
The resulting forest has the following properties:

- For each X there exists at most one tree T_X with root X.
- The leaves of T_S are terminals.
- The leaves of T_X ($X \neq S$) are terminals except for one X-leaf.

We will further reduce the forest until a single (derivation) tree remains.

Correctness of the formula

Consider a tree T with root X and leaf X. If there exists another tree T' containing an X-node, we can insert T into T'.



$\rightarrow$ This reduces the number of trees by one.

Correctness of the formula

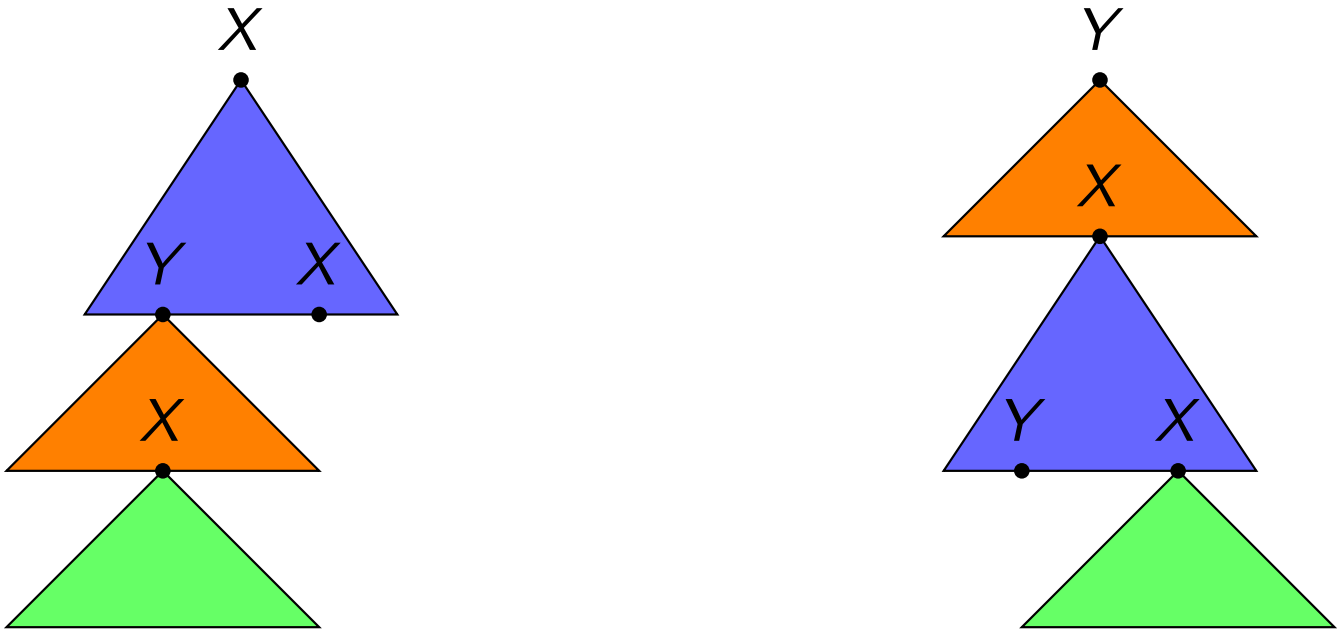
Otherwise, all occurrences of X are in T. By the formula β_X, there exists a production $Y \to u$ occurring in T with $|u|_X > 0$ and $y_X = y_Y + 1$.

If this occurrence of Y lies on the path from the root X to the leaf X, we can rearrange the tree as follows.

$\to$ The index of the root has decreased by one!

Correctness of the formula

Otherwise, we can rearrange the tree as follows:



$\rightarrow$ The index of the root has decreased by one!

Eventually, we obtain a derivation tree for a word w with $|w|_a = z_a$ for all $a \in \Sigma$. $\square$

Corollary

Downwards closures of context-free languages are computable.

Proof.

Recall that downwards closures are computable for effective full trios if and only if the *simultaneous unboundedness problem* is decidable.

The context-free languages form an effective full trio (product construction of a PDA with the rational transducer).

Given context-free grammar G for $L(G) \subseteq a_1^* a_2^* \cdots a_n^*$. We can compute a Presburger formula $\varphi(x_1, \ldots, x_n)$ for its Parikh image. Then $L(G)$ is simultaneously unbounded if and only if

$$\forall k \exists \mathbf{x} \colon \bigwedge_{i=1}^{n} x_i \geq k \wedge \varphi(\mathbf{x}). \qquad \square$$

Part V

Integer VASS

Section 20

Reachability in $\mathbb{Z}$-VASS

Integer VASS

We proved: The Parikh image of a context-free language is definable by an existential Presburger formula of linear size in the grammar.
Together with the fact that existential Presburger arithmetic is in NP we can solve the reachability over various infinite state systems in NP.

Definition

A *d-dimensional integer VASS (or $\mathbb{Z}$-VASS)* $V = (Q, T, q_0)$ is syntactically the same as a d-dimensional VASS.
Configurations in V are pairs $(q, \mathbf{u}) \in Q \times \mathbb{Z}^d$, and we define $(q, \mathbf{u}) \to (q', \mathbf{u}')$ if there exists $(q, \mathbf{v}, q') \in T$ such that $\mathbf{u}' = \mathbf{u} + \mathbf{v}$.

Theorem

Given a d-dimensional $\mathbb{Z}$-VASS $V = (Q, T, q_0)$ and a state $q_f \in Q$, one can compute in linear time an existential Presburger formula defining $\{\mathbf{u} \in \mathbb{Z}^d \mid (q_0, \mathbf{0}) \xrightarrow{*} (q_f, \mathbf{u})\}$.

Proof.

Let $T = \{\tau_1, \ldots, \tau_k\}$ be the set of transitions where τ_i has effect $\mathbf{v}_i \in \mathbb{Z}^d$. We view V as an NFA $\mathcal{A} = (Q, \{\mathbf{v}_1, \ldots, \mathbf{v}_k\}, T, q_0, \{q_f\})$. In linear time we construct an existential Presburger formula $\varphi_{\mathcal{A}}(x_1, \ldots, x_k)$ defining the Parikh image of $L(\mathcal{A})$.
Then

$$\varphi(\mathbf{y}) = \exists \mathbf{x}(\varphi_{\mathcal{A}}(\mathbf{x}) \wedge \mathbf{y} = \sum_{i=1}^{k} x_i \mathbf{v}_i)$$

defines the set of reachable configurations $(q_f, \mathbf{u})$. $\square$

Since existential Presburger arithmetic is in NP we obtain:

Corollary

The reachability problem for $\mathbb{Z}$-VASS is in NP*.*

What happens if we add a stack to the $\mathbb{Z}$-VASS?

Pushdown systems

Let Γ be a finite stack alphabet and let $\bar{\Gamma} = \{\bar{a} \mid a \in \Gamma\}$.
The symbols $a \in \Gamma$ are *push* symbols and $\bar{a} \in \bar{\Gamma}$ are *pop* symbols.

Definition

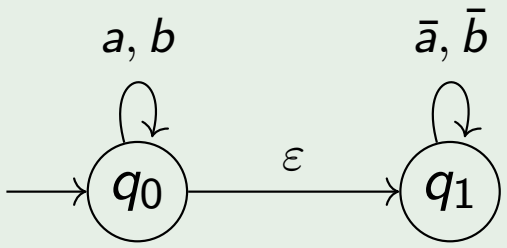
A *pushdown system* $P = (Q, \Gamma, T, q_0)$ consists of

- a finite set of states Q,
- a finite stack alphabet Γ,
- an initial state $q_0 \in Q$,
- a set of transitions $T \subseteq Q \times (\Gamma \cup \bar{\Gamma} \cup \{\varepsilon\}) \times Q$.

A *configuration* in P has the form $(q, s) \in Q \times \Gamma^*$.
We have $(q, s) \xrightarrow{\tau}_P (q', s')$ if $\tau = (q, \alpha, q') \in T$ and:

- if $\alpha = a \in \Gamma$ then $as = s'$,
- if $\alpha = \bar{a} \in \bar{\Gamma}$ then $s = as'$,
- if $\alpha = \varepsilon$ then $s = s'$.

Example



$$(q_0, \varepsilon) \to (q_0, a) \to (q_0, ba)$$
$$\to (q_1, ba) \to (q_1, a) \to (q_1, \varepsilon)$$

Comparison to pushdown automata

- Transitions have no labels
- Alternative definition: Pushdown contains an initial stack symbol #. Every transition pops a symbol and pushes (possibly zero) symbols.

$\to$ If we add transition labels, our model of pushdown systems/automata captures the context-free languages.

Definition

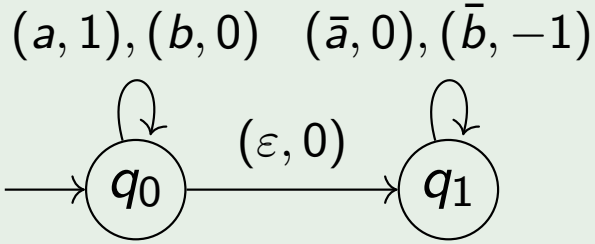
A *d-dim. $\mathbb{Z}$-VASS with pushdown* $V = (Q, \Gamma, T, q_0)$ consists of

- a finite set of states Q,
- a finite stack alphabet Γ,
- an initial state $q_0 \in Q$,
- a finite set of transitions $T \subseteq Q \times (\Gamma \cup \bar{\Gamma} \cup \{\varepsilon\}) \times \mathbb{Z}^d \times Q$.

A *configuration* in V has the form $(q, s, \mathbf{u}) \in Q \times \Gamma^* \times \mathbb{Z}^d$.
We have $(q, s, \mathbf{u}) \xrightarrow{\tau} (q', s', \mathbf{u}')$ if $\tau = (q, \alpha, \mathbf{v}, q') \in T$ and:

- $\mathbf{u}' = \mathbf{u} + \mathbf{v}$,
- if $\alpha = a \in \Gamma$ then $as = s'$,
- if $\alpha = \bar{a} \in \bar{\Gamma}$ then $s = as'$,
- if $\alpha = \varepsilon$ then $s = s'$.

Example



$$(q_0,\varepsilon,0) \to (q_0,a,1) \to (q_0,ba,1)$$
$$\to (q_1,ba,1) \to (q_1,a,0) \to (q_1,\varepsilon,0)$$

Theorem

Given a d-dim. $\mathbb{Z}$-VASS $V = (Q, \Gamma, T, q_0)$ with pushdown and a state $q_f \in Q$, one can compute in polynomial time an existential Presburger formula defining $\{\mathbf{u} \in \mathbb{Z}^d \mid (q_0, \varepsilon, \mathbf{0}) \xrightarrow{*} (q_f, \varepsilon, \mathbf{u})\}$.

Proof.

Let $T = \{\tau_1, \ldots, \tau_k\}$. We construct a context-free grammar G over T with nonterminals $A_{p,q}$ for $p, q \in Q$ such that
$A_{p,q} \Rightarrow_G^* \sigma_1 \ldots \sigma_n \iff$ there is a run $(p, \varepsilon, \mathbf{0}) \xrightarrow{\sigma_1} \ldots \xrightarrow{\sigma_n} (q, \varepsilon, \mathbf{u})$.

The productions are

- $A_{p,q} \to A_{p,r} A_{r,q}$ for all $p, q, r \in Q$
- $A_{q,q} \to \varepsilon$ for all $q \in Q$,
- $A_{p,q} \to \tau$ for all $\tau = (p, \varepsilon, \mathbf{v}, q) \in T$,
- $A_{p,q} \to \tau_a A_{p',q'} \tau_{\bar{a}}$ for all $\tau_a = (p, a, \mathbf{u}, p')$, $\tau_{\bar{a}} = (q', \bar{a}, \mathbf{v}, q) \in T$.

Proof.

Declare A_{q_0,q_f} as the start nonterminal of G. Let $\varphi_G(x_1, \ldots, x_k)$ be an existential Presburger formula defining the Parikh image of $L(G)$.
Let $\mathbf{v}_i \in \mathbb{Z}^d$ be the effect of transition τ_i. Then

$$\varphi(\mathbf{y}) = \exists \mathbf{x}(\varphi_G(\mathbf{x}) \wedge \mathbf{y} = \sum_{i=1}^{k} x_i \mathbf{v}_i) \qquad \square$$

Corollary

The reachability problem for $\mathbb{Z}$-VASS with pushdown is in NP.

Proof.

If the given target configuration $(q_f, s, \mathbf{u})$ does not have an empty stack add new states/transitions which pop s from the stack and reach a state q'_f.
Then $(q_f, s, \mathbf{u})$ is reachable iff $(q'_f, \varepsilon, \mathbf{u})$ is reachable. $\square$

Part V

Integer VASS

Section 21

Reversal bounded counter machines

Reversal bounded counter machines

Let $\mathbf{c} = (c_0, \ldots, c_n)$ be a sequence of natural numbers. We say that a pair (i, j) with $i + 1 < j$ is a *reversal* in $\mathbf{c}$ if $c_i < c_{i+1} = \cdots = c_{j-1} > c_j$ or $c_i > c_{i+1} = \cdots = c_{j-1} < c_j$.

If $\mathbf{c}$ has r reversals then it can be divided into $r + 1$ *phases*, i.e. there exist $0 = i_0 < i_1 < \cdots < i_{r+1} = n$ such that $(c_{i_j}, \ldots, c_{i_{j+1}})$ is ascending or descending for all $0 \leq j \leq r$.

A sequence of vectors

$$(u_{1,0}, \ldots, u_{d,0}), (u_{1,1}, \ldots, u_{d,1}), \ldots, (u_{1,n}, \ldots, u_{d,n})$$

in $\mathbb{N}^d$ is *r-reversal-bounded* if for every $1 \leq i \leq d$ the number of reversals in $(u_{i,0}, \ldots, u_{i,n})$ is bounded by r.

Reversal bounded counter machines

We consider counter machines with explicit *nonzero-tests*.

Definition

A *d-counter machine* $M = (Q, T, q_0)$ consists of

- a finite set of states Q,
- a finite set $T \subseteq Q \times (\mathbb{Z}^d \cup \{\mathsf{z}_1, \mathsf{nz}_1, \ldots \mathsf{z}_d, \mathsf{nz}_d\}) \times Q$ of transitions
- an initial state $q_0 \in Q$.

A configuration is a pair $(q, \mathbf{u}) \in Q \times \mathbb{N}^d$. Consider configurations $(q, \mathbf{u}), (q', \mathbf{u}')$ where $\mathbf{u} = (u_1, \ldots, u_d)$ and $\mathbf{u}' = (u'_1, \ldots, u'_d)$.
For $\tau \in T$ we define $(q, \mathbf{u}) \xrightarrow{\tau} (q', \mathbf{u}')$ if

- $\tau = (q, \mathbf{v}, q')$ for some $\mathbf{v} \in \mathbb{Z}^d$ and $\mathbf{u}' = \mathbf{u} + \mathbf{v}$,
- $\tau = (q, \mathsf{z}_i, q')$, $u_i = 0$ and $\mathbf{u} = \mathbf{u}$,
- $\tau = (q, \mathsf{nz}_i, q')$, $u_i \neq 0$ and $\mathbf{u} = \mathbf{u}$.

Reversal bounded counter machines

Definition

An *r-reversal-bounded d-dimensional counter machine* or short (d, r)-*RBCM* is a d-counter machine $M = (Q, T, q_0)$ such that for every run $(q_0, \mathbf{0}) \to (q_1, \mathbf{u}_1) \to \cdots \to (q_n, \mathbf{u}_n)$ the sequence $(\mathbf{0}, \mathbf{u}_1, \ldots, \mathbf{u}_n)$ is r-reversal-bounded.

Theorem

The reachability problem for RBCM is in NP.

Assumptions

- The reversal bound of the RBCM is given and unary encoded.
- The transitions and target configuration are unary encoded.
- The target configuration is $(q_f, \mathbf{0})$.
- All counter updates are from $\{\mathbf{e}_1, -\mathbf{e}_1, \ldots, \mathbf{e}_d, -\mathbf{e}_d\}$.
  We call such RBCM *normalized*.

Lemma

Reachability for normalized (d, r)-RBCM is polynomial-time reducible to reachability for normalized $(d', 1)$-RBCM where $d' = d(r + 1 + \lceil \frac{r+1}{2} \rceil)$.

Proof.

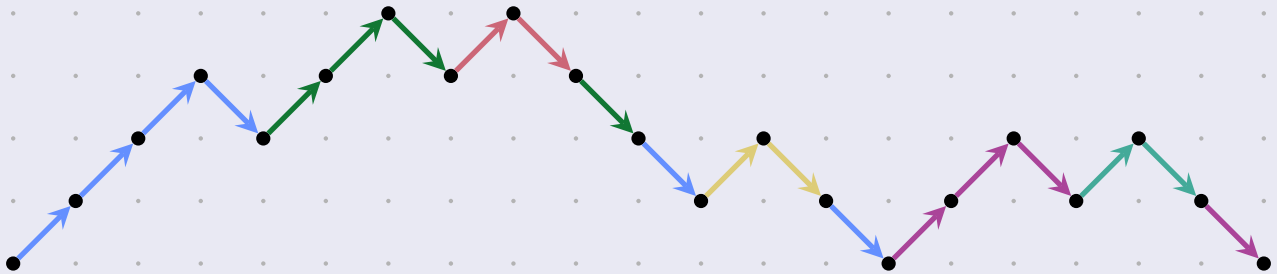
First, we assume a single counter, i.e. $d = 1$.

Use $r + 1$ many 1-RB counters $p_1, \ldots, p_{r+1}$ to track the phase: If the counter is in phase k then $p_k = 1$ and $p_j = 0$ for all $j \neq k$.

- Initialize $p_1 = 1$ and $p_i = 0$ for all $i > 1$.
- On every increment or decrement:
  1. Guess a number $1 \leq k \leq r + 1$ and test $p_k \neq 0$.
  2. If k is odd and the counter was decremented, or k is even and the counter was incremented, decrement p_k and increment p_{k+1}.

Proof.

From an r-RB counter c to $m := \lceil \frac{r+1}{2} \rceil$ many 1-RB counters $c_1, \ldots, c_m$ whose sum is c:



Guess the current phase $1 \leq k \leq r+1$ and test $p_k \neq 0$.

- Increment: Increment $c_{\lceil k/2 \rceil}$.
- Decrement: Guess $1 \leq i \leq m$, check $c_j = 0$ for $j > i$ and decrement c_i.
- Zero test: Test all counters $c_1, \ldots, c_m$ for zero.
- Nonzero test: Guess $1 \leq i \leq m$ and test c_i for nonzero.

Proof.

Given a $(1, r)$-RBCM $M = (Q, T, q_0)$ and a target configuration $(q_f, 0)$, we can compute in polynomial time an $(r + 1 + m, 1)$-RBCM $M' = (Q', T', q_0')$ such that:

$$(q_f, 0) \text{ is reachable in } M \iff (q_f, \mathbf{0}) \text{ is reachable in } M'$$

For $d > 1$ apply the construction to all counters. This establishes the reduction. $\square$

In $(d, 1)$-RBCM

- zero tests only occur at the "start" and at the "end" of a run,
- nonzero tests in the "middle".

$\rightarrow$ Simulate $(d, 1)$-RBCM by $\mathbb{Z}$-VASS.

Lemma

The reachability problem for normalized $(d, 1)$*-RBCM is in* NP*.*

Proof.

Let $M = (Q, T, q_0)$ be a $(d, 1)$-RBCM.
We maintain two sets $S, E \subseteq \{1, \ldots, d\}$:

- S: counters which are still zero and will be incremented.
- E: counters which have reached the final value of zero.

The $\mathbb{Z}$-VASS V simulates M as follows:

- If a counter is incremented it is removed from S.
- If a counter is decremented it may be added to E.
- Counters in E may not be incremented or decremented.

Proof.

Formally V has states $(q, S, E) \in Q \times 2^{\{1,\dots,d\}} \times 2^{\{1,\dots,d\}}$ and the following transitions

- $(q, S, E) \xrightarrow{\mathbf{e}_i} (q', S \setminus \{i\}, E)$ for all $q \xrightarrow{\mathbf{e}_i} q'$ if $i \notin E$
- $(q, S, E) \xrightarrow{-\mathbf{e}_i} (q', S, E)$
  $(q, S, E) \xrightarrow{-\mathbf{e}_i} (q', S, E \cup \{i\})$ for all $q \xrightarrow{-\mathbf{e}_i} q'$ if $i \notin E$
- $(q, S, E) \xrightarrow{\mathbf{0}} (q', S, E)$ for all $q \xrightarrow{\mathbf{0}} q'$
- $(q, S, E) \xrightarrow{\mathbf{0}} (q', S, E)$ for all $q \xrightarrow{\mathsf{z}_i} q'$ if $i \in S \cup E$
- $(q, S, E) \xrightarrow{\mathbf{0}} (q', S, E)$ for all $q \xrightarrow{\mathsf{nz}_i} q'$ if $i \notin S \cup E$

Proof.

We claim that:

$$(q_0, \mathbf{0}) \xrightarrow{*}_M (q_f, \mathbf{0})$$
$$\Longleftrightarrow$$
$$((q_0, S, E), \mathbf{0}) \xrightarrow{*}_V ((q_f, S', \{1, \ldots, d\}), \mathbf{0}) \text{ for some } S, S', E.$$

($\Rightarrow$): Consider a run $(q_0, \mathbf{u}_0) \xrightarrow{\tau_1}_M \ldots \xrightarrow{\tau_n}_M (q_n, \mathbf{u}_n)$ in M with $\mathbf{u}_0 = \mathbf{u}_n = \mathbf{0}$. Define

- $S_k = \{i \mid \tau_j \neq \mathbf{e}_i \text{ for all } j \leq k \text{ and } \tau_j = \mathbf{e}_i \text{ for some } j > k\}$
- $E_k = \{i \mid \tau_j \notin \{-\mathbf{e}_i, \mathbf{e}_i\} \text{ for all } j > k\}$

Then $((q_0, S_0, E_0), \mathbf{u}_0) \rightarrow_V \cdots \rightarrow_V ((q_n, S_n, E_n), \mathbf{u}_n)$ is a run.

Proof.

($\Leftarrow$): Let $((q_0, S_0, E_0), \mathbf{u}_0) \xrightarrow{\tau_1}_V \ldots \xrightarrow{\tau_n}_V ((q_n, S_n, E_n), \mathbf{u}_n)$ be a run with $\mathbf{u}_0 = \mathbf{u}_n = \mathbf{0}$ and $E_n = \{1, \ldots, d\}$.
Then $(q_0, \mathbf{u}_0) \rightarrow_M \cdots \rightarrow_M (q_n, \mathbf{u}_n)$:

- Increments and decrements can be simulated in M.
- Suppose that τ_k is a $\mathbf{0}$-transition which originates from a zero-test $(q_{k-1}, \mathrm{z}_i, q_k) \in T$. Then $\mathbf{u}_{k-1} = \mathbf{u}_k$ and $i \in S_{k-1} \cup E_{k-1}$, i.e. at position k counter i was not incremented yet or is not changed again. This implies $\mathbf{u}_{k-1}(i) = 0$.
- Suppose that τ_k is a $\mathbf{0}$-transition which originates from a nonzero-test $(q_{k-1}, \mathrm{nz}_i, q_k) \in T$. Then $\mathbf{u}_{k-1} = \mathbf{u}_k$ and $i \notin S_{k-1} \cup E_{k-1}$, i.e. at position k counter i was incremented and will be decremented again. This implies $\mathbf{u}_{k-1}(i) > 0$.

This establishes the claim.

Proof.

Problem: The $\mathbb{Z}$-VASS has size $|Q| \cdot 2^{2d}$.

However, in every run of V the S-component is decreasing and the E-component is increasing.

- Guess a descending chain $\mathcal{S} \subseteq 2^{\{1,\ldots,d\}}$ and an ascending chain $\mathcal{E} \subseteq 2^{\{1,\ldots,d\}}$ such that $\{1,\ldots,d\} \in \mathcal{E}$.
- Observe that $|\mathcal{S}|, |\mathcal{E}| \leq d+1$.
- Construct the restriction $V_{\mathcal{S},\mathcal{E}}$ of V to $Q \times \mathcal{S} \times \mathcal{E}$ in polynomial-time.
- Guess $S_0, S_f \in \mathcal{S}$ and $E_0 \in \mathcal{E}$ and test whether
$$((q_0, S_0, E_0), \mathbf{0}) \xrightarrow{*} ((q_f, S_f, \{1,\ldots,d\}), \mathbf{0})$$
in $V_{\mathcal{S},\mathcal{E}}$.

Therefore the reachability problem for $(d,1)$-RBCM is in NP. $\square$

Part V

Integer VASS

Section 22

Recurrent state reachability in VASS

Recurrent state reachability in VASS

Recall the *recurrent state reachability problem* for a class $\mathcal{C}$ of systems with a finite-state component.

Given A system $\mathcal{S}$ from $\mathcal{C}$ and a state q of $\mathcal{S}$.

Question Is there an infinite run in $\mathcal{S}$ that visits q infinitely often?

In this section we show that the recurrent state reachability in VASS is decidable, by viewing the Karp-Miller graph as a $\mathbb{Z}$-VASS.

Recall that recurrent state reachability is undecidable for the class of reset VASS and lossy channel systems.

Recurrent state reachability in VASS

Let $V = (Q, T, q_0)$ be a d-VASS. A run

$$(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \cdots \to (q_n, \mathbf{u}_n)$$

is *self-covering on* $q \in Q$ if there exists $i < n$ such that $q_i = q_n = q$ and $\mathbf{u}_i \leq \mathbf{u}_n$.

Lemma

There is a run in V that visits $q \in Q$ infinitely often if and only if V has a self-covering run on q.

Proof.

($\Rightarrow$): If a run $(q_0, \mathbf{u}_0) \to (q_1, \mathbf{u}_1) \to \ldots$ visits q infinitely often then there exist $i < j$ such that $q_i = q_j = q$ and $\mathbf{u}_i \leq \mathbf{u}_j$.

($\Leftarrow$): Suppose that $(q_0, \mathbf{0}) \xrightarrow{*} (q, \mathbf{u}) \xrightarrow{+} (q, \mathbf{u} + \mathbf{v})$ for some $\mathbf{u}, \mathbf{v} \in \mathbb{N}^d$, where $\xrightarrow{+}$ is the transitive closure of $\to$. By compatibility of $\leq$ we can construct the run

$$(q_0, \mathbf{0}) \xrightarrow{*} (q, \mathbf{u}) \xrightarrow{+} (q, \mathbf{u} + \mathbf{v}) \xrightarrow{+} (q, \mathbf{u} + 2\mathbf{v}) \xrightarrow{+} \ldots$$

that visits q infinitely often. $\square$

Recurrent state reachability in VASS

Recall the *Karp-Miller graph* of V:

- Every node v is labelled by $\gamma_v = (q, \mathbf{u}) \in Q \times \mathbb{N}_\omega^d$.
- Every node v has an outgoing edge $v \xrightarrow{\tau} v'$ for every transition $\tau \in T$ which is activated in γ_v.
  In that case we have $\gamma_{v'} \geq \gamma$ where $\gamma_v \xrightarrow{\tau} \gamma$.

Furthermore, for every node labelled by $(q, \mathbf{u}) \in Q \times \mathbb{N}_\omega^d$ in the Karp-Miller graph and every $k \in \mathbb{N}$ there exists a reachable configuration $(q, \mathbf{u}') \in Q \times \mathbb{N}^d$ with $\mathbf{u}' \geq \mathbf{u}^{(k)}$.
($\mathbf{u}^{(k)}$ is obtained from $\mathbf{u}$ by replacing every ω-component by k.)

A path $v_0 \xrightarrow{\tau_1} v_1 \xrightarrow{\tau_2} \ldots \xrightarrow{\tau_n} v_n$ is a *cycle* if $n \geq 1$ and $v_0 = v_n$.
It is a q-cycle if $\gamma_{v_i} = (q, \mathbf{u})$ for some $0 \leq i \leq n$ and some $\mathbf{u}$.
Its *effect* is $\sum_{i=1}^{n} \mathbf{v}_i$ where $\tau_i = (q_{i-1}, \mathbf{v}_i, q_i)$ for all $1 \leq i \leq n$.

Proposition

There is a self-covering run on $q \in Q$ if and only if the Karp-Miller graph of V contains a q-cycle with nonnegative effect.

Recurrent state reachability in VASS

Lemma

If there is a self-covering run on $q \in Q$ then the Karp-Miller graph of V contains a q-cycle with nonnegative effect.

Proof.

Since V has a self-covering run on q there exists a run of the form

$$(q_0, \mathbf{0}) \xrightarrow{*} (q, \mathbf{u}) \xrightarrow{+} (q, \mathbf{u} + \mathbf{v}) \xrightarrow{+} (q, \mathbf{u} + 2\mathbf{v}) \xrightarrow{+} \dots$$

in V for some $\mathbf{v} \in \mathbb{N}^d$. Then there exists a path

$$(q_0, \mathbf{0}) \xrightarrow{*} (q, \mathbf{u}_0) \xrightarrow{+} (q, \mathbf{u}_1) \xrightarrow{+} (q, \mathbf{u}_2) \xrightarrow{+} \dots$$

in the Karp-Miller graph such that $\mathbf{u} + k\mathbf{v} \leq \mathbf{u}_k$ for all $k \in \mathbb{N}$. Since the Karp-Miller graph is finite there exists $i < j$ such that $\mathbf{u}_i = \mathbf{u}_j$. Then $(q, \mathbf{u}_i) \xrightarrow{*} (q, \mathbf{u}_j)$ is the desired q-cycle with effect $(j - i)\mathbf{v} \in \mathbb{N}^d$. $\square$

Recurrent state reachability in VASS

Lemma

If the Karp-Miller graph of V contains a q-cycle with nonnegative effect then there is a self-covering run on $q \in Q$.

Proof.

Consider a cycle $(q, \mathbf{u}) \xrightarrow{\tau_1} \ldots \xrightarrow{\tau_n} (q, \mathbf{u})$ where τ_i has effect $\mathbf{v}_i$ and $\mathbf{v} := \sum_{i=1}^{n} \mathbf{v}_i \geq \mathbf{0}$. If we set $m = n \cdot \max\{\|\mathbf{v}_i\| : 1 \leq i \leq n\}$ then $\sum_{i=1}^{k} \mathbf{v}_i \geq (-m, \ldots, -m)$ for all $1 \leq k \leq n$. By the property of the Karp-Miller graph there exists a reachable configuration $(q, \mathbf{u}')$ in V where $\mathbf{u}' \geq \mathbf{u}^{(m)}$. Then $\mathbf{u}' + \sum_{i=1}^{k} \mathbf{v}_i \geq \mathbf{0}$ for all $1 \leq k \leq n$ and

$$(q_0, \mathbf{0}) \xrightarrow{*} (q, \mathbf{u}') \xrightarrow{\tau_1} \ldots \xrightarrow{\tau_n} (q, \mathbf{u}' + \mathbf{v})$$

is indeed a self-covering run on V. $\square$

Recurrent state reachability in VASS

Theorem

The recurrent state reachability problem for VASS is decidable.

Proof.

We view the Karp-Miller graph of the given d-VASS V as a $\mathbb{Z}$-VASS V_{KM}:

- The states of V_{KM} are the nodes in the graph.
- Its transitions are the labels of the edges in the graph.

We need to test whether there is a run from $(q, \mathbf{0})$ to $(q, \mathbf{u})$ of length ≥ 1 where $\mathbf{u} \in \mathbb{N}^d$.
We proved that the reachability set of a $\mathbb{Z}$-VASS is effectively Presburger definable. By a simple modification of the proof one can show that $\{\mathbf{u} \in \mathbb{Z}^d \mid (q, \mathbf{0}) \xrightarrow{+} (q, \mathbf{u})\}$ is effectively Presburger definable by a formula $\varphi(\mathbf{x})$.
Finally we test whether $\exists \mathbf{x}(\mathbf{x} \geq \mathbf{0} \wedge \varphi(\mathbf{x}))$ is true. $\square$

Part VI

Pushdown systems

Section 23

Pushdown systems

Pushdown systems

Definition

A *pushdown system* $P = (Q, \Gamma, T, q_0)$ consists of

- a finite set of states Q,
- a finite stack alphabet Γ,
- an initial state $q_0 \in Q$,
- a set of transitions $T \subseteq Q \times (\Gamma \cup \bar{\Gamma} \cup \{\varepsilon\}) \times Q$.

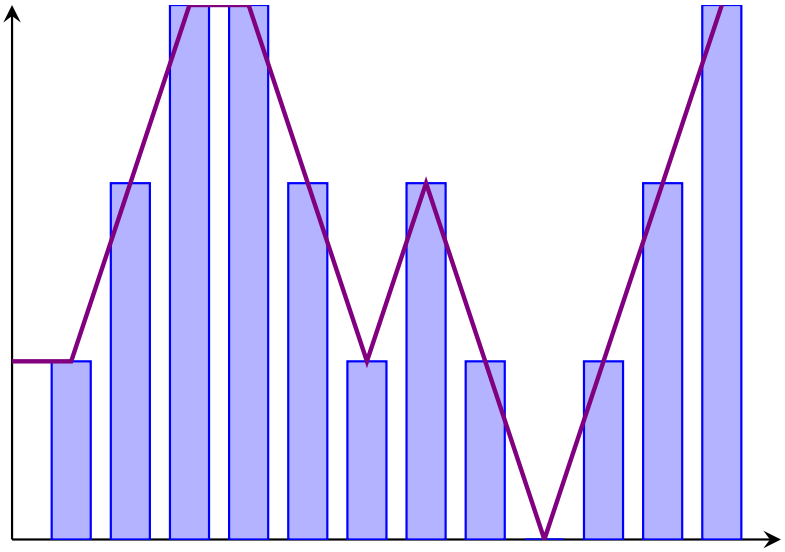
We write configurations $(q, w) \in Q \times \Gamma^*$ also as a word $qw \in Q\Gamma^*$.

Given configurations γ_0, γ_n and $w \in (\Gamma \cup \bar{\Gamma})^*$. We write $\gamma_0 \xrightarrow{w}_P \gamma_n$ if there exists a run $\gamma_0 \xrightarrow{\tau_1}_P \gamma_1 \xrightarrow{\tau_2}_P \ldots \xrightarrow{\tau_n}_P \gamma_n$ such that $w = \alpha_1 \ldots \alpha_n$ where $\tau_i = (q_{i-1}, \alpha_i, q_i)$ for all $1 \leq i \leq n$.
We will show:

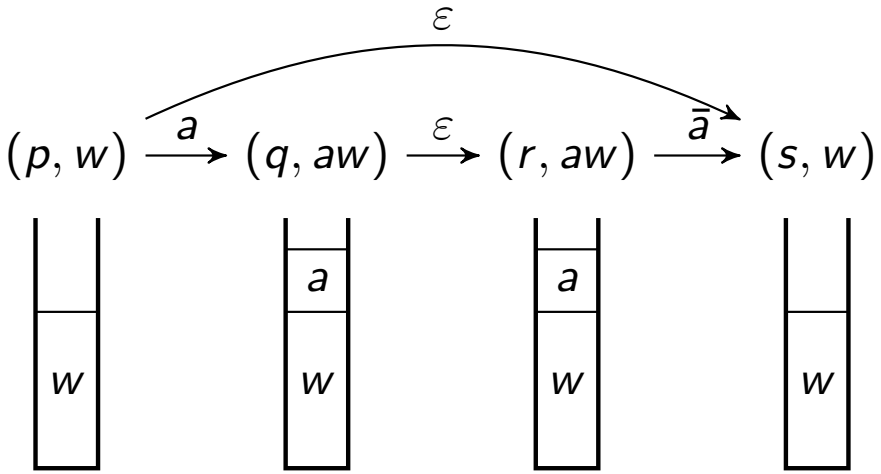
- The reachability problem is decidable in polynomial time.

Saturating a PDS: Idea

Consider the following run of the PDS:



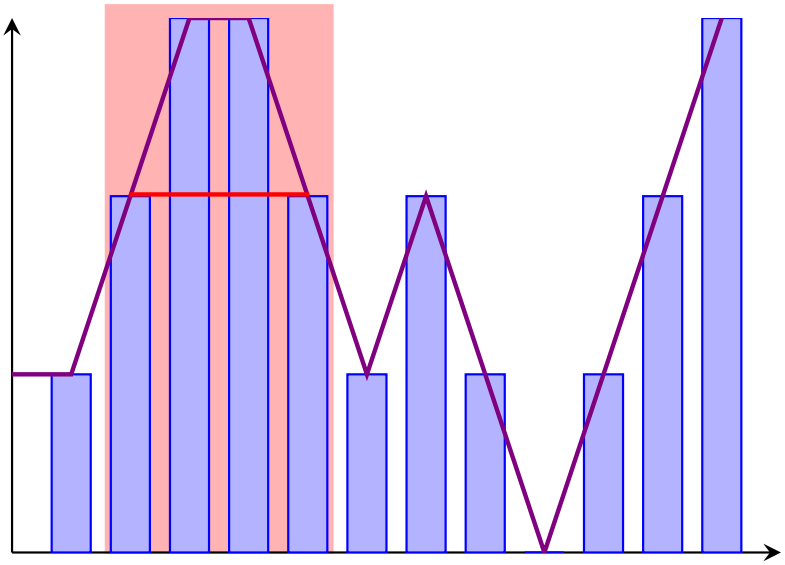
We add transitions to the PDS, preserving the reachability relation:



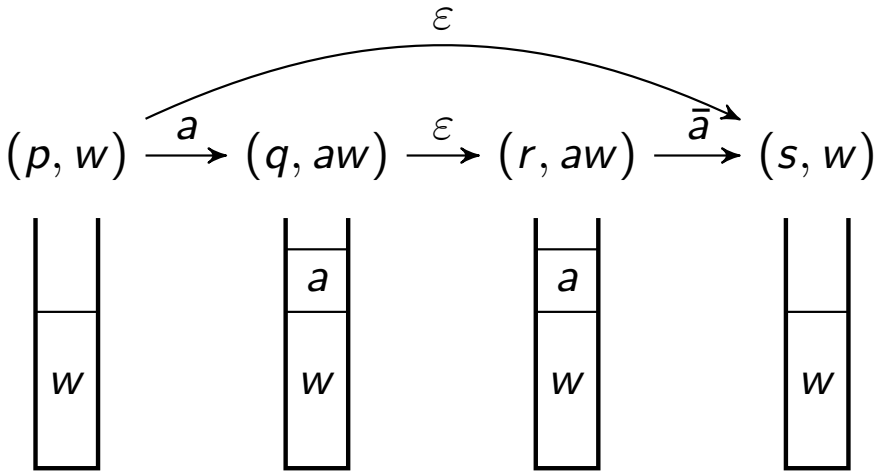
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



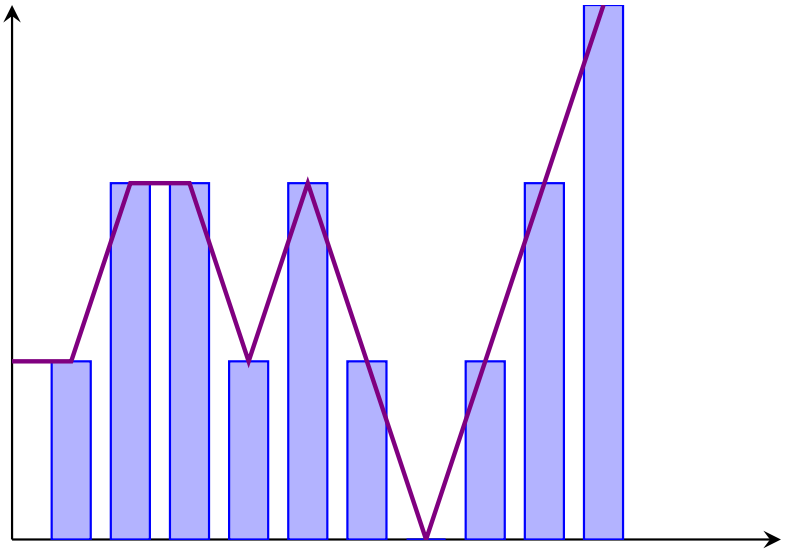
We add transitions to the PDS, preserving the reachability relation:



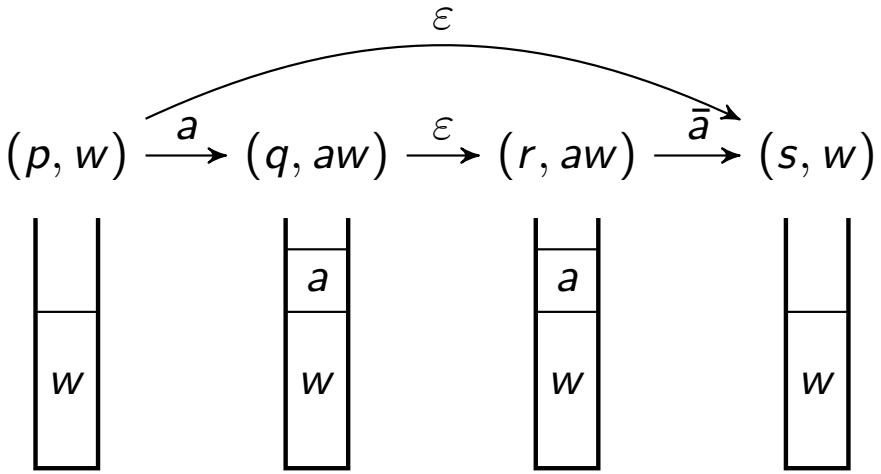
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



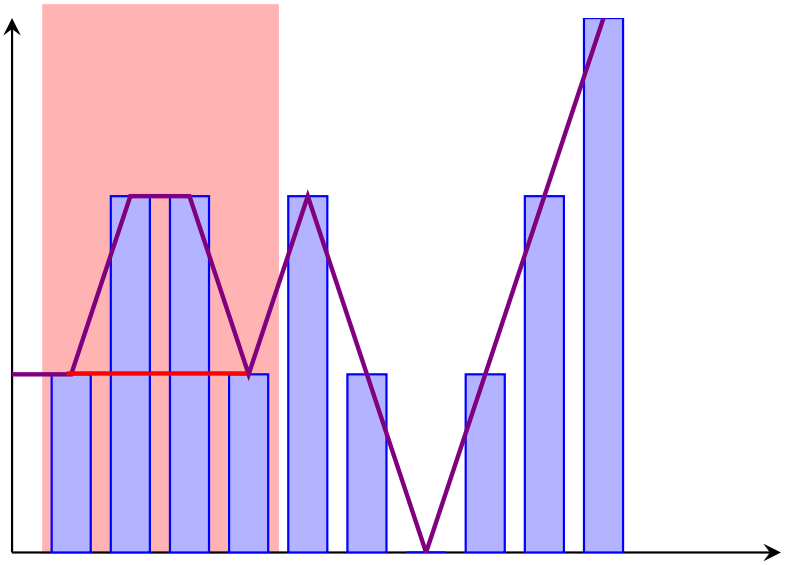
We add transitions to the PDS, preserving the reachability relation:



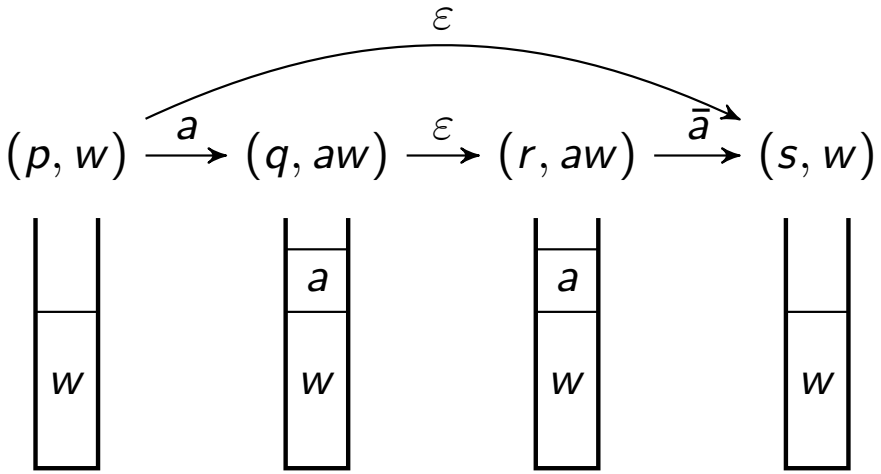
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



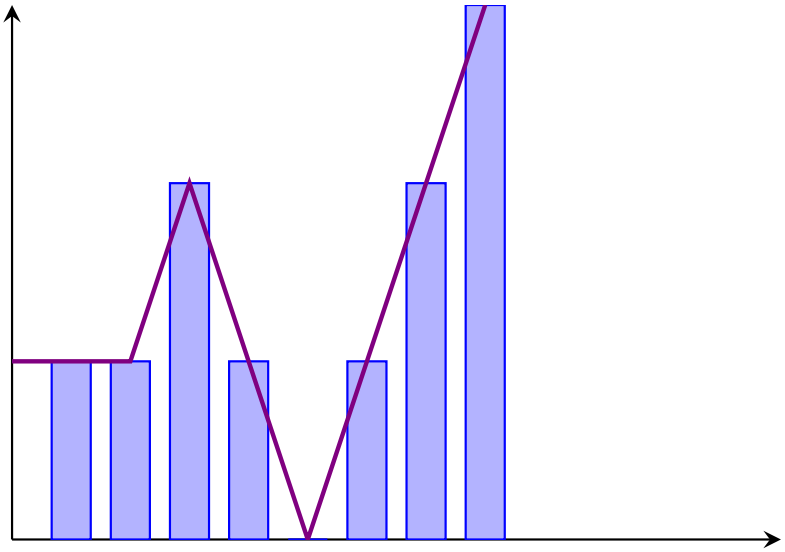
We add transitions to the PDS, preserving the reachability relation:



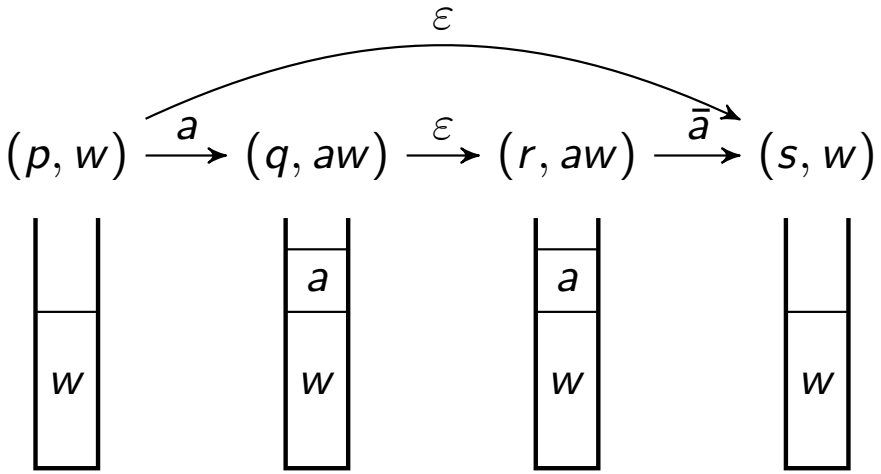
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



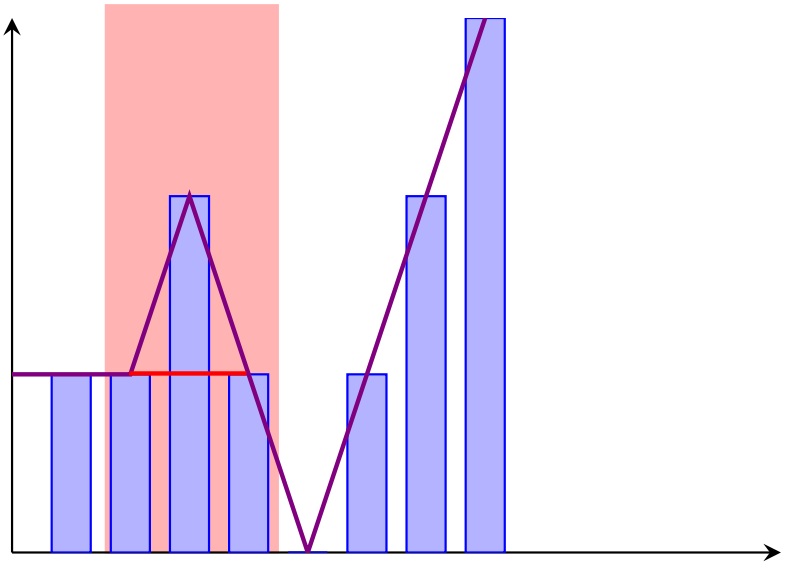
We add transitions to the PDS, preserving the reachability relation:



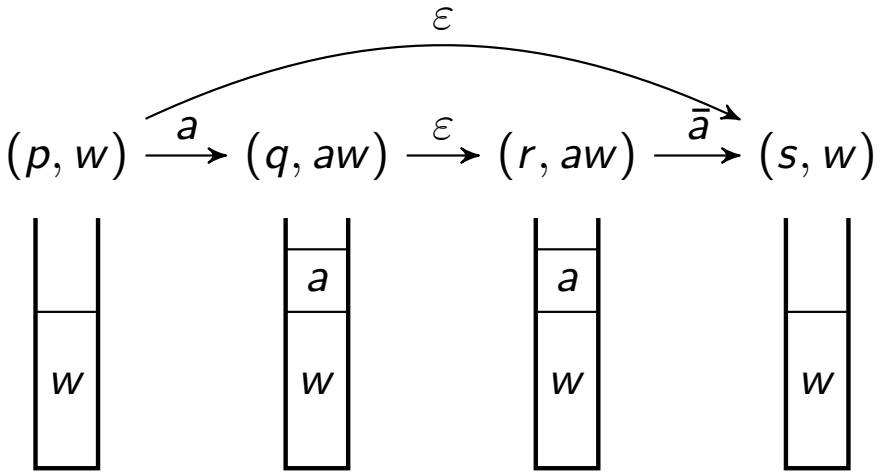
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



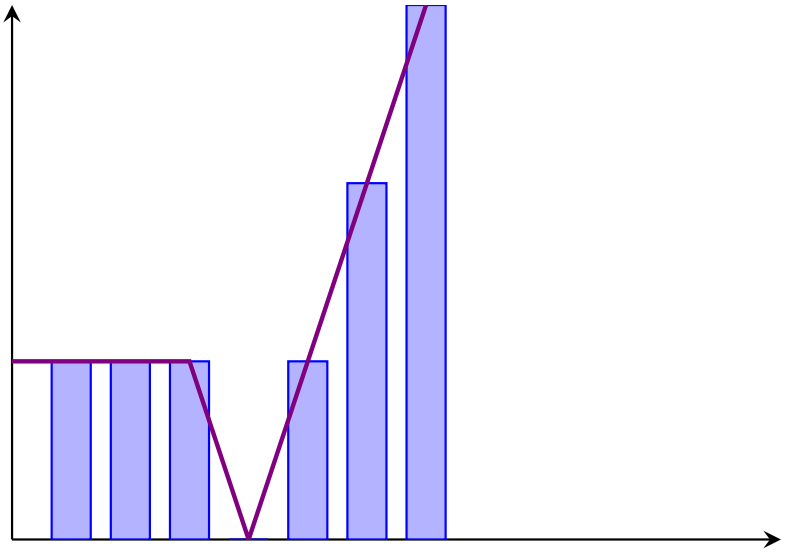
We add transitions to the PDS, preserving the reachability relation:



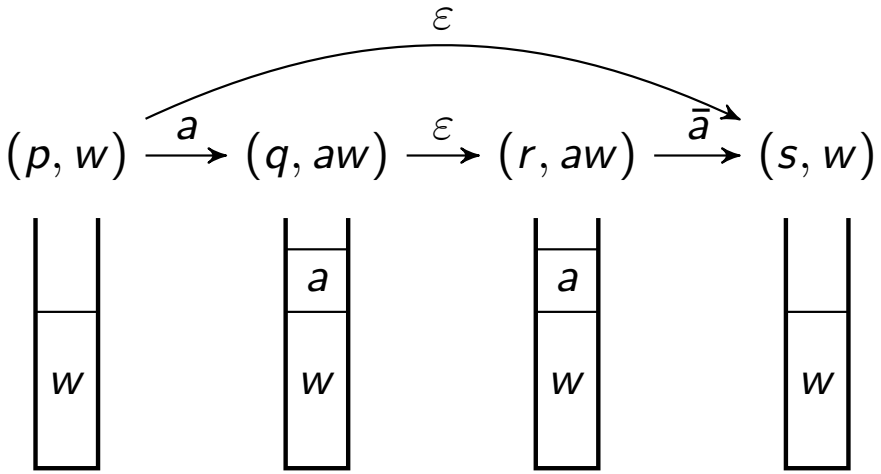
$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Idea

Consider the following run of the PDS:



We add transitions to the PDS, preserving the reachability relation:



$\rightarrow$ for each run $\gamma_0 \rightarrow^* \gamma_f$ in the PDS, we find a "simple" run $\gamma_0 \rightarrow^* \gamma_f$ in the saturated PDS

Saturating a PDS: Algorithm

Saturation method: Given $P = (Q, \Gamma, T, q_0)$

while there exist $(p, a, q), (r, \bar{a}, s) \in T$
with $q \xrightarrow{\varepsilon}_P r$ and $(p, \varepsilon, s) \notin T$ **do**
    add (p, ε, s) to T

Complexity

- $q \xrightarrow{\varepsilon}_P r$ can be tested in linear time (reachability in a finite directed graph).
- The while-condition can be tested in polynomial time.
- Every iteration adds a new transition (p, ε, s) to T
  $\rightarrow$ at most $|Q|^2$ iterations.

Let $\hat{P} = (Q, \Gamma, \hat{T}, q_0)$ be the resulting *saturated* pushdown system with $\hat{T} \supseteq T$.

Lemma (Correctness)

$\gamma \xrightarrow{*}_P \gamma'$ if and only if $\gamma \xrightarrow{*}_{\hat{P}} \gamma'$

Proof.

Let $P_i = (Q, \Gamma, T_i, q_0)$ be the pushdown system after i iterations.
If $\gamma \xrightarrow{*}_{P_{i-1}} \gamma'$ then $\gamma \xrightarrow{*}_{P_i} \gamma'$.
Conversely, consider a run π from γ to γ' in P_i. Assume that $(p, a, q), (r, \bar{a}, s) \in T_{i-1}$, $q \xrightarrow{\varepsilon}_{P_{i-1}} r$ and the transition (p, ε, s) is added to T_i in the i-th iteration.
We replace in π every step $\gamma_1 \xrightarrow{(p,\varepsilon,s)}_{P_i} \gamma_2$ by a run $\gamma_1 \xrightarrow{a\bar{a}}_{P_{i-1}} \gamma_2$.
This yields a run from γ to γ' in P_{i-1}. $\square$

Lemma

If $\gamma \xrightarrow{ua\bar{a}v}_{\hat{P}} \gamma'$ *then* $\gamma \xrightarrow{uv}_{\hat{P}} \gamma'$, *for all* $u, v \in (\Gamma \cup \bar{\Gamma})^*$, $a \in \Gamma$.

Proof.

$\hat{P}$ has the following property: If $(p, a, q), (r, \bar{a}, s) \in \hat{T}$ and $q \xrightarrow{\varepsilon}_{\hat{P}} r$ then $(p, \varepsilon, s) \in \hat{T}$. Hence any run

$$\gamma \xrightarrow{u}_{\hat{P}} (p, w) \xrightarrow{a}_{\hat{P}} (q, aw) \xrightarrow{\varepsilon}_{\hat{P}} (r, aw) \xrightarrow{\bar{a}}_{\hat{P}} (s, w) \xrightarrow{v}_{\hat{P}} \gamma'$$

can be shortened to

$$\gamma \xrightarrow{u}_{\hat{P}} (p, w) \xrightarrow{\varepsilon}_{\hat{P}} (s, w) \xrightarrow{v}_{\hat{P}} \gamma'. \qquad \square$$

Theorem

Given a pushdown system $P = (Q, \Gamma, T, q_0)$, one can compute in polynomial time a pushdown system $\hat{P} = (Q, \Gamma, \hat{T}, q_0)$ with $\hat{T} \supseteq T$ such that for all configurations γ, γ' we have:

$$\gamma \xrightarrow{*}_P \gamma' \iff \text{there exists } w \in (\bar{\Gamma})^*\Gamma^* \text{ such that } \gamma \xrightarrow{w}_{\hat{P}} \gamma'.$$

Proof.

The "$\Leftarrow$"-direction follows from the correctness lemma.
For "$\Rightarrow$" we show that $\gamma \xrightarrow{w}_{\hat{P}} \gamma'$ implies $\gamma \xrightarrow{\hat{w}}_{\hat{P}} \gamma'$ for some $\hat{w} \in (\bar{\Gamma})^*\Gamma^*$.
Induction on $|w|$:

- If $w = \varepsilon$ then $w \in (\bar{\Gamma})^*\Gamma^*$.
- If $w \notin (\bar{\Gamma})^*\Gamma^*$ then $w = w_1 a \bar{b} w_2$ for some $a, b \in \Gamma$.
  Clearly we must have $a = b$.
  By the previous lemma we know $\gamma \xrightarrow{w_1 w_2}_{\hat{P}} \gamma'$, and by induction we obtain $\gamma \xrightarrow{\hat{w}}_{\hat{P}} \gamma'$ with $\hat{w} \in (\bar{\Gamma})^*\Gamma^*$. $\square$

Theorem

The reachability problem for pushdown systems is decidable in polynomial time.

Proof.

Given a pushdown system P. We can assume that the target configuration is of the form (q, ε): If the target configuration is (q, w), extend P by a sequence of popping transitions to a new state q_f, so that (q, w) is reachable if and only if (q_f, ε) is reachable.

First we compute the saturated pushdown system $\hat{P}$ in polynomial time. To test whether $(q_0, \varepsilon) \xrightarrow{*}_P (q, \varepsilon)$, it suffices to check whether $q_0 \xrightarrow{\varepsilon}_{\hat{P}} q$, which can be decided in linear time. $\square$

Part VII

The reachability problem for VASS

Section 24

The reachability problem for VASS

Reachability for VASS

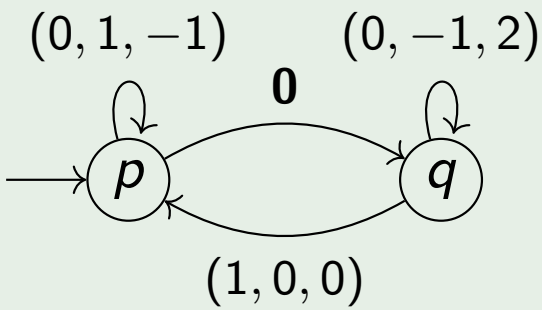
Short history of the reachability problem for VASS:

- 1976 Lipton: EXPSPACE-hard
- 1981 Mayr: decidable
- 1982 Kosaraju: decidable
- 1992 Lambert: decidable
- 2015 Leroux, Schmitz: "cubic Ackermann" upper bound
- 2019 Leroux, Schmitz: Ackermann upper bound
- 2019 Czerwiński, Lasota, Lazić, Leroux, Mazowiecki: non-elementary lower bound
- 2021 Czerwiński, Orlikowski; Leroux: Ackermann lower bound

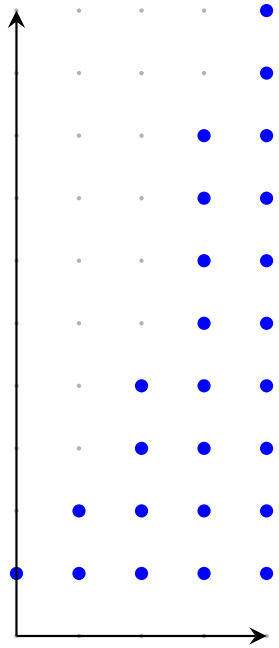
Reachability for VASS

In contrast to $\mathbb{Z}$-VASS, reachability sets of VASS are in general *not semilinear*.

Example (Hopcroft, Pansiot 1979)



$$\text{post}^*(p,(0,0,1)) = \{(p,(x,y,z)) \mid 1 \leq y + z \leq 2^x\} \cup \{(q,(x,y,z)) \mid 1 \leq 2y + z \leq 2^{x+1}\}$$



Projection to $z = 0$
(in state p)

Remark: In dimension two, reachability sets are still semilinear.

A simple idea to decide reachability for VASS is to run two *semi-decision procedures* in parallel:

- The first algorithm enumerates all runs and accepts if it finds a run that reaches the target configuration.
- The second algorithm tries to prove that the target configuration is not reachable.

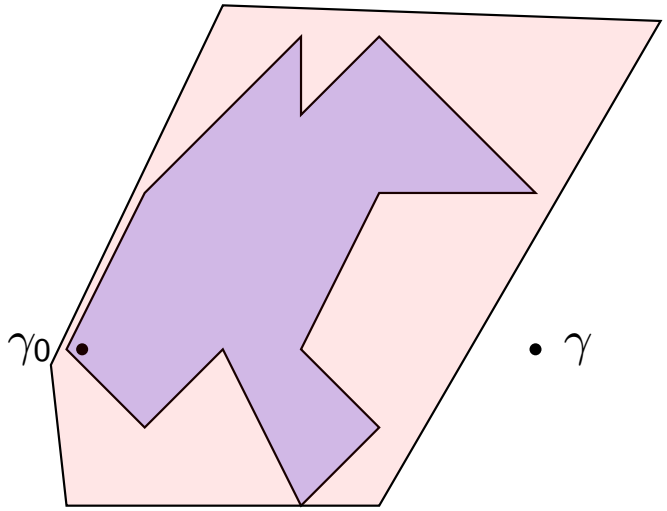
What is a witness for non-reachability?

Definition

Let $V = (Q, T, q_0)$ be a d-VASS. A set $I \subseteq Q \times \mathbb{N}^d$ is an *inductive invariant* if $\gamma \in I$ implies $\mathrm{post}(\gamma) \subseteq I$.

If an inductive invariant I contains the initial configuration $(q_0, \mathbf{0})$ then $\mathrm{Reach}(V) \subseteq I$, i.e. I is an overapproximation of the reachability set.
$\rightarrow$ If $\gamma \notin I$ then γ is certainly not reachable.

Of course Reach(V) itself is an inductive invariant.



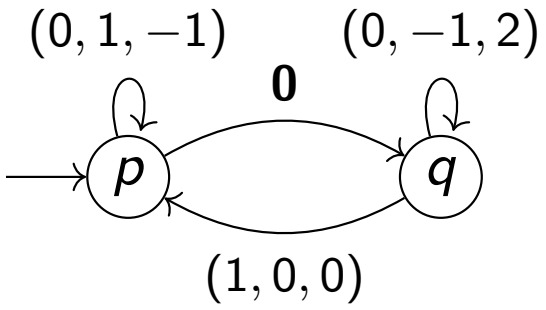
A set of configurations $I \subseteq Q \times \mathbb{N}^d$ is *Presburger definable* if there are Presburger formulas φ_q for each $q \in Q$ such that φ_q defines $\{\mathbf{u} \in \mathbb{N}^d \mid (q, \mathbf{u}) \in I\}$.

Theorem (Leroux 2011)

Let $V = (Q, T, q_0)$ be a d-VASS. If $(q, \mathbf{u})$ is not reachable then there exists Presburger definable inductive invariant $I \subseteq Q \times \mathbb{N}^d$ such that $(q_0, \mathbf{0}) \in I$ and $(q, \mathbf{u}) \notin I$.

(without proof)

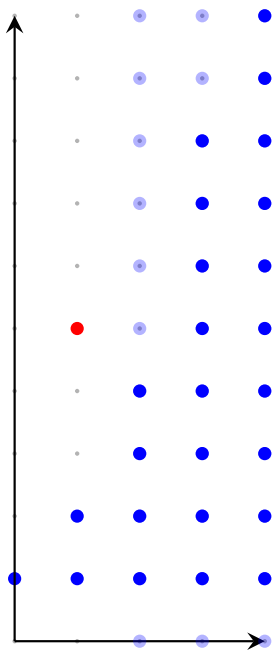
Example (inductive invariant)



If $(s,(a,b,c)) \notin \text{post}^*(p,(0,0,1))$ then

$$I_a = \{(p,(x,y,z)) \mid 1 \leq y+z \leq 2^x \text{ or } x > a\}$$
$$\cup \{(q,(x,y,z)) \mid 1 \leq 2y+z \leq 2^{x+1} \text{ or } x > a\}$$

is an inductive invariant containing $(p,(0,0,1))$ because no transition decreases the value of x.
Furthermore I_a is Presburger definable.



Projection to $z = 0$
(in state p)

Lemma

Given a d-VASS $V = (Q, T, q_0)$ and Presburger formulas $(\varphi_q(x_1, \ldots, x_d))_{q \in Q}$ defining a set $I \subseteq Q \times \mathbb{N}^d$, one can decide whether I is an inductive invariant.

Proof.

The following Presburger formula expresses that I is an inductive invariant:

$$\psi = \forall \mathbf{x} \bigwedge_{(q,\mathbf{v},q') \in T} (\varphi_q(\mathbf{x}) \wedge \mathbf{x} + \mathbf{v} \geq \mathbf{0}) \rightarrow \varphi_{q'}(\mathbf{x} + \mathbf{v}) \qquad \square$$

Therefore, we can enumerate all Presburger definable inductive invariants $I \subseteq Q \times \mathbb{N}^d$ by enumerating all $|Q|$-tuples of Presburger formulas and testing whether they define an inductive invariant.
If we find an inductive invariant I which contains the initial configuration but not the target configuration then we have proved non-reachability.

Decidability of the reachability problem for VASS

Given A d-VASS $V = (Q, T, q_0)$ and a conf. $(q_f, \mathbf{u})$.

Question Is $(q_0, \mathbf{0}) \xrightarrow{*} (q_f, \mathbf{u})$?

Semi-decision procedure for reachability

for all runs ρ in V **do**
  if ρ is a run to $(q_f, \mathbf{u})$ **then**
    return "yes".

Semi-decision procedure for non-reachability

for all sets $I \subseteq Q \times \mathbb{N}^d$ given by PA-formulas $(\varphi_q(\mathbf{x}))_{q \in Q}$ **do**
  if I is a inductive invariant and $\varphi_{q_0}(\mathbf{0}) \wedge \neg\varphi_{q_f}(\mathbf{u})$ is true **then**
    return "no".

One of the algorithms will return a correct answer. By running both algorithms in parallel, we obtain a decision procedure for the reachability problem for VASS.